

KakaoCloud Kubernetes Engine Intermediate Course

T6-03
Hands on
실습교재

kakaocloud

KakaoCloud Kubernetes Engine Intermediate Course

본 과정에서는 웹 서비스 (WS), 웹 애플리케이션 서버 (WAS), 데이터베이스(DB)를 컨테이너 형태로 구성하고 쿠버네티스 환경에서 운영하는 방법을 소개합니다.

교육은 쿠버네티스 클러스터 설정, 컨테이너 이미지 빌드, 배포, 모니터링, 보안 등을 다루며, KakaoCloud의 인프라와 서비스를 활용하여 MSA 웹 서비스를 최적화된 방식으로 개발 및 관리하는 데 필요한 핵심 기술과 노하우를 제공합니다.

본 교육에서는 카카오클라우드의 이해를 위한 이론 과정과 실습 과정을 포함하며 실습을 위한 카카오클라우드 계정을 제공합니다. 공유되는 계정은 본 교육 과정의 교육 용도로만 사용해야 합니다.

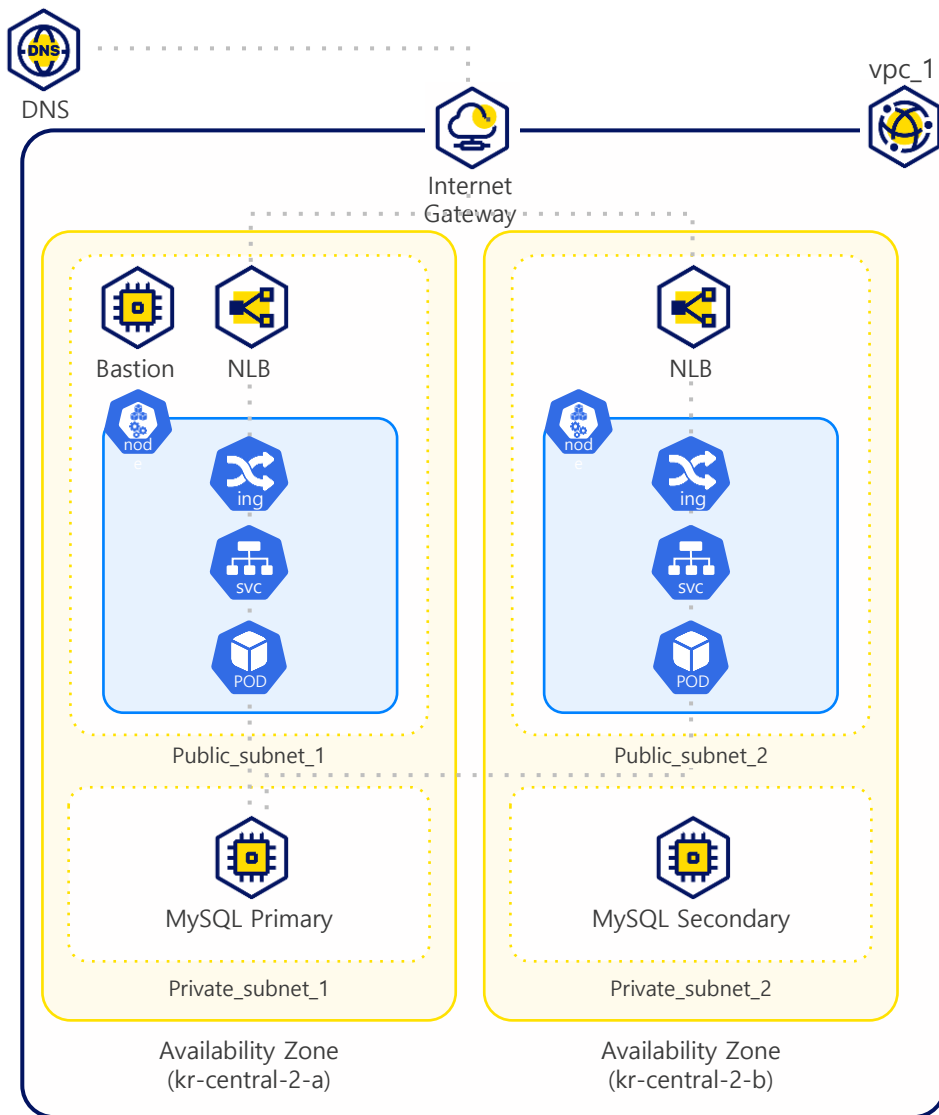
CONTENTS

본 교재는 카카오클라우드 심화 교육을 위한 실습 교재이며, 총 11개의 실습 중 Demo(시연)가 포함될 수 있습니다. 실습 과정은 카카오클라우드 환경에서의 쿠버네티스 클러스터 사용을 위한 클러스터 구축 및 실습 과정을 다룹니다.

본 실습에서 사용되는 명령어는 깃허브에서 확인할 수 있습니다. [\(Github Link\)](#)

Lab1 : 기본 환경구성	4
Lab2 : Kubernetes Engine Cluster 생성	13
Lab3 : Kubernetes Engine Cluster 관리 VM 생성 실습	21
Lab4 : Container Image 만들기	43
Lab5 : 인그레스 컨트롤러 배포	52
Lab6 : Kubernetes Engine 클러스터에 웹서버 수동 배포	59
Lab7 : Kubernetes Engine 활용하기	73
Lab8 : Kubernetes Engine 클러스터에 웹서버 자동화 배포	82
Lab9 : HPA (Horizontal Pod Autoscaler) 테스트	97
Lab10 : DNS 서비스를 통해 도메인 주소 연결	102
Lab11 : 리소스 삭제	107

실습을 통해 카카오클라우드에 구축할 아키텍처는 아래와 같습니다.



Lab1:

기본 환경 구성

이론 교재 7p

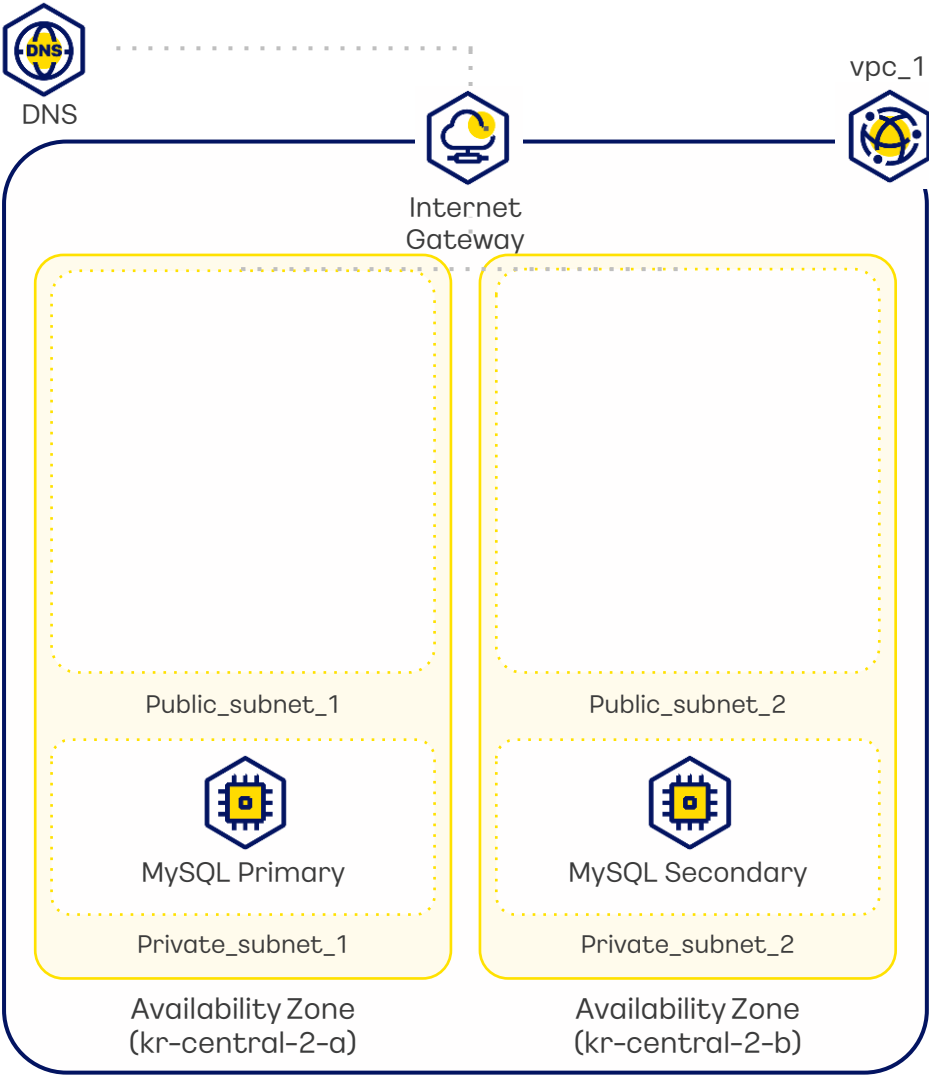


실습 내용

클러스터 구축을 위한 프로젝트 기본 환경 실습을 진행합니다. VPC, MySQL, IAM 액세스 키를 생성합니다.

실습 순서

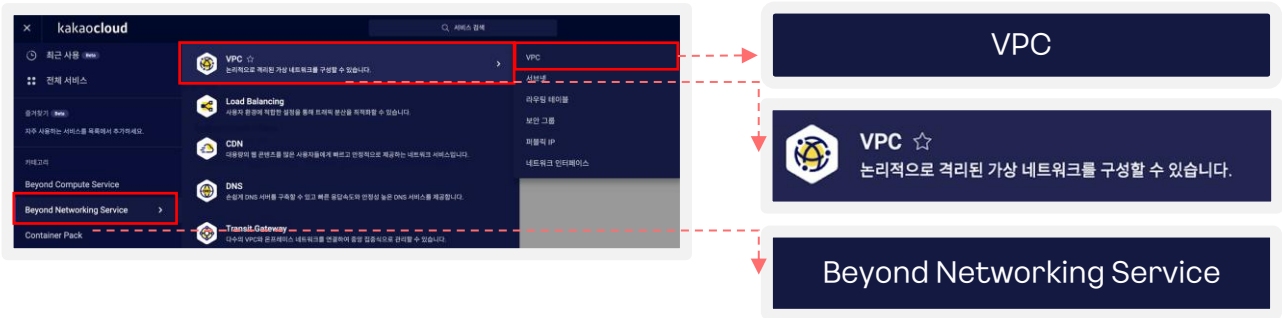
- 1 VPC 생성
- 2 MySQL 인스턴스 그룹 생성
- 3 IAM 액세스 키 생성





VPC 생성

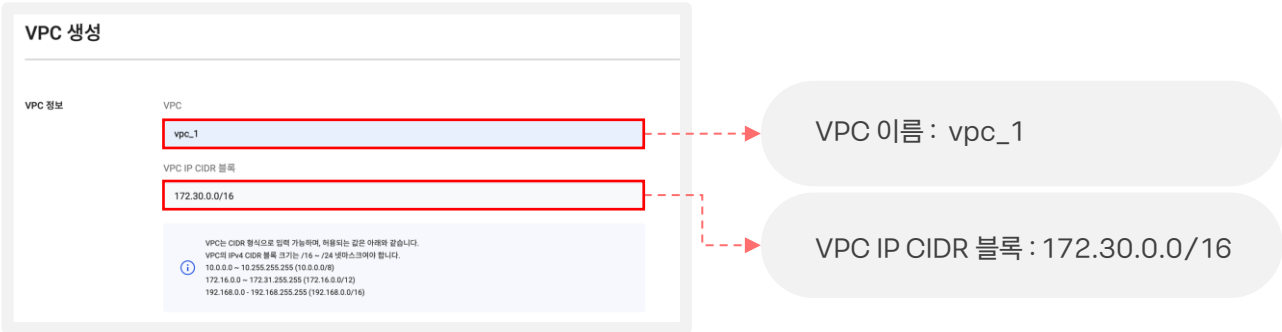
1. 카카오클라우드 로고 옆  버튼 클릭> Beyond Networking Service > VPC



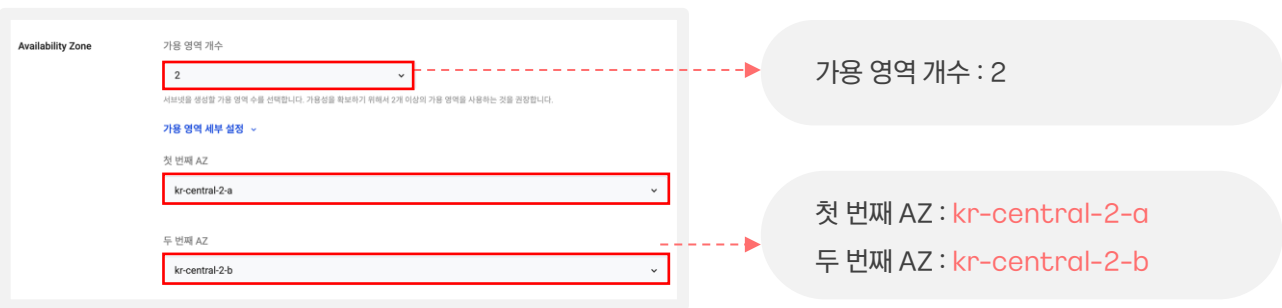
2. VPC탭 > [VPC 생성] 클릭



3. VPC 정보 작성



4. Availability Zone 정보 작성





5. 서브넷 설정 정보 작성

서브넷 설정

가용 영역당 퍼블릭 서브넷 개수 1

가용 영역당 프라이빗 서브넷 개수 1

kr-central-2-a

퍼블릭 서브넷 IPv4 CIDR 블록 172.30.0.0/20

프라이빗 서브넷 IPv4 CIDR 블록 172.30.16.0/20

kr-central-2-b

퍼블릭 서브넷 IPv4 CIDR 블록 172.30.32.0/20

프라이빗 서브넷 IPv4 CIDR 블록 172.30.48.0/20

가용 영역당 퍼블릭 서브넷 개수 : 1

가용 영역당 프라이빗 서브넷개수 : 1

kr-central2-a의 퍼블릭 서브넷 IPv4 CIDR 블록 : 172.30.0.0/20

kr-central2-a의 프라이빗 서브넷 IPv4 CIDR 블록 : 172.30.16.0/20

kr-central2-b의 퍼블릭 서브넷 IPv4 CIDR 블록 : 172.30.32.0/20

kr-central2-b의 프라이빗 서브넷 IPv4 CIDR 블록 : 172.30.48.0/20

6. 구성도 확인 후 [생성] 클릭

VPC

Subnet (4)

Route Table (2)

Network Connection (1)

생성

클릭

7. VPC 생성 상태 확인 (생성 약 4~5분 소요)

VPC

VPC 생성

VPC ID

상태

IP CIDR 블록

기본 VPC

VPC 소유자 프로젝트 ID

Pending Create

172.30.0.0/20

172.30.16.0/20

172.30.32.0/20

172.30.48.0/20

5683763113014680844800

상태 값 Active 확인

- Active: 활성화(생성완료)

- Creating: 생성 중

- Updating: 업데이트 중

- Deleting: 삭제 중

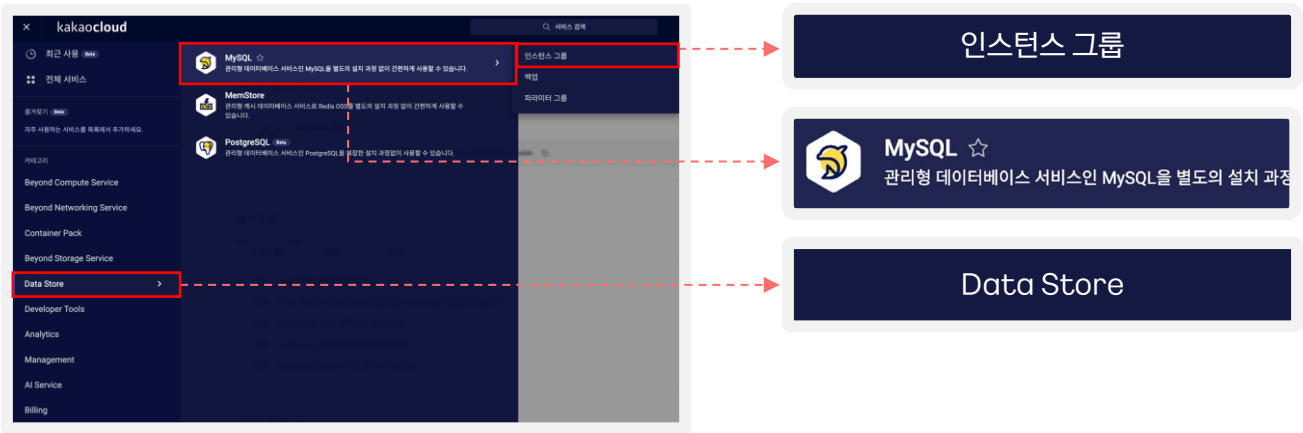
- Error: 에러 발생

실습 순서

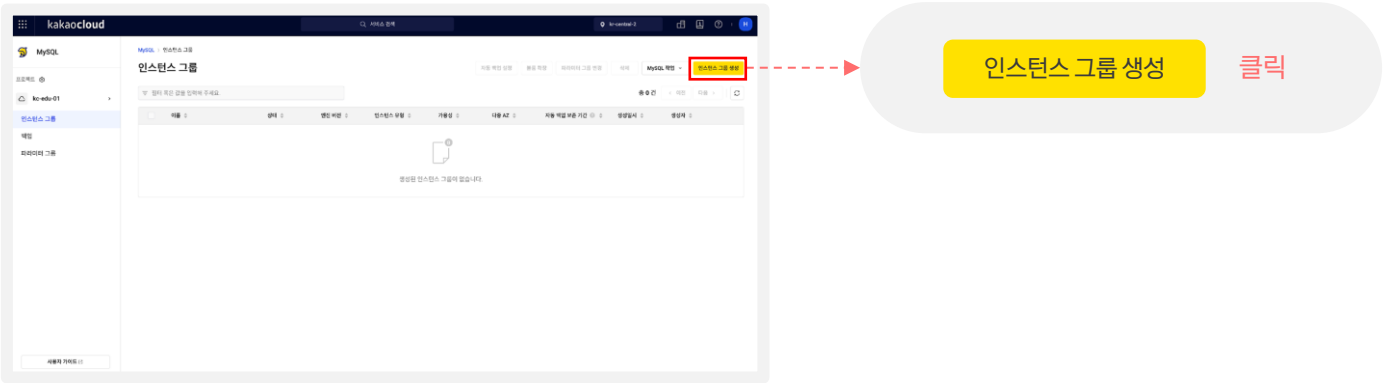


MySQL 인스턴스 그룹 생성

1. 카카오클라우드 로고 옆 버튼 클릭> Data Store > MySQL > 인스턴스 그룹



2. [인스턴스 그룹 생성] 클릭



실습 순서



MySQL 인스턴스 그룹 생성

MySQL > 인스턴스 그룹 > 인스턴스 그룹 생성

인스턴스 그룹 생성

기본 설정

인스턴스 그룹 이름

설명 (선택)

인스턴스 가용성

☒ 고가용성 (Primary, Standby 인스턴스)
Primary 인스턴스와 다수의 Standby 인스턴스를 생성할 수 있습니다. 단일 AZ 또는 다중 AZ를 이용할 수 있습니다.

☐ 단일 (Primary 인스턴스)
단일 Primary 인스턴스만 생성합니다. 단일 AZ만 이용할 수 있습니다.

MySQL 설정

엔진 버전

Primary 포트

Standby 포트

사용자 이름

비밀번호 [비밀번호 자동 생성](#) [복사](#)

파라미터 그룹 [관리 페이지](#)

인스턴스 유형

조건 검색

유형 2 vCPU | 8 GiB Memory | kr-central-2-a, kr-central-2-b, kr-central-2-c

가용 스토리지

유형/크기

최대 IOPS

로그 스토리지

유형/크기

최대 IOPS

- 이름 : database
- 인스턴스 가용성 : 고가용성(Primary, Standby 인스턴스)
- 엔진 버전 : MySQL 8.0.41
- Primary 포트 : 3306
- Standby 포트 : 3307
- 이름 : admin
- 비밀번호 : admin1234
- 파라미터 그룹 : 기본 설정 값
- 인스턴스 유형: m2a.large
- 기본 스토리지 크기 : 100GB
- 로그 스토리지 크기 : 100GB

다음페이지(이어서) >>

실습 순서

- 1
VPC
생성
- 2
MySQL 인스턴스
그룹 생성
- 3
IAM 액세스
키 생성

MySQL 인스턴스 그룹 생성

네트워크 설정

다중 AZ 옵션

VPC

서브넷 및 인스턴스 개수 설정

vpc_1

인스턴스 개수 (2개)

인스턴스 개수 (2개)

보안 그룹

선택된 항목 (0)

+ 새 보안 그룹 생성

보안 그룹 생성

보안 그룹 이름

보안 그룹 설명 (선택)

적용된 정책

인바운드

아웃바운드

생성

자동 백업

테이블 대소문자 구분

완료

생성

MySQL 인스턴스 그룹

인스턴스 그룹

데이터베이스

Available

◦ vpc_1 선택

◦ AZ-a의 Private 서브넷 선택 (172.30.16.0/20)

◦ 인스턴스 개수 : 1

◦ AZ-b의 Private 서브넷 선택 (172.30.48.0/20)

◦ 인스턴스 개수 : 1

◦ 선택된 항목 클릭

◦ 새 보안 그룹 생성 클릭

◦ 보안 그룹 이름 : mysql

◦ 보안 그룹 정책 입력

방향	프로토콜	패킷출발지	포트번호
인바운드	TCP	0.0.0.0/0	3306
인바운드	TCP	0.0.0.0/0	3307
아웃바운드	TCP	0.0.0.0/0	ALL

◦ 생성 클릭

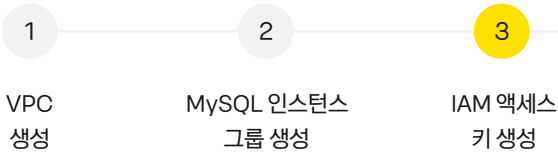
◦ 자동 백업 옵션 : 미사용

◦ 테이블 대소문자 구분 : 사용

◦ 생성 클릭

Available

실습 순서



IAM 액세스 키 생성

1. 콘솔 우측 사용자 아이콘 클릭 > 자격 증명

아이콘 클릭

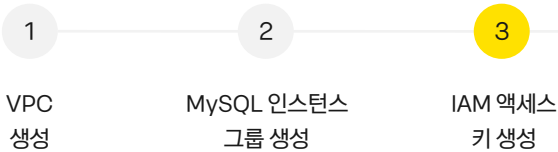
자격 증명 클릭

2. 비밀번호 재확인 > 아이디/비밀번호 입력 후 > 비밀번호 확인

3. IAM 액세스 키 탭 > [IAM 액세스 키 생성] 클릭

IAM 액세스 키 생성 클릭

실습 순서



IAM 액세스 키 생성

IAM 액세스 키

IAM 액세스 키 생성

사용자 자격 증명인 IAM 액세스 키를 통해 카카오클라우드 API 인증 토큰을 발급받을 수 있습니다.
IAM 액세스 키는 발급 시 선택한 프로젝트의 사용자 역할 권한을 상속받습니다.
단, 사용자 역할이 변경되면, 역할 변경 이전에 발급된 액세스 키는 사용자 역할과 일치하지 않아 정상적으로 동작하지 않을 수 있습니다.
IAM 액세스 키 정보는 생성 후 수정이 불가능하며, 계정당 프로젝트별로 최대 2개까지 생성할 수 있습니다.

1 프로젝트 지정

kakaocloud-class (kakaocloud-class)

2 IAM 액세스 키 이름

test-key

IAM 액세스 키 이름은 생성 후 수정할 수 없습니다.

3 IAM 액세스 키 정보 (선택)

test-key

IAM 액세스 키 설명은 생성 후 수정할 수 없습니다.

4 만료 기한

☐ 무제한

취소

생성

1 프로젝트 지정

- * IAM 액세스 키에 IAM 역할을 할당할 프로젝트 선택
- * 선택한 프로젝트의 역할(프로젝트 관리자 또는 프로젝트 멤버) 권한을 IAM 액세스 키에 부여함

2 IAM 액세스 키 이름

- * 공백 없이 알파벳 소문자(a-z), 숫자(0-9), 하이픈 (-)만 입력 가능(4~20자)
- * 다른 액세스 키와 구분할 수 있는 이름을 입력 필요
- * 중복된 이름 사용 불가

3 IAM 액세스 키 정보 (선택)

- * 액세스 키에 대한 부가 정보
- * IAM 액세스 키가 생성된 이후에는 수정 불가
- * 글자 수 : 4~30자

4 만료 기한

- * 입력한 일자의 23:59:59까지 액세스 키 사용 가능
- * Off 시 기간 없이 무제한 사용 가능

IAM 액세스 키 생성

IAM 액세스 키

- 복사  클립보드 등에 붙여 넣기 해두기

! 유의사항

IAM 액세스 키 생성 팝업창을 닫은 이후 IAM 액세스 보안 키 정보 다시 조회 불가

- 발급한 IAM 액세스 키를 콘솔에서 삭제할 경우 액세스 키는 자동 만료
- 액세스 키를 일시적으로 정지 불가

IAM 액세스 키 정보

i

IAM 액세스 키를 생성했습니다. 보안상 이유로 IAM 액세스 키 생성 이후에는 보안 액세스 키 정보를 확인할 수 없으므로, 보안 액세스 키를 복사하여 안전하게 보관해 주세요.

- 보안 액세스 키를 분실하신 경우, 기존 액세스 키는 삭제하고 새로운 액세스 키를 생성해 주세요.
- 프로젝트 삭제 시, IAM 액세스 키는 자동으로 만료됩니다.

IAM 액세스 키 ID

보안 액세스 키

확인

Lab2 : Kubernetes Engine Cluster 생성

이론 교재 69p

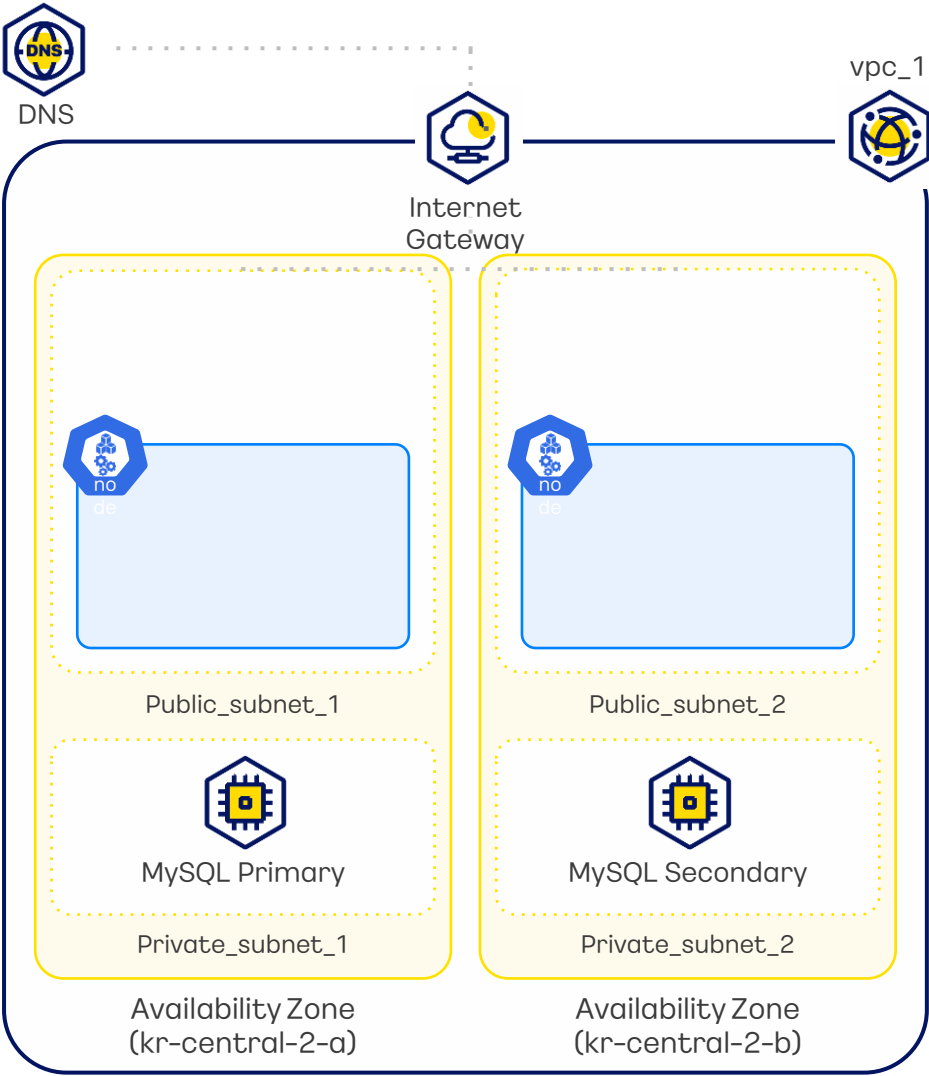


실습 내용

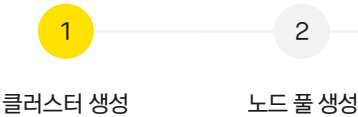
KakaoCloud Kubernetes Engine Cluster 생성에 대한 실습입니다.

실습 순서

- 1 클러스터 생성
- 2 노드 풀 생성

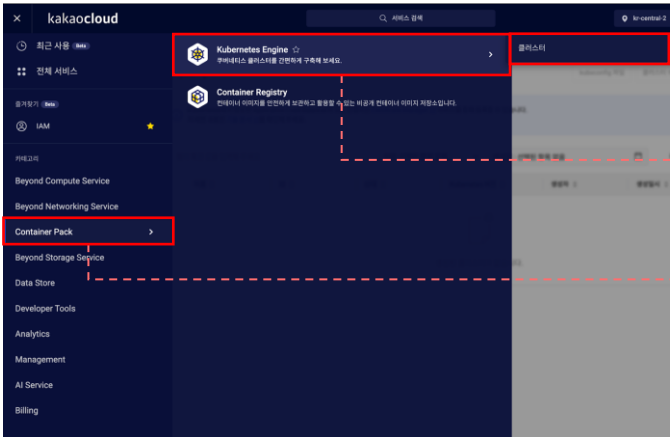


실습 순서

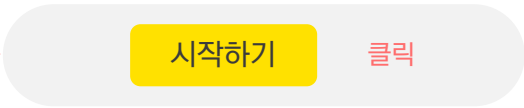


클러스터 생성

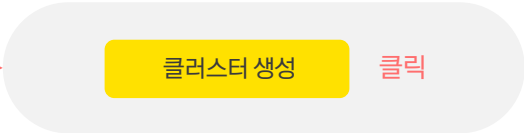
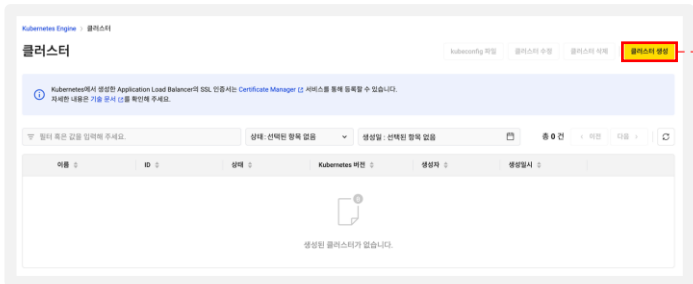
1. 카카오클라우드 로고 옆  버튼 클릭> Container Pack > Kubernetes Engine > **클러스터**



2. Cluster 서비스 시작하기



3. [클러스터 생성] 클릭



실습 순서

1

2

클러스터 생성

노드 풀 생성

4. 기본 설정 정보 작성

클러스터 생성

기본 설정

클러스터 이름

kakao-k8s-cluster

설명

60자 이내로 작성

Kubernetes 버전

1.30

클러스터 이름 :
kakao-k8s-cluster

Kubernetes 버전: 1.30

5. 클러스터 Network 설정

클러스터 네트워크 설정

VPC

vpc_1 (172.30.0.0/16)

서브넷

선택한 항목 (2)

kr-central-2-a (1), kr-central-2-b (1)

main (172.30.0.0/20) x vpc_1_79d45_sn_1 (172.30.32.0/20) x 전체 삭제

클러스터 엔드포인트 액세스

☒ 퍼블릭 엔드포인트

외부 인터넷에서 연결이 가능한 API 서버 엔드포인트를 제공합니다.

☐ 프라이빗 엔드포인트

VPC 내부 네트워크로 연결이 제한된 API 서버 엔드포인트를 제공합니다.

서브넷

main (172.30.0.0/20)

kr-central-2-a

클러스터의 서브넷은 최소 두 개 이상으로 설정되어야 하며, 각각은 서로 다른 가용 영역에 위치해야 합니다.

main (172.30.0.0/20) x 전체 삭제

vpc_1 선택

Public Subnet 2개 선택
(172.30.0.0/20, 172.30.32.0/20)

퍼블릭 엔드포인트 선택

! 유의사항
클러스터의 서브넷은 서로 다른 AZ로 최소 2개 이상의 서브넷이 설정 되어야함

6. 클러스터 Network 설정 후 [생성] 클릭

CNI

Calico

클러스터에 설치할 CNI를 직접 선택할 수 있습니다. 서비스와 파드 IP CIDR을 별도로 설정하지 않는다면 Calico CNI의 기본 설정 조건이 적용됩니다.

클러스터 생성 후에는 서비스 IP와 파드 IP를 변경할 수 없습니다.
서비스 IP와 파드 IP의 허용 조건은 아래와 같습니다.
• CIDR 형식이며, IPv4 RFC-1918로 규정된 사용 네트워크 범위 내에 존재해야 함
• 서비스 IP의 최소 허용 크기는 /28, 최대 허용 크기는 /12
• 파드 IP의 최소 허용 크기는 /24, 최대 허용 크기는 /16
• 서비스 IP와 파드 IP는 서로 중첩될 수 없음
• 클러스터 네트워크로 설정한 VPC 범위와 중첩될 수 없음
• Kubernetes Engine 내부에서 사용 중인 198.18.0.0/16과 중첩될 수 없음
• IPv4만 지원

서비스 IP CIDR 블록 (선택)

클러스터 서비스의 IP CIDR을 입력해 주세요.

파드 IP CIDR 블록 (선택)

클러스터 파드의 IP CIDR을 입력해 주세요.

취소 생성

Calico 선택

생성 클릭

실습 순서

1

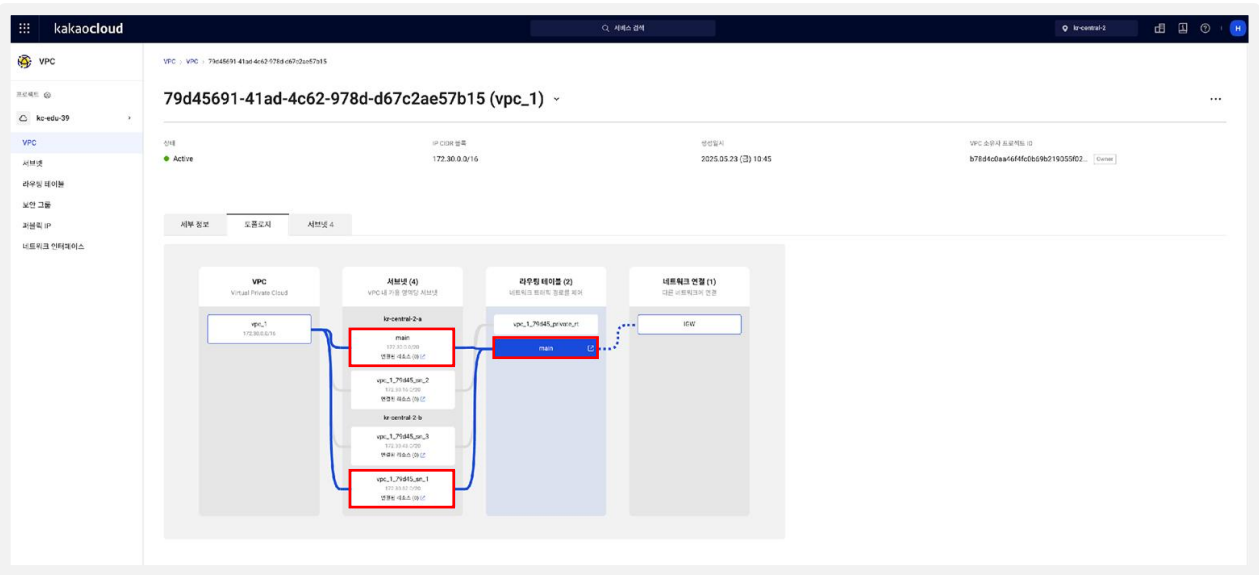
클러스터 생성

2

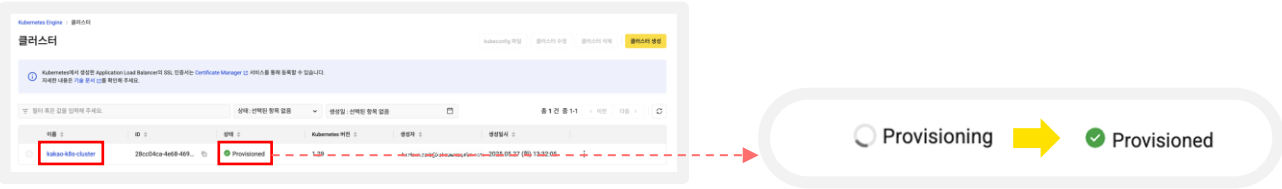
노드 풀 생성

* 잠깐 🍵 Tip!: Public Subnet 확인하는 방법

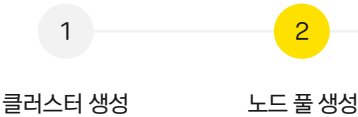
Public Subnet 확인하는 방법 :
전체 서비스 > VPC > vpc_1 클릭 > Topology 선택 > kr-central-2-a과 kr-central-b의
Public 서브넷 IP CIDR 블록 확인



7. 클러스터 생성 상태 확인 (생성 약 10~15분 소요)



실습 순서



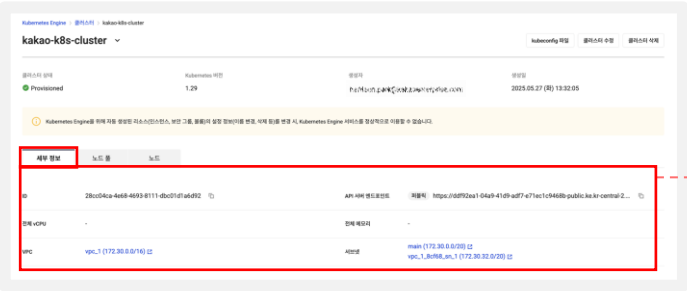
노드 풀 생성

1. 생성된 [kakao-k8s-cluster] 클릭



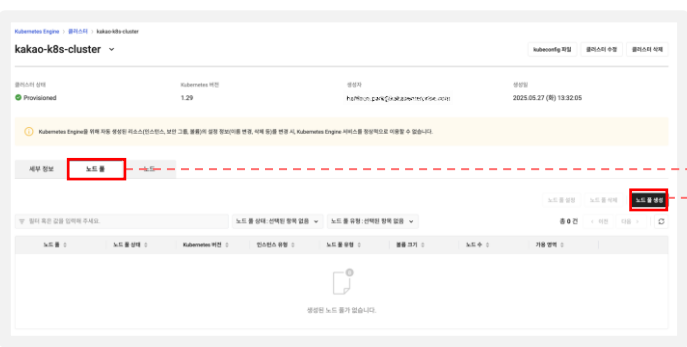
kakao-k8s-cluster 클릭

2. 생성된 [kakao-k8s-cluster] 세부 정보 확인



세부 정보 클릭

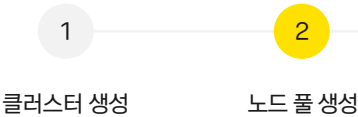
3. [노드 풀] > [노드 풀 생성] 클릭



노드 풀 클릭

노드 풀 생성 클릭

실습 순서



4. [Virtual Machine] 클릭

Kubernetes Engine > 클러스터 > kakao-k8s-cluster > 노드 풀 생성

노드 풀 생성

노드 풀 유형

Virtual Machine
워크 노드를 Virtual Machine 인스턴스에서 실행합니다.

GPU
워크 노드를 GPU 인스턴스에서 실행합니다.

Bare Metal Server
워크 노드를 Bare Metal 단독 서버에서 실행합니다.

Virtual Machine 선택

5. 기본 설정 정보 작성

기본 설정

노드 풀 이름

설명
60자 이내로 작성

이미지
필터 혹은 값을 입력해 주세요.
총 1 건 중 1-1 < 이전 다음 >

이름	아키텍처
Ubuntu 22.04 (VM)	x86_64

인스턴스 유형
> 조건 검색

비용 m2a.large
2 vCPU | 8 GiB Memory | kr-central-2-a, kr-central-2-b, kr-central-2-c

볼륨
SSD GB

노드 수

- 노드 풀 이름 : node-pool
- 이미지 : Ubuntu 22.04
- 인스턴스 타입 : m2a.large
- 볼륨 크기 : 50 GB
- 노드 수 : 2

6. 노드 풀 Network 설정

노드 풀 네트워크 설정

VPC

서브넷
인스턴스 유형을 먼저 선택해 주세요.
main (172.30.0.0/20) vpc_1_subnet_1 (172.30.32.0/20) 전체 서브넷

보안 그룹 (선택)
선택된 항목 (0)

- vpc_1
- public 서브넷들만 선택 (172.30.0.0/20, 172.30.32.0/20)

실습 순서



7. Key Pair 설정

키 페어

키 페어 선택

SSH 접근을 위한 키 페어를 설정합니다.

➡

◦ Key Pair 선택 누르기

➡

+ 신규 키 페어 생성

➡

◦ 신규 Key Pair 생성 누르기

➡

키 페어 생성

생성 방법

☒ 신규 키 페어 생성하기 ☐ 기존 키 업로드하기

이름

keypair

유형 설정

SSH

취소 생성

➡

◦ 이름 : `keypair`

➡

➡

클릭

➡

키 페어

keypair

SSH 접근을 위한 키 페어를 설정합니다.

➡

◦ 생성한 키페어 선택

8. 고급 설정

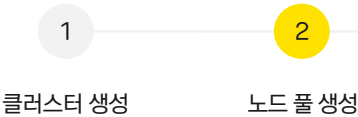
고급 설정

취소 생성

➡

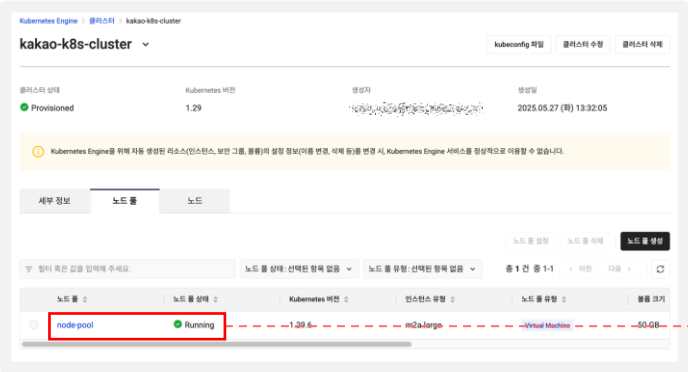
◦ 고급 설정 생략 후 생성 클릭

실습 순서



노드 풀 생성

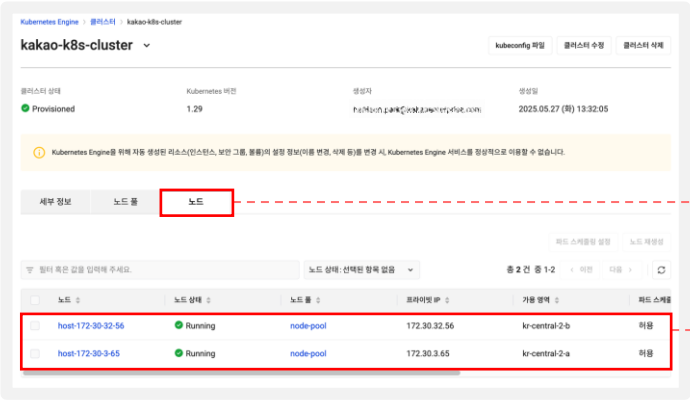
1. 노드 풀 생성 확인



○ 노드 풀 생성 확인

노드 목록 확인

1. [노드] 클릭 후 노드 목록 확인



노드 클릭

○ 노드 2개 생성 확인

Lab3 : Kubernetes Engine Cluster 관리 VM 생성 실습

이론 교재 70p



실습 내용

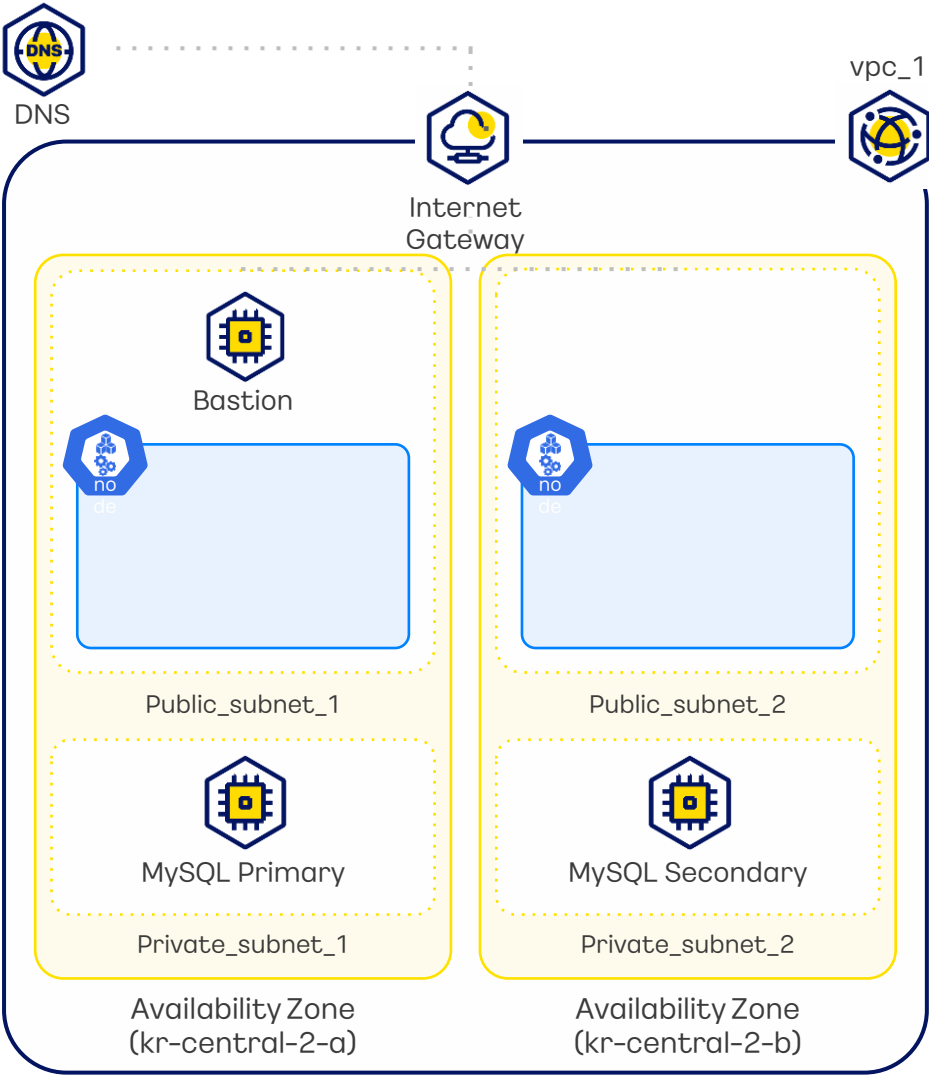
Kubernetes Engine Cluster를 관리하기 위한 Bastion VM 인스턴스를 생성합니다. VM 내부에 클러스터를 사용하기 위한 패키지들을 설치하고, 실습 환경을 구성합니다.

실습 순서

- 1

Bastion VM 인스턴스 생성
- 2

Bastion VM 인스턴스를 통해 클러스터 확인



실습 순서

1

2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성 VM 생성 전 스크립트 작성

1. 깃허브 lab3-1-6스크립트를 메모장에 붙여넣기

아래의 스크립트를 복사 후 고급 설정에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab03 link)



```
#!/bin/bash

echo "kakaocloud: 1.Starting environment variable setup"
# 환경 변수 설정: 사용자는 이 부분에 자신의 환경에 맞는 값을 입력해야 합니다.

command=$(cat <<EOF
export ACC_KEY='IAM 액세스 키 ID 입력'
export SEC_KEY='보안 액세스 키 입력'
export EMAIL_ADDRESS='사용자 이메일 입력'
export CLUSTER_NAME='클러스터 이름 입력'
export API_SERVER='클러스터의 API 엔드포인트 입력'
export AUTH_DATA='클러스터의 certificate-authority-data 입력'
export PROJECT_NAME='프로젝트 이름 입력'
.
.
.
)
```

◦ GitHub 사이트에서 전체 코드 확인

2. Lab1-4 에서 복사해둔 IAM 액세스 키 ID와 보안 액세스 키 메모장에 붙여넣기

IAM 액세스 키 정보

IAM 액세스 키를 생성했습니다. 보안상 이유로 IAM 액세스 키 생성 이후에는 보안 액세스 키 정보를 확인할 수 없으므로, 보안 액세스 키를 복사하여 안전하게 보관해 주세요.

- 보안 액세스 키를 분실하신 경우, 기존 액세스 키는 삭제하고 새로운 액세스 키를 생성해 주세요.
- 프로젝트 삭제 시, IAM 액세스 키는 자동으로 만료됩니다.

IAM 액세스 키 ID

보안 액세스 키

확인

실습 순서

- 1
- 2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성
VM 생성 전 스크립트 작성

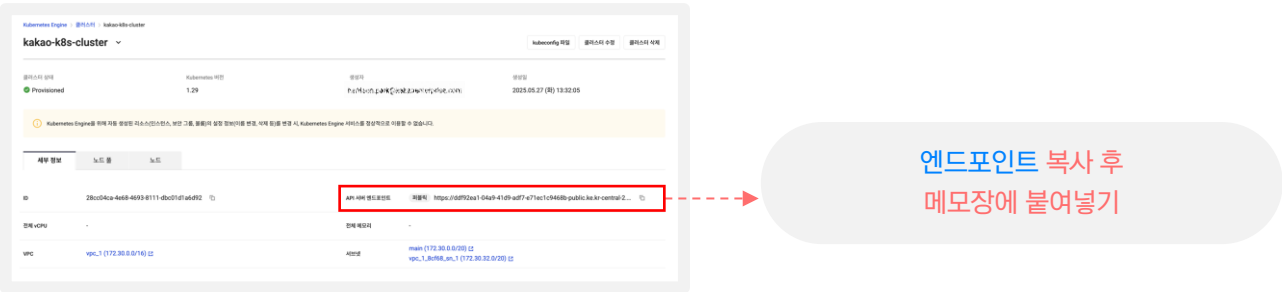
3. 카카오클라우드 로고 옆  버튼 클릭 > Container Pack > Kubernetes Engine > Cluster



4. 생성 된 Kubernetes Engine cluster 클릭



5. API 서버 엔드포인트 메모장에 붙여넣기



실습 순서

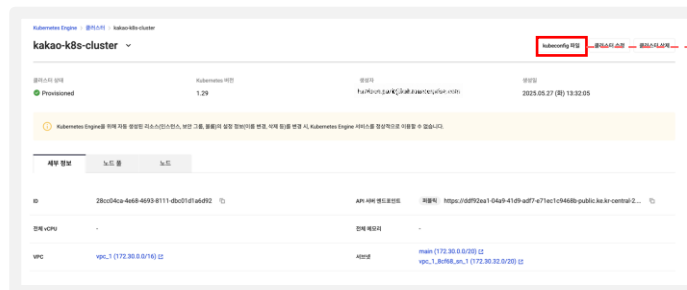
1

2

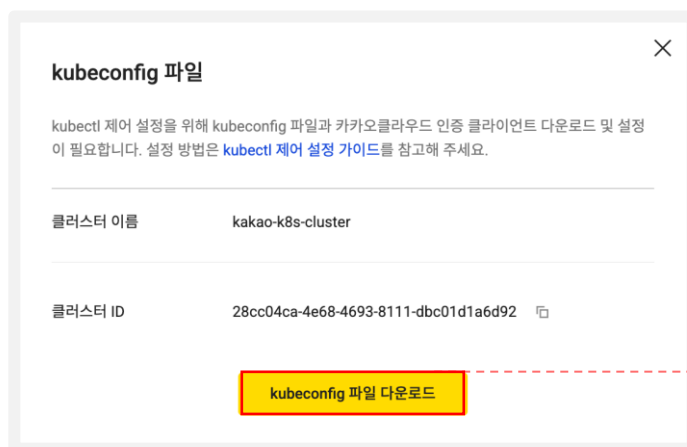
Bastion VM 인스턴스 생성
Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성 VM 생성 전 스크립트 작성

6. kubeconfig 파일 > kubeconfig 파일 다운로드

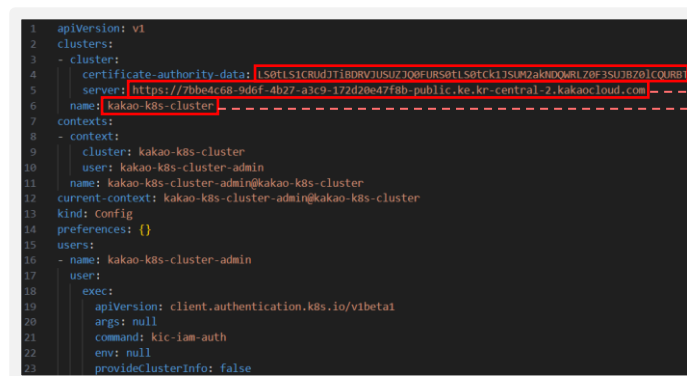


Kubeconfig 파일 클릭



다운로드 후 파일 열기

7. clusters 하위의 certificate-authority-data, name 값 메모장에 붙여넣기



Certificate-authority-data: 클러스터 인증데이터
Server: 클러스터 API 서버 값
(위장 5번에서 복사한 값과 동일함)
name: 클러스터명

위의 값들을
메모장에 복사해두기

실습 순서

1

2

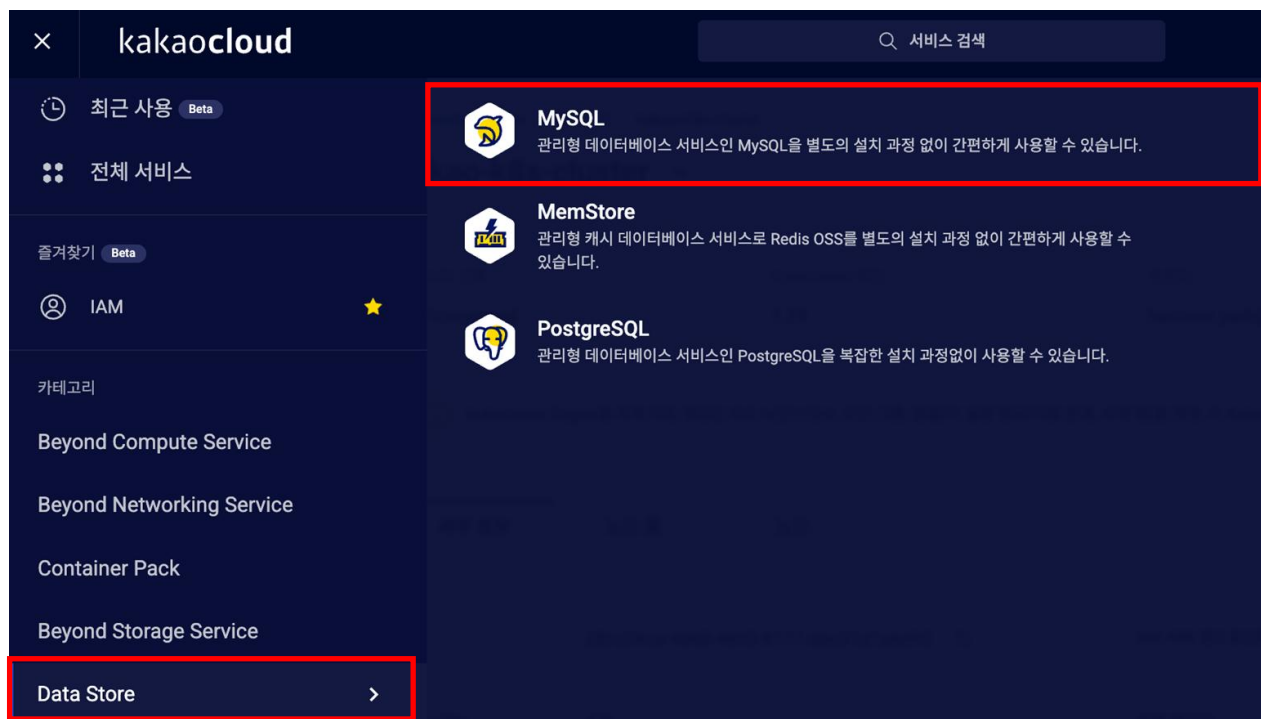
Bastion VM 인스턴스 생성

Bastion VM 인스턴스를 통해 클러스터 확인

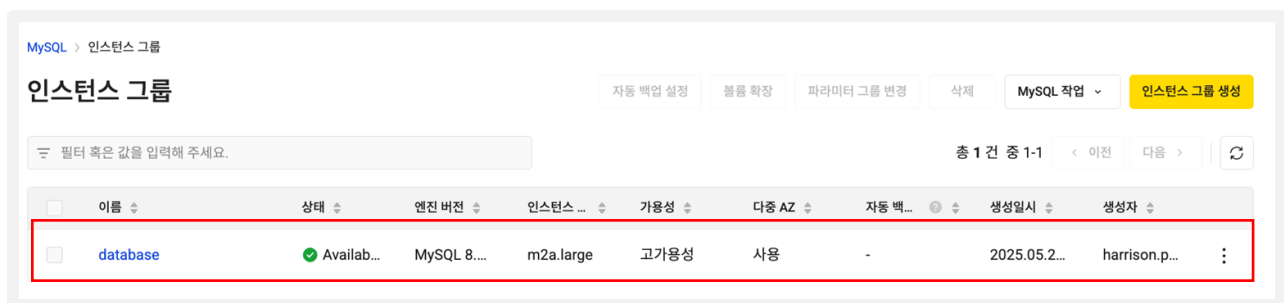
Bastion VM 인스턴스 생성

VM 생성 전 스크립트 작성

8. 카카오클라우드 로고 옆  버튼 클릭 > Data Store > MySQL



9. 생성했던 MySQL 인스턴스 그룹 클릭



실습 순서

1

2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

VM 생성 전 스크립트 작성

10. 두 엔드포인트 값 과 프로젝트 이름 메모장에 붙여넣기

프로젝트 이름
메모장에 복사하기

① az-a의 엔드포인트 주소
② az-b의 엔드포인트 주소

순서대로 메모장에 복사하기
(서버-DB 간 통신시 순서대로 진행
하므로 중요!)

11. 스크립트 환경변수 부분에 복사해둔 값들을 넣어 메모장에 작성

```
#!/bin/bash

echo "kakaocloud: 1.Starting environment variable setup"
# 환경 변수 설정: 사용자는 이 부분에 자신의 환경에 맞는 값을 입력해야 합니다.
command=$(cat <<EOF
export ACC_KEY='[REDACTED]'
export SEC_KEY='[REDACTED]'
export EMAIL_ADDRESS='사용자 이메일 입력(직접 입력)'
export CLUSTER_NAME='클러스터 이름 입력'
export API_SERVER='클러스터의 API 엔드포인트 입력'
export AUTH_DATA='클러스터의 certificate-authority-data 입력'
export PROJECT_NAME='프로젝트 이름 입력'
export INPUT_DB_EP1='Primary의 엔드포인트 입력' 1 az-a의 엔드포인트 주소
export INPUT_DB_EP2='Standby의 엔드포인트 입력' 2 az-b의 엔드포인트 주소
export DOCKER_IMAGE_NAME='이미지 이름 입력(demo-spring-boot)'
export DOCKER_JAVA_VERSION='자바 버전 입력(17-jdk-slim)'
export JAVA_VERSION='17'
export SPRING_BOOT_VERSION='3.1.0'
export DB_EP1=$(echo -n "${INPUT_DB_EP1}" | base64 -w 0)
export DB_EP2=$(echo -n "${INPUT_DB_EP2}" | base64 -w 0)
EOF
)
```

! 유의사항

맥 OS 사용자의 경우 사용자 입력 시에 작은따옴표가 자동 변환되는 경우가 빈번하니,
<https://n.lrl.kr> 같은 메모장 사이트를 이용해 주세요.

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

VM 생성 전 스크립트 작성

작성한 스크립트가 실행하는 내용

```
#!/bin/bash

echo "kakaocloud: 1.Starting environment variable setup"
# 환경 변수 설정: 사용자는 이 부분에 자신의 환경에 맞는 값을 입력해야 합니다.
command=$(cat <<EOF
export ACC_KEY='IAM 액세스 키 ID 입력'
export SEC_KEY='보안 액세스 키 입력'
export EMAIL_ADDRESS='사용자 이메일 입력(직접 입력)'
export CLUSTER_NAME='클러스터 이름 입력'
export API_SERVER='클러스터의 API 엔드포인트 입력'
export AUTH_DATA='클러스터의 certificate-authority-data 입력'
export PROJECT_NAME='프로젝트 이름 입력'
export INPUT_DB_EP1='Primary의 엔드포인트 입력'
export INPUT_DB_EP2='Standby의 엔드포인트 입력'
export DOCKER_IMAGE_NAME='이미지 이름 입력(demo-spring-boot)'
export DOCKER_JAVA_VERSION='자바 버전 입력(17-jdk-slim)'
export JAVA_VERSION='17'
export SPRING_BOOT_VERSION='3.1.0'
export DB_EP1=$(echo -n "$INPUT_DB_EP1" | base64 -w 0)
export DB_EP2=$(echo -n "$INPUT_DB_EP2" | base64 -w 0)
EOF
)

eval "$command"
echo "$command" >> /home/ubuntu/.bashrc
echo "kakaocloud: Environment variable setup completed"

echo "kakaocloud: 2.Checking the validity of the script download site"
curl --output /dev/null --silent --head --fail "https://github.com/kakaocloud-
edu/tutorial/raw/main/AdvancedCourse/src/script/script.sh" || { echo "kakaocloud: Script download site is not valid"; exit 1; }
echo "kakaocloud: Script download site is valid"

wget https://github.com/kakaocloud-edu/tutorial/raw/main/AdvancedCourse/src/script/script.sh
chmod +x script.sh
sudo -E ./script.sh
```

1. 스크립트 실행을 위한 환경 변수 설정

2. 설정한 환경 변수 적용

3. DNS 리졸빙 테스트

4. 초기 설정용 스크립트 실행

실습 순서

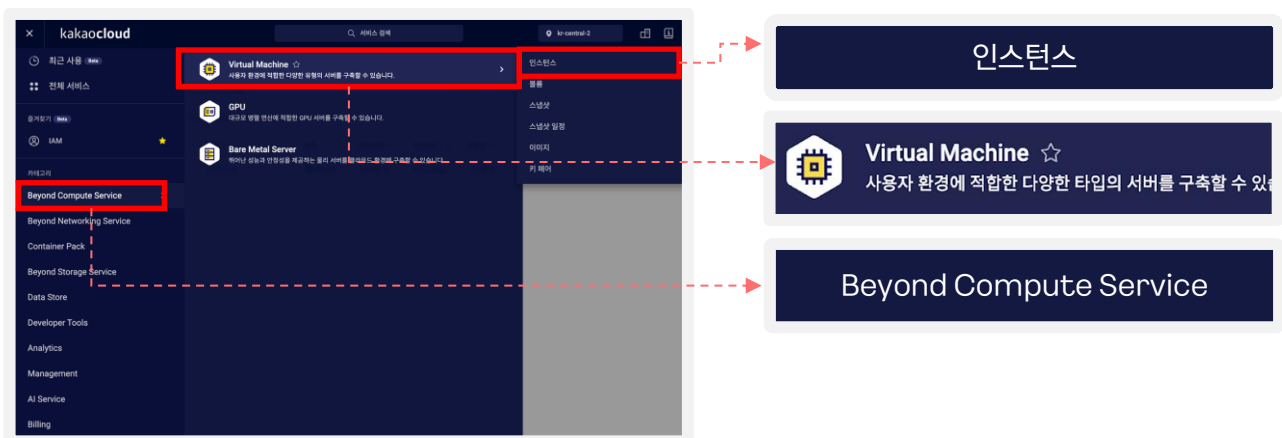
1

2

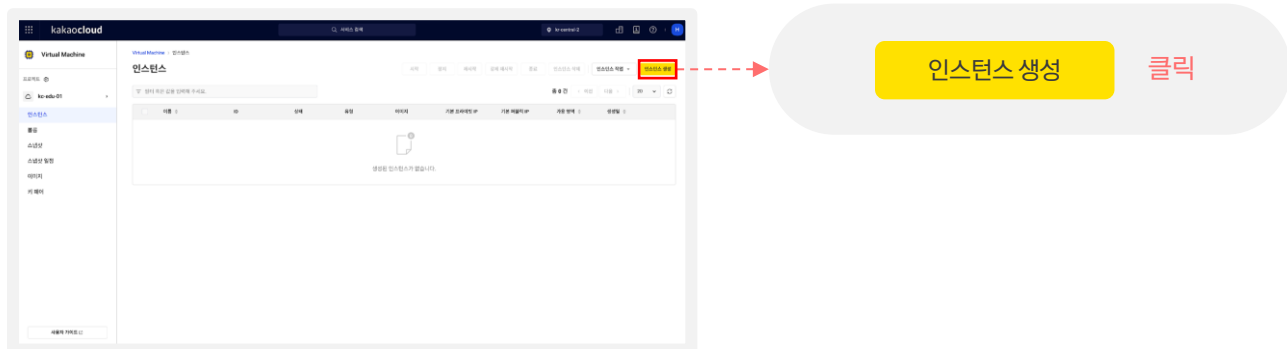
Bastion VM 인스턴스 생성
Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성 Instance 생성

1. 카카오클라우드 로고 옆  버튼 클릭 > Beyond Compute Service > Virtual Machine > 인스턴스



2. [인스턴스 생성] 클릭



실습 순서

1

2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

1. 이름 및 개수 입력 : bastion / 1개
2. 이미지 선택 : Ubuntu 20.04
3. 인스턴스 유형 선택 : m2a.large
4. 볼륨 입력 : 30 GB

인스턴스 생성

기본 정보

이름: 개수:

2개 이상의 인스턴스 생성 시, 인스턴스 이름에 자동으로 번호가 붙어 이름이 부여됩니다.
예) 인스턴스 개수 2개 선택 후 인스턴스 이름에 'test' 입력 시, 인스턴스 이름은 'test-1', 'test-2'로 부여

세부 설정 ▾

- 이름 : **bastion**
- 개수 : **1**

이미지

기본 내 이미지

Ubuntu Cent OS Windows Rocky Linux Alma Linux

필터 혹은 값을 입력해 주세요.

이름	아키텍처
Ubuntu 24.04 Kernel 6.8.0-39 6f8f7e1c-b801-46c5-940c-603ffc05247a	x86_64
Ubuntu 22.04 Kernel 5.15.0-117 07b055b9-e0e4-4713-b699-5a492ebc7890	x86_64
Ubuntu 20.04 Kernel 5.4.0-190 044eae16-ec2c-4f74-9345-5a9f69d80a9	x86_64
Ubuntu 22.04 - NVIDIA Kernel 5.15.0-100 with NVIDIA R550 Driver 550.54.14 and CUDA 12.4 (x86... c829b321-0ed9-4c79-886f-5a8fc477bf3a	x86_64
Ubuntu 20.04 - NVIDIA Kernel 5.4.0-173 with NVIDIA R550 Driver 550.54.14 and CUDA 12.4 (x86... 0071a59f-1bb8-458d-973a-c1f1f1642fc	x86_64

- 이미지 : **Ubuntu 20.04**

인스턴스 유형

> 조건 검색

선택 t1i.xlarge
4 vCPU | 16 GiB Memory | kr-central-2-a, kr-central-2-b

가장 빈용적으로 사용하는 머신 유형이며 가격 및 유연성을 위해 최적화되었습니다. [자세히 보기](#)

- 인스턴스 유형 : **t1i.xlarge**

볼륨

> 루트 볼륨

GB

+ 볼륨 추가

설정할 루트 볼륨의 크기가 이 이미지를 생성할 때 사용된 인스턴스의 볼륨 크기보다 작을 경우, 오류 상태의 인스턴스가 생성됩니다.

- 볼륨 : **30 GB**

실습 순서

1

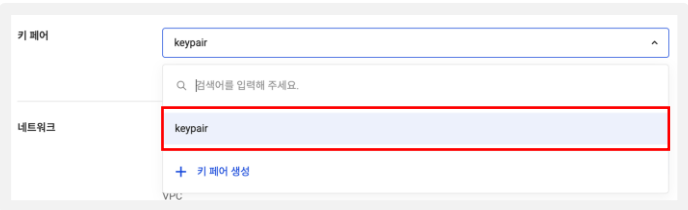
Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

5. 생성한 keypair 선택



확인

실습 순서

1

2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

6. VPC 선택 : vpc_1
7. 서브넷 선택 : main
8. 유형 : 새 인터페이스
9. IP 할당 방식 : 자동
10. 보안 그룹 생성 클릭

11. 보안 그룹 이름 입력 : bastion
12. 인바운드 정보 입력 :
 - 프로토콜: TCP
 - 출발지: 사용자 IP/32
 - 포트 번호: 22

VPC : vpc_1

서브넷 : main

유형 : 새 인터페이스

IP 할당 방식 : 자동

선택한 항목 클릭

클릭



이름 입력 : bastion

인바운드 정보

프로토콜	패킷출발지	포트번호
TCP	교육장의 사설 IP/32	22
TCP	0.0.0.0/0	8080

이어서 아웃바운드 설정을 진행합니다.
('생성' 버튼 클릭 X)



실습 순서

1

Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM
인스턴스 생성

11. 아웃바운드 정보 입력:

- 프로토콜: ALL
- 목적지: 0.0.0.0/0
- 포트 번호: ALL

12. [생성] 버튼 클릭

13. 보안 그룹 선택확인

보안 그룹 생성

보안 그룹 이름: bastion

보안 그룹 설명(선택): 100자 이내로 작성해 주세요.

적용된 규칙

인바운드: 아웃바운드

프로토콜: ALL, 목적지: 0.0.0.0/0, 포트 번호: ALL, 설명(선택): 50자 이내로 작성해 주세요.

+ 추가하기

목적지가 0.0.0.0/0이거나 포트 번호가 ALL 규칙은 모든 IP 주소와 모든 포트에 접속을 허용합니다. 알려진 IP 주소나 반드시 필요한 포트 번호만 허용하도록 보안 그룹을 설정하는 것이 좋습니다.

취소 생성

아웃바운드 정보

- 프로토콜: ALL
- 목적지: 0.0.0.0/0
- 포트 번호: ALL

생성

클릭



보안 그룹 선택 / 인바운드/아웃바운드 정책 확인

보안 그룹

선택된 항목 (1)

선택한 보안 그룹의 상세 규칙을 아래에서 확인할 수 있습니다.

bastion × [전체 삭제](#)

[인바운드/아웃바운드 규칙](#) ^

인바운드 아웃바운드

총 2건 중 1-2 < 이전 다음 >

보안 그룹	프로...	출발지	포트 ...	설명
bastion	TCP	0.0.0.0/0	22	-
bastion	TCP	0.0.0.0/0	8080	-

bastion ×

이어서 고급 설정을 진행합니다.
('생성' 버튼 클릭 X)



실습 순서

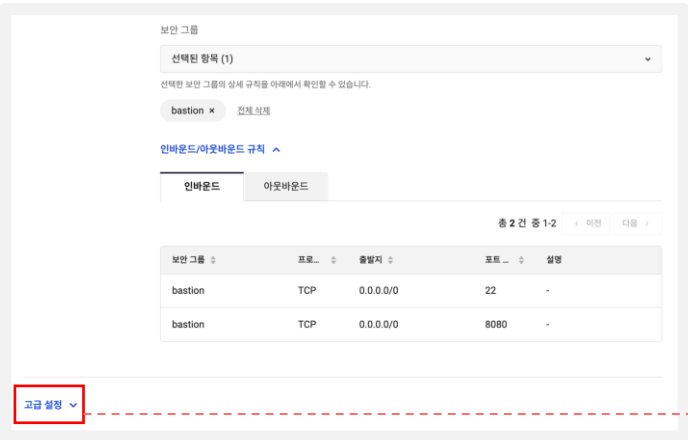
1

2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

14. 고급 설정 클릭
15. 사용자 스크립트 작성
- 고급 설정을 진행하지 않은 채 bastion만 생성하였다더라도 추후에 설정가능(35p~36p 참고)



하단의 [고급설정] 클릭



26page에서 작성한 메모장에 있는 스크립트를 복사 후 사용자 스크립트에 붙여 넣기

실습 순서

1

2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

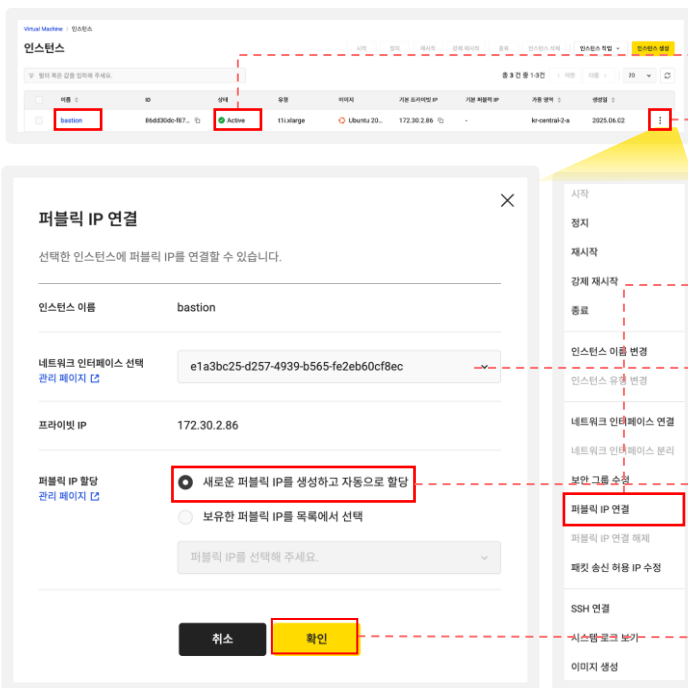
Bastion VM 인스턴스 생성

- 16. CPU 멀티스레딩 활성화 체크 x
- 17. 생성 클릭
- 18. 인스턴스 생성(Active) 확인
- 19. Bastion 더보기 > 퍼블릭 IP 연결
- 20. [새로운 퍼블릭 IP를 생성하고 자동으로 할당] 선택 후 > 확인 클릭



스크립트 작성된 상태

클릭



● Active

① 더보기 버튼 클릭

② 퍼블릭 IP 연결 클릭

③ 기본 선택된 네트워크 인터페이스

④ 선택

⑤ 확인 클릭

실습 순서

1

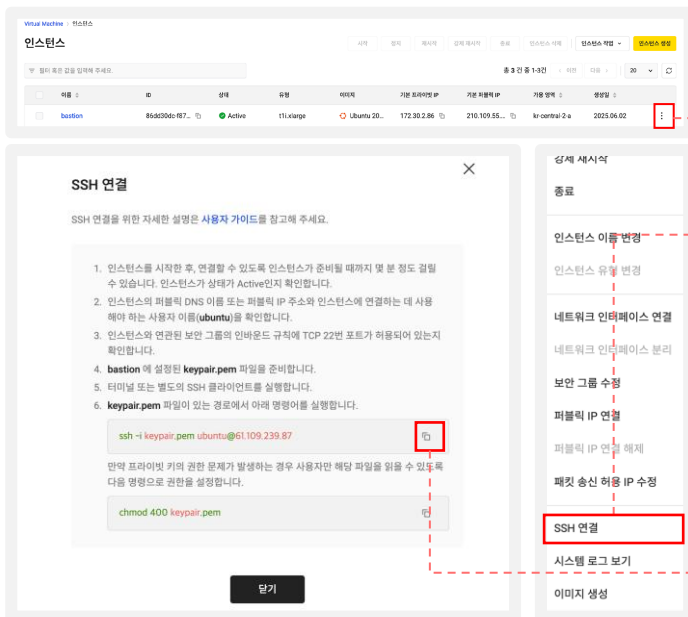
2

Bastion VM 인스턴스 생성
Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

21. 퍼블릭 IP 연결 확인
22. 더보기 > SSH 연결
23. SSH 접속 명령어 복사

24. 터미널 열고 입력하기
 - Keypair를 다운받아 놓은 폴더로 이동
 - 복사해둔 SSH 접속 명령어 붙여넣기
 - yes 입력하기



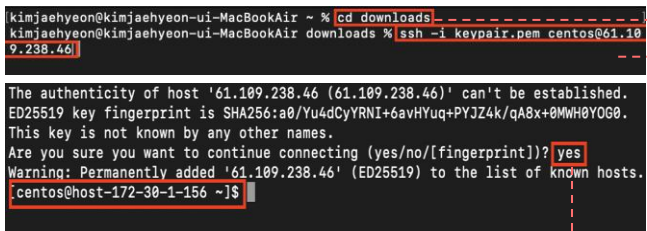
① 더보기 버튼 클릭

② SSH 연결 클릭

③ SSH 접속 명령어 복사

터미널에 명령어 붙여 넣기

- 터미널 열기(윈도우) : Window key + R, "cmd" 입력 후 엔터
- 터미널 열기(Mac) : F4, "터미널" 입력 후 엔터



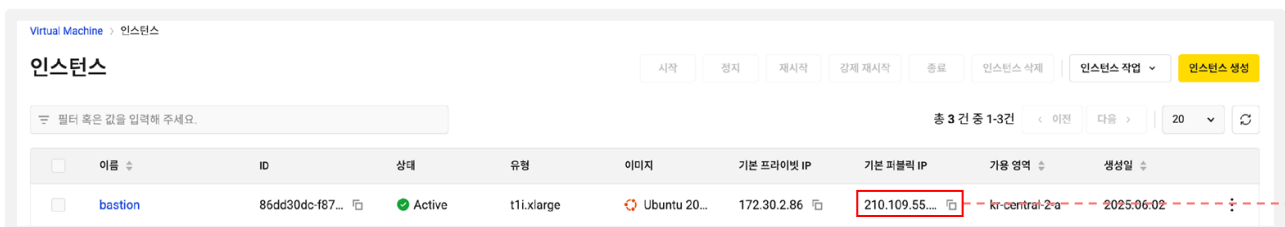
Keypair를 다운 받아 놓은 폴더로 이동

복사해둔 SSH 접속 명령어의 @IP주소 부분에
* Bastion 인스턴스의 Public IP로 붙여넣기

`ssh -i keypair.pem ubuntu@bastion의 Public IP`

Bastion instance 접속 완료

Yes 입력



실습 순서

1

Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM
인스턴스 생성

25. 명령어 수동으로 입력

◦ 고급 설정을 진행하지 않은 채 bastion만
생성하였을 시에 진행

* 잠깐 🍵 Tip!: bastion 생성시 고급 설정에 명령어를 넣지 못하였을 때 수행 방법

1. 메모장에 작성해 놓은 스크립트 중, 환경변수 설정 부분만 복사

```
#!/bin/bash

echo "kakaocloud: 1.Starting environment variable setup"

command=$(cat <<EOF
export ACC_KEY='82435d5e94ab49a0935a657ffc11bf6'
export SEC_KEY=
export EMAIL_ADDRESS=
export CLUSTER_NAME='kako-k8s-cluster3'
export API_SERVER='https://2122bfd0-5dac-4ca6-9ba2-2face2f619b9-public.ke.kr-central-2.kakaocloud.com:443'
export AUTH_DATA='LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR50tLS0tck1JSUM2akNDQWRlZ0Z3SUJBZ0lCQURBTkna3Foa2lHOXcwQkFRc0ZBREFWTVJNd0VRWURWUWFERXdwcmRXSmwK
hVmExZWwFwFlicWicGJJQ1o3bU4vVjB0aHNaVmowcXVksWl3d0J3TVhNN3NlU3lDVXc5CkVkcXN5UjRlFUK1VlYlZlZWVlcysrRlNFWm41QmRNRHU1UUVOVUh6ZWNURXQ4NjNQMMDV
export PROJECT_NAME='kakaocloud-online'
export INPUT_DB_EP1='az-a.hyundb0203.3c598fbfdbc4453690aa111f12f59f6.mysql.managed-service.kr-central-2.kakaocloud.com'
export INPUT_DB_EP2='az-b.hyundb0203.3c598fbfdbc4453690aa111f12f59f6.mysql.managed-service.kr-central-2.kakaocloud.com'
export DOCKER_IMAGE_NAME='hyun-spring3'
export DOCKER_JAVA_VERSION='17-jdk-slim'
export JAVA_VERSION='17'
export SPRING_BOOT_VERSION='3.1.0'
export DB_EP1=$(echo -n "$INPUT_DB_EP1" | base64 -w 0)
export DB_EP2=$(echo -n "$INPUT_DB_EP2" | base64 -w 0)
```

#!/bin/bash 부터 sudo -E ./script.sh 까지 복사

실습 순서

1

Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM
인스턴스 생성

25. 명령어 수동으로 입력

고급 설정을 진행하지 않은 채 bastion만
생성하였을 시에 진행

2. 복사한 스크립트 명령어 bastion 터미널에 붙여 넣기

```
ubuntu@host-10-0-3-113:~$ #!/bin/bash
command=$(cat <<EOF
export ACC_KEY='82435d5e94abubuntu@host-10-0-3-113:~$
ubuntu@host-10-0-3-113:~$
ubuntu@host-10-0-3-113:~$
ubuntu@host-10-0-3-113:~$ echo "kakaocloud: 1.Starting environment variable setup"
kakaocloud: 1.Starting environment variable setup
ubuntu@host-10-0-3-113:~$
EY='i1qzpcjkischTvQGubuntu@host-10-0-3-113:~$
ubuntu@host-10-0-3-113:~$
ubuntu@host-10-0-3-113:~$ command=$(cat <<EOF
2Z4zQ58SmxT_T45XaUzFW>
> export ACC_KEY='82435d5e94ab49a0935a657ffc11bf6'
>
> export SEC_KEY=
>
> export EMAIL_ADDRESS=
>
> export CLUSTER_NAME='kakao-k8s-cluster3'
>
> export API_SERVER='https://2122bfd0-5dac-4ca6-9ba2-2face2f619b9-public.ke.kr-central-2.kakaocloud.com:443'
>
> export AUTH_DATA='LS0tLS1CRUdJTiBDRVJUSUZJODFURSOtLS0tCk1JSUM2akNDQWRLZ0F3SUJBZ0lCQURBTkJna3Foa2lHOXcwQkFRc0ZBREFWTVJN
d0YRMLURWUJFERXdwcmRXSmwKY201bGRHVnpNQjRYRFRJME1ESXI0VEE1TURjd09Gb1hEVEowTURJelU1c0TVNVEl3T0Zvd0ZURVpRb3VHb3R0TFVRR0pBeE1LYTNW
aVpYSnVaWFlJc0Z0Q0FTSXdEUVlKS29aSWh2Y05BUlJVCQlFBGdhRVBBRENDQVZvQ2dnRUJBTlVBCmdoWVM4c1F5ZW8xZU0vOVDczc3B3WVdrNzQ2ZXN0OGk5
ZGhPcTAvaazdFb3gxdDRDMk5aQjRMSmgwK0tHYTItVtVksKNVphdGdXLY8xbDNxb3Vh5V0RrbzROWGhXTDlkZlBk03NS0td1YkZlY0tvdUd9Qm5yWEEdCMlVD
```

실습 순서

1

Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM
인스턴스 생성

26. cloud-init 파일 확인

1. cloud-init log 확인하는 명령어 입력

! 유의사항

터미널 창이 작으면 로그가 안보일 수도 있으니, 터미널 창의 크기를 늘려주세요.

- 모든 스크립트가 진행되면 아래와 같음

```
Every 2.0s: awk "/kakaocloud:/ {gsub(/[0-9]+\.\.\. host-192-168-2-33: Fri Mar 1 16:06:59 2024
kakaocloud: 1.Starting environment variable setup
kakaocloud: Environment variable setup completed
kakaocloud: 2.Checking the validity of the script download site
kakaocloud: script download site is valid
kakaocloud: 3.Variables validity test start
kakaocloud: Variables are valid
kakaocloud: 4.Github Connection test start
kakaocloud: Github Connection succeeded
kakaocloud: 5.Preparing directories and files
kakaocloud: Directories prepared
kakaocloud: 6.Downloading YAML files
kakaocloud: All YAML files downloaded
kakaocloud: 7.Preparing Spring directories and files
kakaocloud: Spring Directories and files prepared
kakaocloud: 8.Installing kubectl
kakaocloud: Kubectl installed
kakaocloud: 9.Installing kic-iam-auth
kakaocloud: kic-iam-auth installation completed
kakaocloud: 10.Installing Docker and setting it up
kakaocloud: Docker installation and setup completed
kakaocloud: 11.Installing additional software
kakaocloud: Additional software installation completed
kakaocloud: 12.Setting .kube/config
kakaocloud: Setting .kube/config completed
kakaocloud: 13.Downloading helm-values.yaml and applying environment substitutions
kakaocloud: Environment substitutions applied and setup completed
kakaocloud: 14.Checking Kubernetes cluster nodes status
kakaocloud: Successfully communicated with Kubernetes cluster
```

아래의 스크립트를 복사하여 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab03 link](#))

```
watch -c 'awk "/kakaocloud:/ {gsub(/[0-9]+\.\.\. host-192-168-2-33: Fri Mar 1 16:06:59 2024
kakaocloud: 1.Starting environment variable setup
kakaocloud: Environment variable setup completed
kakaocloud: 2.Checking the validity of the script download site
kakaocloud: script download site is valid
kakaocloud: 3.Variables validity test start
kakaocloud: Variables are valid
kakaocloud: 4.Github Connection test start
kakaocloud: Github Connection succeeded
kakaocloud: 5.Preparing directories and files
kakaocloud: Directories prepared
kakaocloud: 6.Downloading YAML files
kakaocloud: All YAML files downloaded
kakaocloud: 7.Preparing Spring directories and files
kakaocloud: Spring Directories and files prepared
kakaocloud: 8.Installing kubectl
kakaocloud: Kubectl installed
kakaocloud: 9.Installing kic-iam-auth
kakaocloud: kic-iam-auth installation completed
kakaocloud: 10.Installing Docker and setting it up
kakaocloud: Docker installation and setup completed
kakaocloud: 11.Installing additional software
kakaocloud: Additional software installation completed
kakaocloud: 12.Setting .kube/config
kakaocloud: Setting .kube/config completed
kakaocloud: 13.Downloading helm-values.yaml and applying environment substitutions
kakaocloud: Environment substitutions applied and setup completed
kakaocloud: 14.Checking Kubernetes cluster nodes status
kakaocloud: Successfully communicated with Kubernetes cluster
```

실습 순서

1

2

Bastion VM 인스턴스 생성 Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스 생성

27. config 파일 확인

1. config 파일 확인

```
ubuntu@host-10-0-0-90:~$ cat /home/ubuntu/.kube/config
apiVersion: v1
clusters:
- cluster:
  certificate-authority-data: "LS0tLS1CRUdJTiBDRVJUSUJzQ0FURSo0tLS0tCk1JSUM2akNDQWRlZ0F3SUJBZ0lCQURcOZBREWFwTVJNd0VRWURWUVRERXdwcmRXSmwKY201bGRHVnpNQjRYRFJME1ESXdnakU0TURreU1Wb1hEVE0wTURFek1ERTRNVFVRQ0pBeE1LYTNWbVpYSnVaWFlsY3pDQ0FTSXdEUVlIKS29aSWh2Y05BUUVCQ01FBGdndVBBRENDQVZvQ2dhRUJBTm81CjJVVVNOQSGOVEFoTFNjWUJyTHlOZmtGYVhoNkp3V1plcnJDRXNNUHhmTUhDUFlJZaUMKv2RUJazBBb0Z0ejVTaGVKv0VPZzFONFVRZFRhLUVibkijTExmQkZ0bHRXQ3k5OTJXbGpKbUQwYlloxaE0rODF2UmJiczUUrU3kvYzJ6SUdXUmVTdkhYd2p2MkxtV09ZcTdlUjVh5V0g5YUNLWF3MnUyazlrcGhmVG14SFZGWEOybWMOTG96ZVRlMmVJEvZFa0FdlNEJMcFJXZ3hTSUJlZVzM5dEp0K3gKVWJmVctYVWR1REVxQUlkOQlJZ1hwY0xeZGF1SHFoL0YyaUJwNjF0bDhtWk5uekhkYwpGMGNseGxwdzIRbGNSZDRrWXFjQ0F3RUJBYU5GTUVNd0RnWURWUjBQOVxVWRFd0VCCi93UUlNQVlCQWY4Q0FRQXdIUUVEVlIwTOJCWUUVGRHY5MDZKOWF1MOZGTmZGUkVPRzh4QzdTaWFOTUEwRONTcUcKU0RREJF0UImYmXlVXVVT1BCa2NPNkJKWbFhQdHBVR2ZZVklVUzZSUGtTbVpQay9rdApEdjUzUllkzaXl2TkNGUldNbmlNueTlNcmIDSn3ZUVTenRkVmxxNXhuTDdHdEN0cU9uClZxODZCeG54WUxPZGQxR2tySUhXNGc0RHpyaWNpb0lkCGFKM3REWTU5VzlpSzhvSnF0aU4dFmWRXhpVkfSY05Mb2xUcHlZKzIDVStQdE5ZcWNOa3dadFYwQ1F2ZjR2TlNZN1RNaVlaWWd6VkVMQVpsMApoNGJ4dFJLcjlBNcyFVC83d2MzRy8raWg2STZAM1g0YlgvQStqL001MmVhL0t0V25ScmRjTTRmV0RjY1JXczl5c1hIT1M3SnVxZHU3Q1lQQ2RlSHQrMy"
  server: https://569dc30c-60a2-4576-890b-081906720742-public.ke.kr-central-2.kakaocloud.com:443
name: kakao-k8s-cluster
contexts:
- context:
  cluster: kakao-k8s-cluster
  user: kakao-k8s-cluster-admin
  name: kakao-k8s-cluster-admin@kakao-k8s-cluster
current-context: kakao-k8s-cluster-admin@kakao-k8s-cluster
kind: Config
preferences: {}
users:
- name: kakao-k8s-cluster-admin
  user:
```

아래의 스크립트를 순서대로 복사하여 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab03 link](#))



```
cat /home/ubuntu/.kube/config
```

실습 순서

1

Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM
인스턴스 생성

28. YAML 파일 확인

2. 수동 배포에 필요한 YAML 파일의 존재 확인

```
ubuntu@host-10-0-2-135: ~$ ls /home/ubuntu/yaml/lab6-manifests.yaml
/home/ubuntu/yaml/lab6-manifests.yaml
```

아래의 스크립트를 순서대로 복사하여 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab03 link](#))



```
ls /home/ubuntu/yaml/lab6-manifests.yaml
```

실습 순서

1

Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

Bastion VM 인스턴스에서 클러스터 노드 정보 조회

터미널에 명령어 붙여 넣기

```

[ubuntu@host-172-16-1-159:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
host-172-16-19-120                 Ready    <none>    32h   v1.26.6
host-172-16-51-50                 Ready    <none>    32h   v1.26.6

```

아래의 스크립트를 순서대로 복사하여 터미널에 붙여 넣습니다.
 *Github 사이트에서도 복사 가능 ([Lab03 link](#))



```

kubectl get nodes

```

◦생성한 노드 두 개가 정상적으로 나오는지 확인

실습 순서

1

Bastion VM 인스턴스 생성

2

Bastion VM 인스턴스를 통해 클러스터 확인

잠깐 🍵 Tip! : kubectl get nodes 명령어 에러 발생시 해결 방법

Cloud-init 스크립트 실행 로그 확인

- 로그 확인 명령어 실행 (watch "cat /var/log/cloud-init-output.log | grep kakaocloud:")
- 13번까지 잘 실행되었는지 확인, 완료되지 않았다면 해당 오류 로그를 확인 후 스크립트 재작성
- 재작성한 스크립트를 터미널상에 붙여 넣어 다시 적용

config 파일 확인

- 환경 변수 적용 확인 (cat /home/ubuntu/.kube/config)
 - config 파일 변수 란에 공백이 있다면 echo 명령어를 통해 해당 변수 출력해보기 (ex: echo \$API_SERVER)
 - echo 시 공백이 나오는 경우 export 명령어를 통해 환경 변수 새로 적용 (ex: export API_SERVER='https:// ... kakaocloud.com:443')
- 권한 확인 (ls -al /home/ubuntu/.kube/config)


```
-rw----- 1 ubuntu ubuntu 2522 Feb 28 14:52 /home/ubuntu/.kube/config
```

 - 위와 결과가 다른 경우 아래 명령어수행


```
sudo chmod 600 /home/ubuntu/.kube/config
sudo chown ubuntu:ubuntu /home/ubuntu/.kube/config
```

환경 변수 입력이 잘 되었는지 확인하기

- 환경 변수 파일 확인 (cat /home/ubuntu/.bashrc)
 - 환경변수가 잘 적용된 상태인지 확인, 공백이 있다면 환경 변수 부분 스크립트 재작성후 붙여넣기
- 위 사항들을 모두 확인했음에도 문제 발생시 환경 변수 오타 확인 필요
 - 메모장에 복사-붙여넣기 시 공백, 오타 발생 / 작은따옴표(' ') 오류 등

Lab4 :

Container Image 만들기

이론 교재 78p

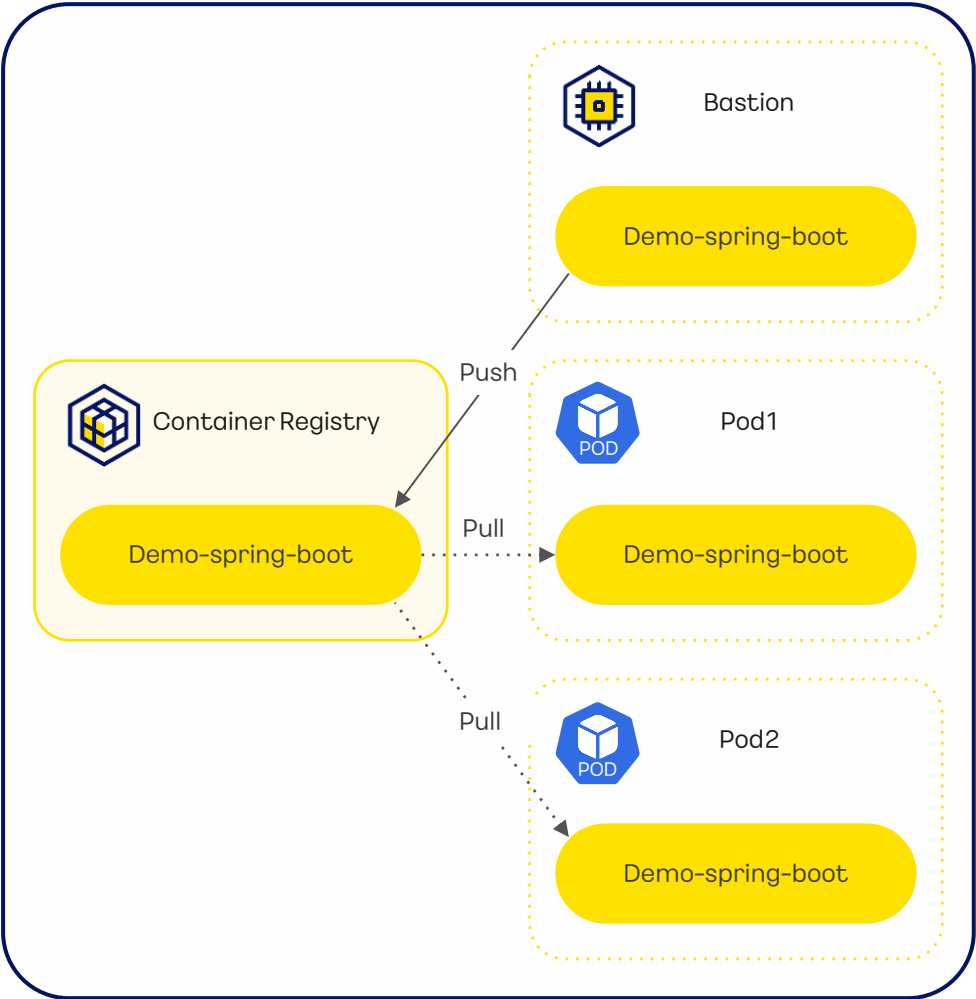


실습 내용

Spring Boot 프로젝트를 생성해 간단한 웹 페이지를 생성합니다. 생성한 프로젝트를 Docker Image 파일로 만들어 KakaoCloud Container Registry에 업로드하는 실습을 진행합니다

실습 순서

- 1 Container Registry 생성
- 2 Spring 어플리케이션을 Container 이미지로 만들기
- 3 Container Registry에 이미지 업로드



실습 순서

1

Container
Registry 생성

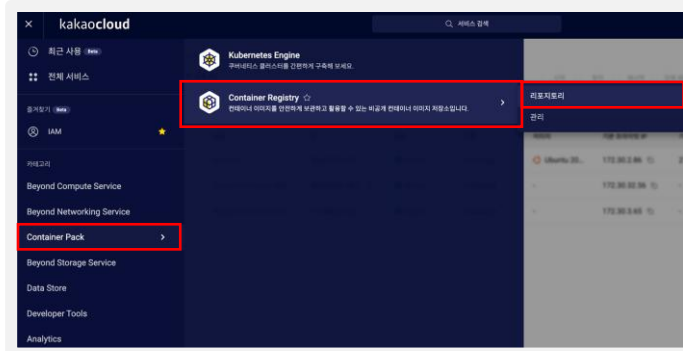
2

Spring 어플리케이션
을 Container 이미지
로 만들기

3

Container Registry에
이미지 업로드

Container Registry 생성

1. 카카오클라우드 로고 옆  버튼 클릭 > Container Pack > Container Registry > 리포지토리

2. 리포지토리 탭 > 리포지토리 생성 클릭



리포지토리 생성

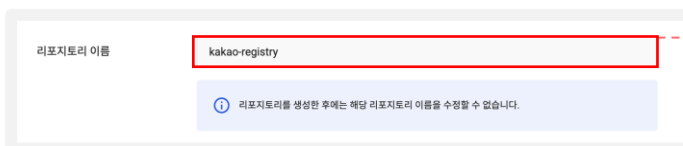
클릭

3. 공개 여부 선택



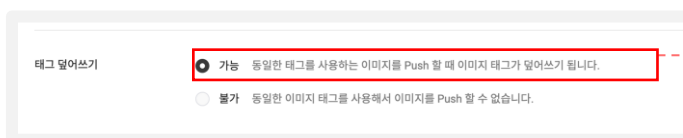
. 비공개 선택

4. 리포지토리 이름 작성



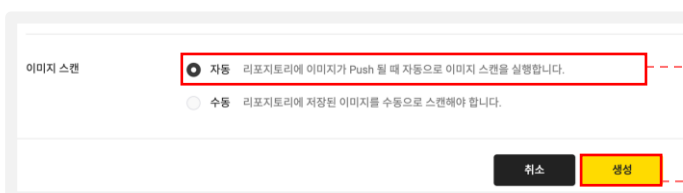
. kakao-registry 입력

5. 태그 덮어쓰기 선택



. 가능 선택

6. 이미지 스캔 선택 및 생성 클릭



. 자동 선택

생성

클릭

실습 순서

1

Container
Registry 생성

2

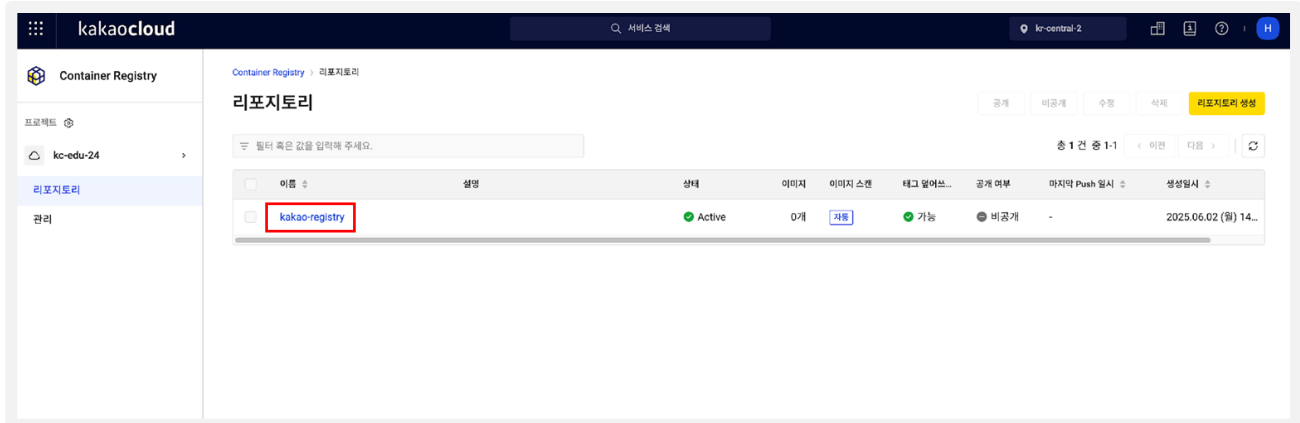
Spring 어플리케이션
을 Container 이미지
로 만들기

3

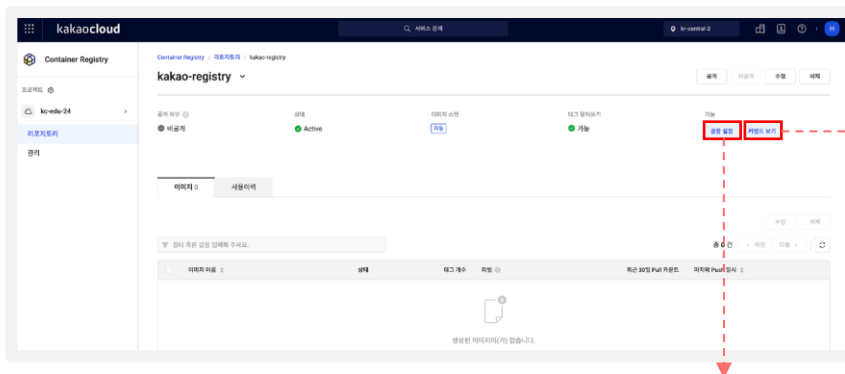
Container Registry에
이미지 업로드

Container Registry 생성

7. 생성된 Container Registry 클릭



8. 생성된 Container Registry 클릭



*커맨드 보기

인증, 이미지 Push,
이미지 Pull과 관련된
Docker 커맨드
(Command) 확인 가능

*잠깐 🍵 Tip! : 권한 설정

- 조직 내 사용자 중 레지스트리 권한이 없는 사용자에게 리포지토리 멤버 또는 뷰어 역할을 추가할 수 있습니다.
- 추가된 사용자는 추가된 역할에 따라 해당 리포지토리의 이미지를 Push하거나 Pull 할 수 있는 권한을 가지게 됩니다.

실습 순서

1

Container
Registry 생성

2

Spring 어플리케이션
을 Container 이미지
로 만들기

3

Container Registry에
이미지 업로드

Spring 어플리케이션을 Container 이미지로 만들기

1. Spring 어플리케이션이 있는 경로로 이동

```
ubuntu@host-10-0-2-96:~$ cd /home/ubuntu/spring
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
cd /home/ubuntu/spring
```

2. Spring 어플리케이션 패키징 및 빌드

```
ubuntu@host-10-0-2-96:~/spring$ if sudo ./mvnw clean package; then
> echo "Maven build successful."
> else
> echo "Maven build failed."
> exit 1
> fi
```

```
Warning: JAVA_HOME environment variable is not set
[INFO] Scanning for projects...
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
if sudo ./mvnw clean package; then
    echo "Maven build successful."
else
    echo "Maven build failed."
    exit 1
fi
```

```
ubuntu@host-10-0-2-96:~/spring$ sudo bash -c "cat <<EOF > Dockerfile
JAVA_VERSION}
RUN apt-get update && apt-get install -y curl
C> FROM openjdk:${DOCKER_JAVA_VERSION}
> RUN apt-get update && apt-get install -y curl
> COPY target/demo-0.0.1-SNAPSHOT.jar demo.jar
> ENTRYPOINT ["java","-jar","/demo.jar"]
> EOF"
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
sudo bash -c "cat <<EOF > Dockerfile
FROM openjdk:${DOCKER_JAVA_VERSION}
RUN apt-get update && apt-get install -y curl
COPY target/demo-0.0.1-SNAPSHOT.jar demo.jar
ENTRYPOINT ["java", "-jar", "/demo.jar"]
EOF"
```

```
ubuntu@host-10-0-2-96:~/spring$ sudo docker build -t ${DOCKER_IMAGE_NAME} .
[+] Building 2.1s (8/8) FINISHED
=> [internal] load build definition
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
sudo docker build -t ${DOCKER_IMAGE_NAME} .
```

1

2

3)

Container Registry에 이미지 업로드

```
ubuntu@host-172-30-0-225:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
demo-spring-boot	latest	37132342afed	12 minutes ago	454MB

[illegible]

Virtual Machine > 인스턴스

인스턴스

시작

정지

재시작

강제 재시작

종료

인스턴스 삭제

인스턴스 작업

인스턴스 생성

필터 혹은 값을 입력해 주세요.

총 3 건 중 1-3건

이전

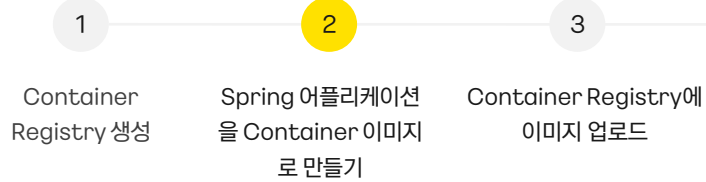
다음

20

☐	이름	ID	상태	유형	이미지	기본 프라이빗 IP	기본 퍼블릭 IP	가용 영역	생성일
☐	bastion	86dd30dc-f87...	Active	t1.xlarge	Ubuntu 20...	172.30.2.86	210.109.55...	kr-central-2-a	2025.06.02

47

실습 순서

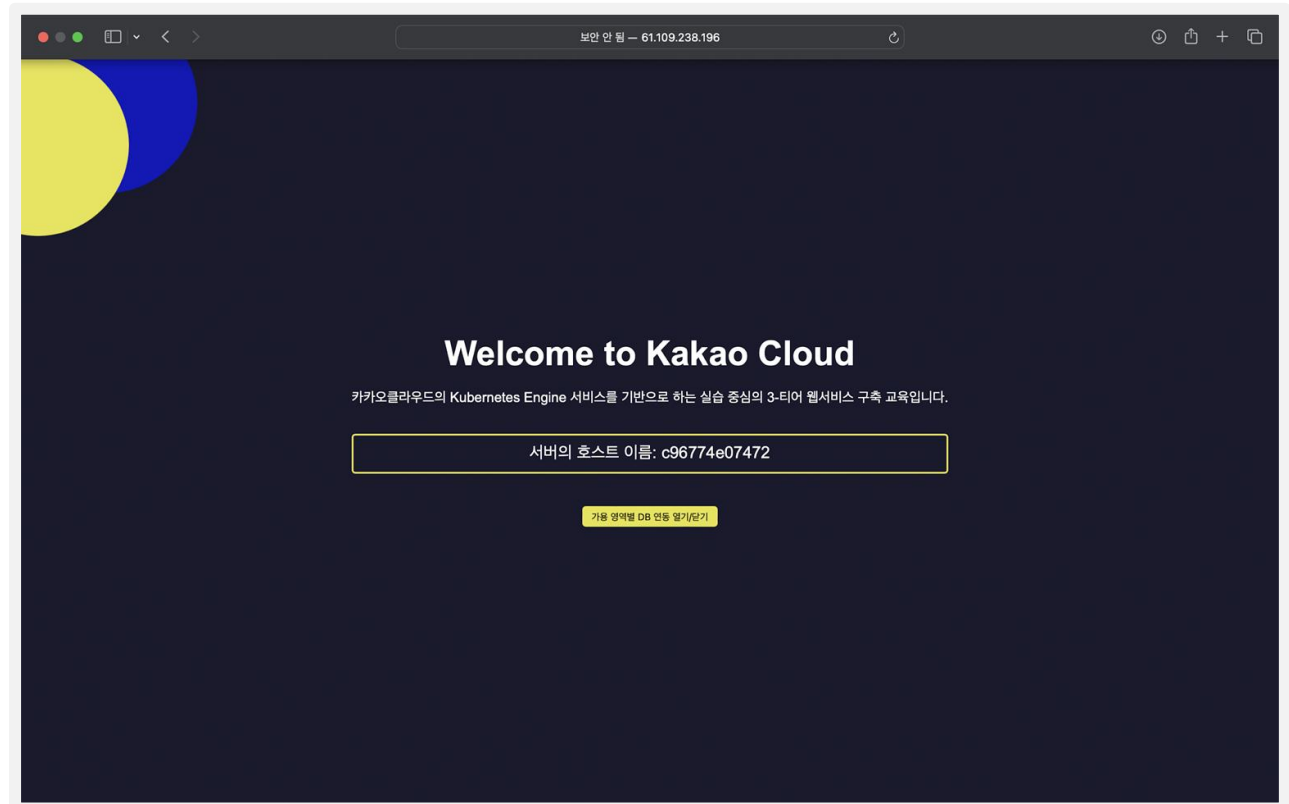


Spring 어플리케이션을 Container 이미지로 만들기

5. 웹 접속



6. 이미지 실행 확인



실습 순서

1

Container
Registry 생성

2

Spring 어플리케이션
을 Container 이미지
로 만들기

3

Container Registry에
이미지 업로드

Container Registry에 이미지 업로드

1. 도커 로그인

로그인 성공 시 Login Succeeded 출력

```
ubuntu@host-172-16-1-159:~$ docker login ${PROJECT_NAME}.kr-central-2.kcr.dev --username ${ACC_KEY} --password ${SEC_KEY}
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /home/ubuntu/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
Login Succeeded
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
docker login ${PROJECT_NAME}.kr-central-2.kcr.dev --username ${ACC_KEY} --password ${SEC_KEY}
```

2. Spring 어플리케이션 이미지 Push를 위한 태깅(tag)

```
ubuntu@host-172-16-1-159:~$ docker tag ${DOCKER_IMAGE_NAME} ${PROJECT_NAME}.kr-central-2.kcr.dev/kakao-registry/${DOCKER_IMAGE_NAME}:1.0
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
docker tag ${DOCKER_IMAGE_NAME} ${PROJECT_NAME}.kr-central-2.kcr.dev/kakao-registry/${DOCKER_IMAGE_NAME}:1.0
```

3. 이미지가 정상적으로 태그 되었는지 확인

```
ubuntu@host-172-16-1-159:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
kakao-sw-club.kr-central-2.kcr.dev/kakao-registry/demo-spring-boot	1.0	4b1f07bf7430	6 minutes ago	454MB
demo-spring-boot	latest	4b1f07bf7430	6 minutes ago	454MB

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
docker images
```

실습 순서

1

Container
Registry 생성

2

Spring 어플리케이션
을 Container 이미지
로 만들기

3

Container Registry에
이미지 업로드

Container Registry에 이미지 업로드

4. Spring 어플리케이션 이미지 Push

```
ubuntu@host-172-16-1-159:~$ docker push ${PROJECT_NAME}.kr-central-2.kcr.dev/kakao-registry/${DOCKER_IMAGE_NAME}:1.0
The push refers to repository [kakao-sw-club.kr-central-2.kcr.dev/kakao-registry/demo-spring-boot]
49b0bff9168d: Pushed
cb158bd486a0: Pushed
6be690267e47: Pushed
13a34b6fff78: Pushed
9c1b6dd6c1e6: Pushed
1.0: digest: sha256:d93d1749ba9bb8f3d364ba5211699fcbe16a95b462c8b6f9217b4516b56330e0 size: 1377
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab04 link](#))



```
docker push ${PROJECT_NAME}.kr-central-2.kcr.dev/kakao-registry/${DOCKER_IMAGE_NAME}:1.0
```

5. 카카오클라우드 로고 옆 버튼 클릭 > Container Pack > Container Registry > 리포지토리

실습 순서

1

Container
Registry 생성

2

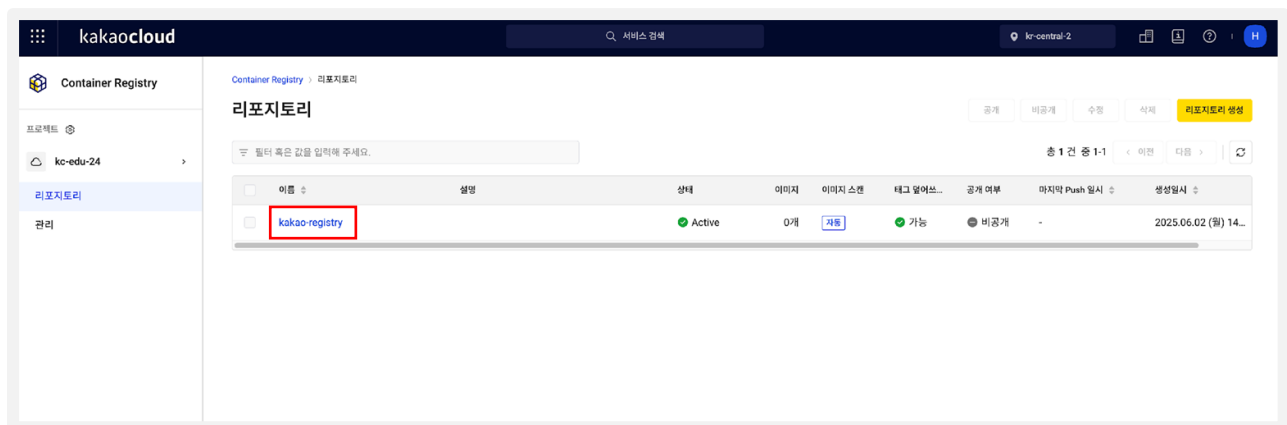
Spring 어플리케이션
을 Container 이미지
로 만들기

3

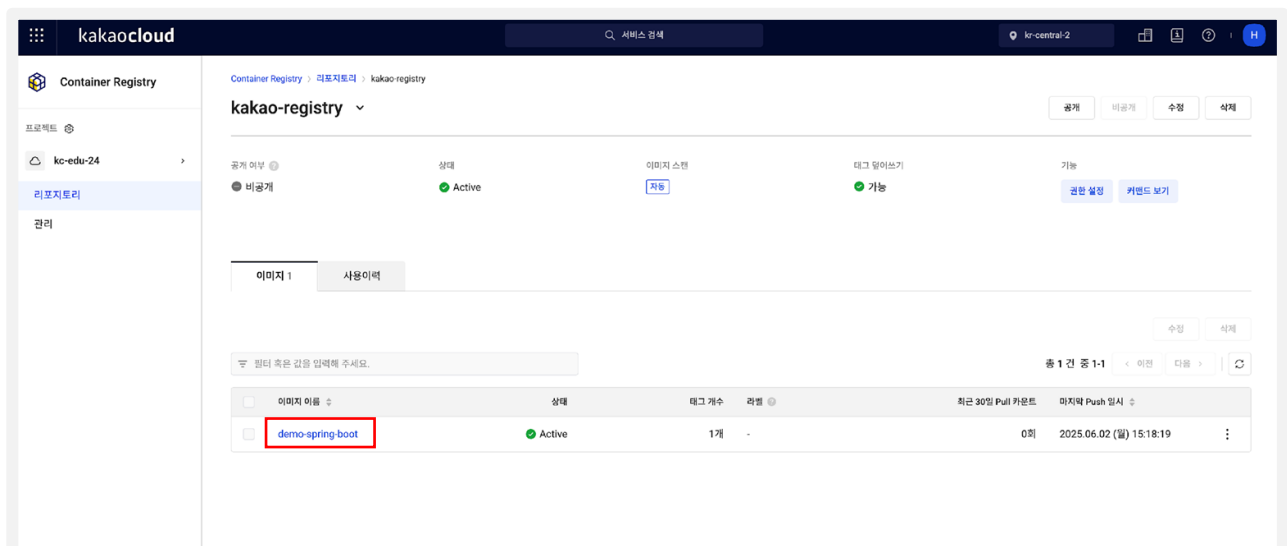
Container Registry에
이미지 업로드

Container Registry에 이미지 업로드

6. Repository 클릭



7. 업로드된 이미지 확인



Lab5 :

인그레스 컨트롤러 배포

이론 교재 93p

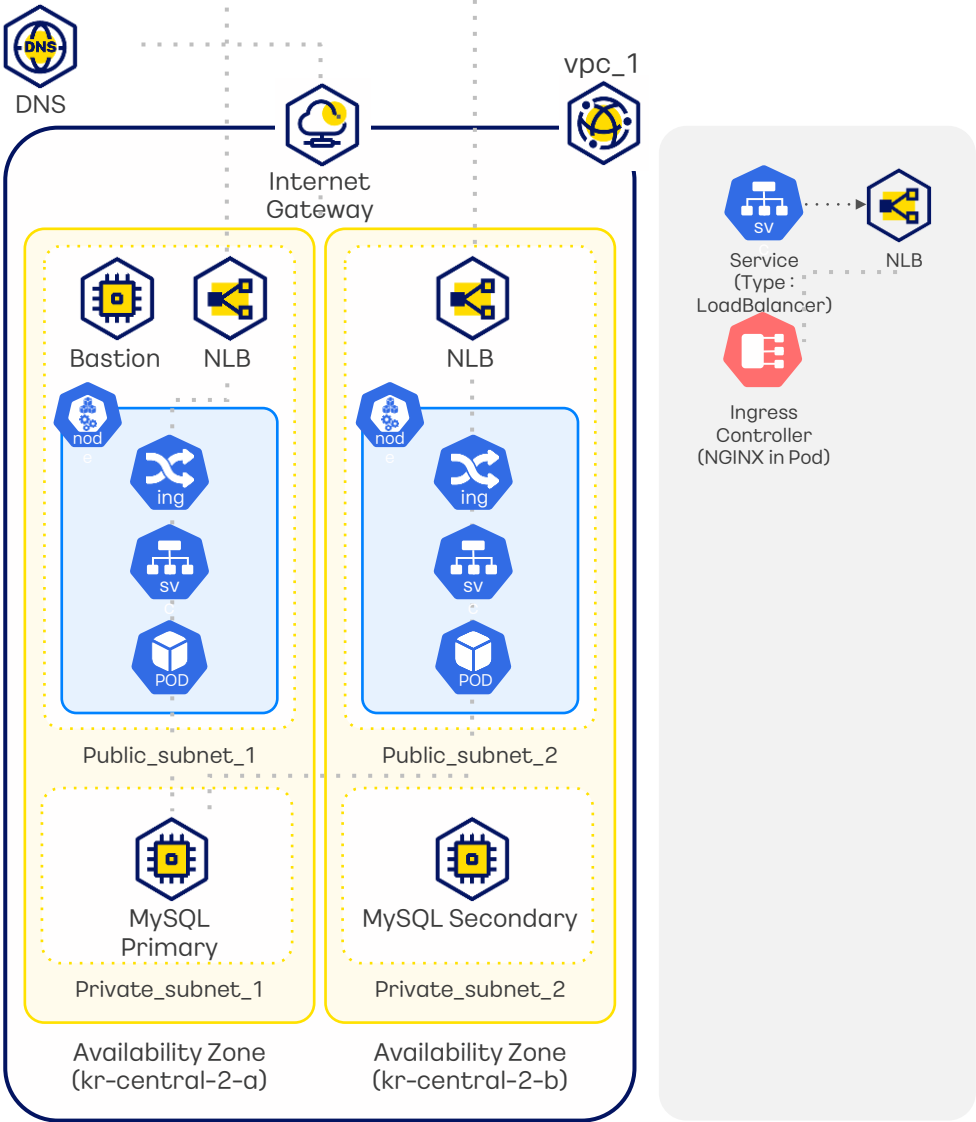


실습 내용

nginx pod을 위한 deployment, LoadBalancer type의 서비스가 포함된 ingress-nginx controller를 배포하고 ingress-nginx 파드, 서비스, AZ별로 생성된 loadbalancer를 확인하는 실습입니다.

실습 순서

- 1 ingress-nginx controller 배포
- 2 ingress-nginx 파드 및 서비스 상태 확인
- 3 로드 밸런서 확인



실습 순서

1

ingress-nginx controller 배포

2

ingress-nginx 파드 및 서비스 상태 확인

3

로드 밸런서 확인

ingress-nginx controller 배포

1. ingress-nginx-controller 배포

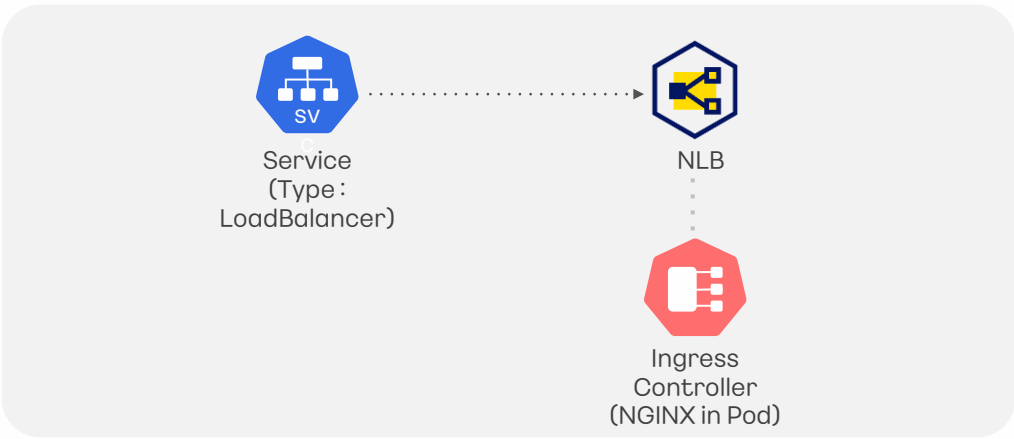
```
ubuntu@host-172-30-0-190:~$ kubectl apply -f https://github.com/kakaocloud-edu/tutorial/raw/main/AdvancedCourse/src/ingress-nginx-controller.yaml
namespace/ingress-nginx created
serviceaccount/ingress-nginx created
```

스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab05 link)



```
kubectl apply -f https://github.com/kakaocloud-edu/tutorial/raw/main/AdvancedCourse/src/manifests/ingress-nginx-controller.yaml
```

[컨트롤러 배포 시 포함되는 구성요소]



- ingress-controller 구동 pod을 위한 deployment (replicaset 포함)
- 트래픽 분배를 위한 LoadBalancer type의 service
- Ingress 리소스를 생성 및 변경할 때 필요한 admission controller 구동 pod 및 job
- (다음 페이지에서 자세히 확인 가능)

실습 순서

- 1
- 2
- 3
- ingress-nginx controller 배포
- ingress-nginx 파드 및 서비스 상태 확인
- 로드 밸런서 확인

ingress-nginx 파드 및 서비스 상태 확인

ingress-nginx
배포 상태 확인

!유의 사항
ingress-nginx-controller-*가 Running 상태여야 함

```
ubuntu@host-172-30-0-190:~$ kubectl get all -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS	AGE
pod/ingress-nginx-admission-create-glitzb	0/1	Completed	0	2d19h
pod/ingress-nginx-admission-patch-vtgx2	0/1	Completed	1	2d19h
pod/ingress-nginx-controller-6877dbbcb-qkqx	1/1	Running	0	2d19h
pod/ingress-nginx-controller-6877dbbcb-pj9tg	1/1	Running	0	2d19h

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab05 link)



```
kubectl get all -n ingress-nginx
```

NAME	TYPE	PORT(S)	CLUSTER-IP	EXTERNAL-IP	AGE
service/ingress-nginx-controller	LoadBalancer	80:30314/TCP, 443:32344/TCP	10.108.228.11	k8s-ingress-nginx-8a7128a70c-df9352bc841e4	2d19h
service/ingress-nginx-controller-admission	ClusterIP	443/TCP	10.108.65.119	<none>	2d19h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/ingress-nginx-controller	2/2	2	2	2d19h

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/ingress-nginx-controller-6877dbbcb	2	2	2	2d19h

NAME	COMPLETIONS	DURATION	AGE
job.batch/ingress-nginx-admission-create	1/1	11s	2d19h
job.batch/ingress-nginx-admission-patch	1/1	13s	2d19h

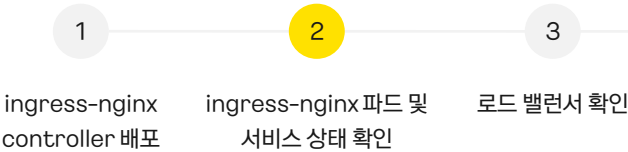
◦Pod의 상태 이외에도 Service, Deployment, Replicaset, Job의 상태를 확인할 수 있음



kubectl get all -n ingress-nginx 결과에 나타나는 리소스

- Pod : 총 4개
 - admission controller를 위한 pod 2개 (pod/ingress-nginx-admission) : **Completed** 상태
 - 메인 ingress controller pod 2개 (pod/ingress-nginx-controller) : **Running** 상태
- Service : 총 2개
 - 트래픽 분배를 위한 LoadBalancer 타입의 Service 1개 (service/ingress-nginx-controller)
 - admission controller의 내부 통신을 위한 Service 1개 (service/ingress-nginx-controller-admission)
- Deployment : 총 1개
 - 메인 ingress controller를 위한 Deployment 1개 (deployment.apps/ingress-nginx-controller)
- Replicaset : 총 1개
 - 메인 ingress controller를 위한 Replicaset 1개 (replicaset.apps/ingress-nginx-controller)
- Job : 총 2개
 - Ingress 리소스를 생성 및 변경할 때 필요한 admission controller 위한 Job 2개 (job.batch/ingress-nginx-admission)

실습 순서



ingress-nginx 파드 및 서비스 상태 확인

ingress-nginx 서비스 상태 확인

! 유의사항
ingress-nginx-controller 서비스(LoadBalancer type)의 External-IP에 DNS 주소 값이 나타나야 함

1. ingress-nginx-controller 배포

```
ubuntu@host-172-30-1-118:~$ kubectl get svc -n ingress-nginx
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
ingress-nginx-controller            LoadBalancer        10.104.207.50    k8s-ingress-ingress-9f74bd9591-04a79554c22349abab4238153d0f5683.ke.kr-central-2.kakaocloud.com  80:32478/TCP,443:32706/TCP  4m7s
ingress-nginx-controller-admission ClusterIP            10.111.245.112   <none>           443/TCP          4m7s
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab05 link)



`kubectl get svc -n ingress-nginx`

복사 후 아래 명령어 (DNS 주소 값)에 입력

2. nslookup 결과 확인

```
ubuntu@host-172-30-1-118:~$ nslookup k8s-ingress-ingress-9f74bd9591-04a79554c22349abab4238153d0f5683.ke.kr-central-2.kakaocloud.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   k8s-ingress-ingress-9f74bd9591-04a79554c22349abab4238153d0f5683.ke.kr-central-2.kakaocloud.com
Address: 172.30.49.99
Name:   k8s-ingress-ingress-9f74bd9591-04a79554c22349abab4238153d0f5683.ke.kr-central-2.kakaocloud.com
Address: 172.30.0.136
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab05 link)



`nslookup {DNS 주소 값}`

실습 순서

- 1

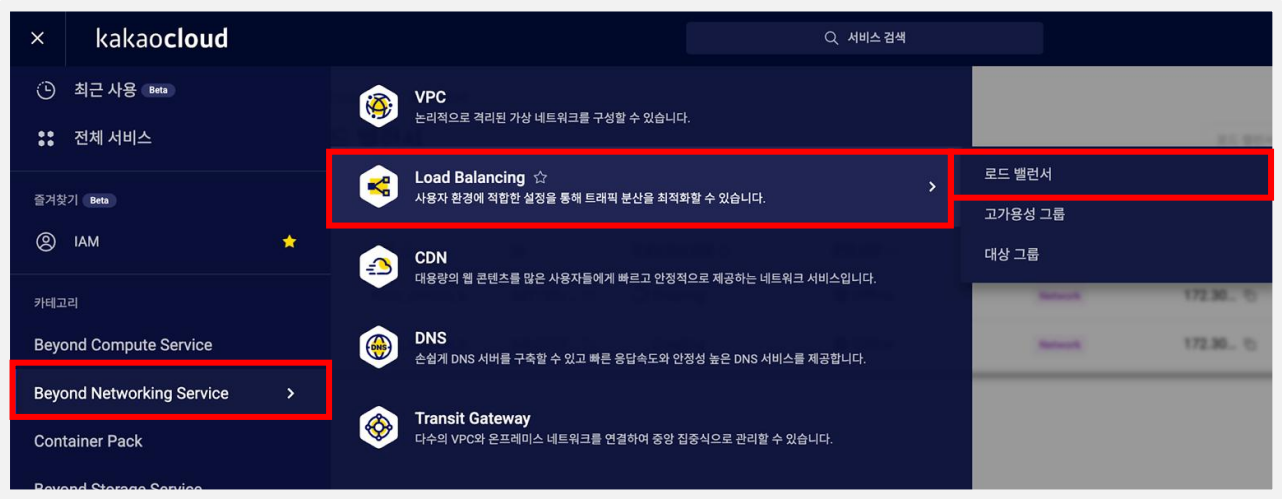
ingress-nginx controller 배포
- 2

ingress-nginx 파드 및 서비스 상태 확인
- 3

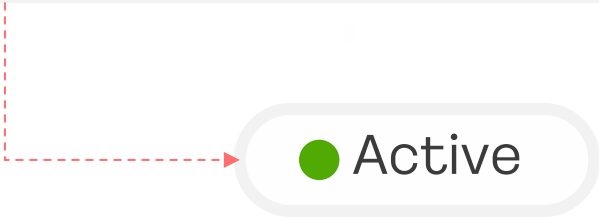
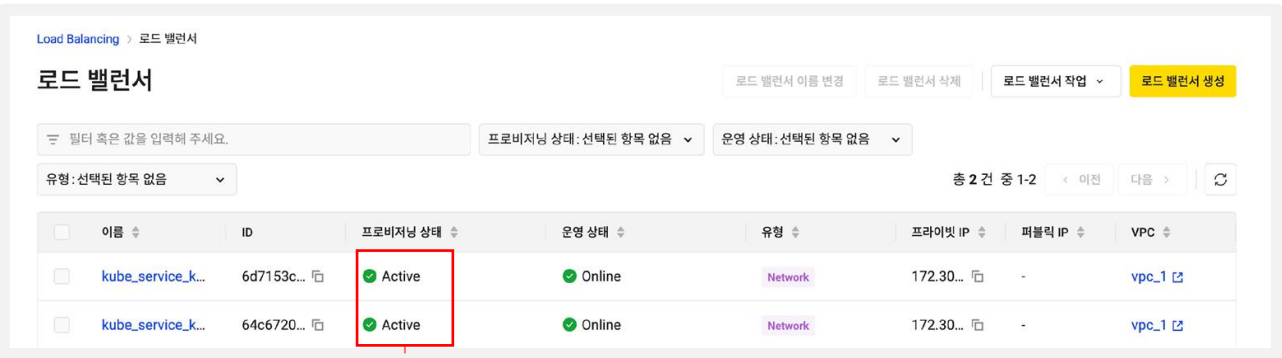
로드 밸런서 확인

로드 밸런서 확인

1. 카카오클라우드 로고 옆  버튼 클릭 > Beyond Networking Service > Load Balancing > 로드 밸런서



2. 로드 밸런서 콘솔창에서 AZ별로 생성된 로드 밸런서 확인



실습 순서

- 1
- 2
- 3

ingress-nginx
controller 배포

ingress-nginx 파드 및
서비스 상태 확인

로드 밸런서 확인

로드 밸런서 확인

3. AZ별로 생성된 로드 밸런서에 퍼블릭 IP 부여

로드 밸런서

이름	리전	프론트엔드 IP	상태	타겟	프론트엔드 IP	백엔드 IP	타겟	타겟	생성일
kube_service_ko...	627753...	Active	On-line	Private	172.30...	ip-102	kr-central2-a	2025-06-02	
kube_service_ko...	646672...	Active	On-line	Private	172.30...	ip-102	kr-central2-a	2025-06-02	

퍼블릭 IP 연결

선택한 로드 밸런서에 퍼블릭 IP를 연결할 수 있습니다.

1

퍼블릭 IP를 연결하는 데는 보안과 안정성을 위해 일정 시간이 소요될 수 있습니다. 연결 시간은 요
청한 시점에 따라 다를 수 있으며, 몇 초에서 몇 분 사이에 완료됩니다.

로드 밸런서

kube_service_ko-edu-24-kakao-k8s-cluster_ingress-nginx_ingre
ss-nginx-controller_kr-central-2-a

프라이빗 IP

172.30.0.80

퍼블릭 IP 할당

● 새로운 퍼블릭 IP를 생성하고 자동으로 할당

○ 보유한 퍼블릭 IP를 목록에서 선택

퍼블릭 IP를 선택해 주세요.

취소

적용

로드 밸런서 이름 변경

퍼블릭 IP 연결

퍼블릭 IP 연결 해제

액세스 로그 설정

로드 밸런서 삭제

① 더보기 버튼 클릭

② 퍼블릭 IP 연결 클릭

③ 선택

©Kakao Enterprise Corp. All Rights Reserved.

57

실습 순서

- 1

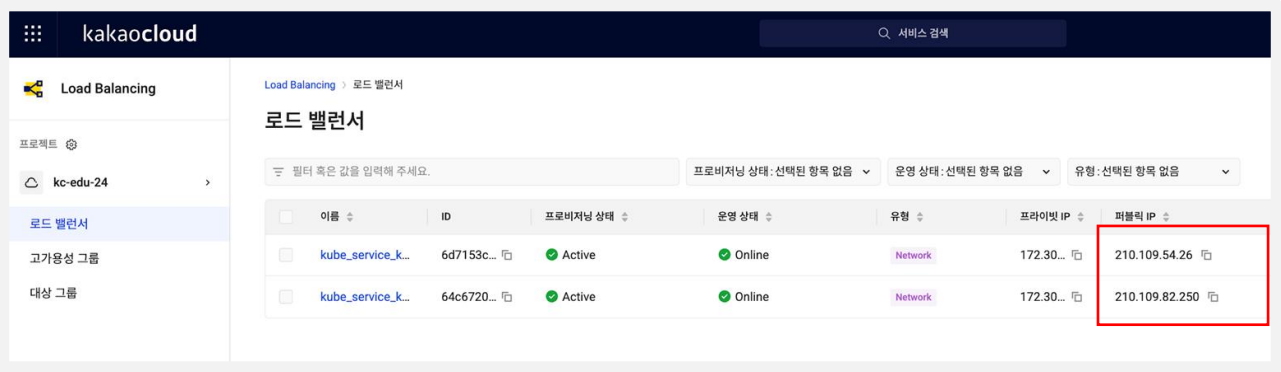
ingress-nginx
controller 배포
- 2

ingress-nginx 파드 및
서비스 상태 확인
- 3

로드 밸런서 확인

로드 밸런서 확인

4. 두 로드 밸런서의 퍼블릭 IP로 웹 브라우저에 접속



5. nginx에 의한 error 창 확인

○ingress 컨트롤러가 떠있는 nginx는 리버스 프록시 역할을 수행하기 때문에 error로 표시되는 것이 정상인 상태임

kr-central-2-a LoadBalancer 접속 확인



kr-central-2-b LoadBalancer 접속 확인



Lab6 : Kubernetes Engine 클러스터에 웹서버 수동 배포

이론 교재 94p

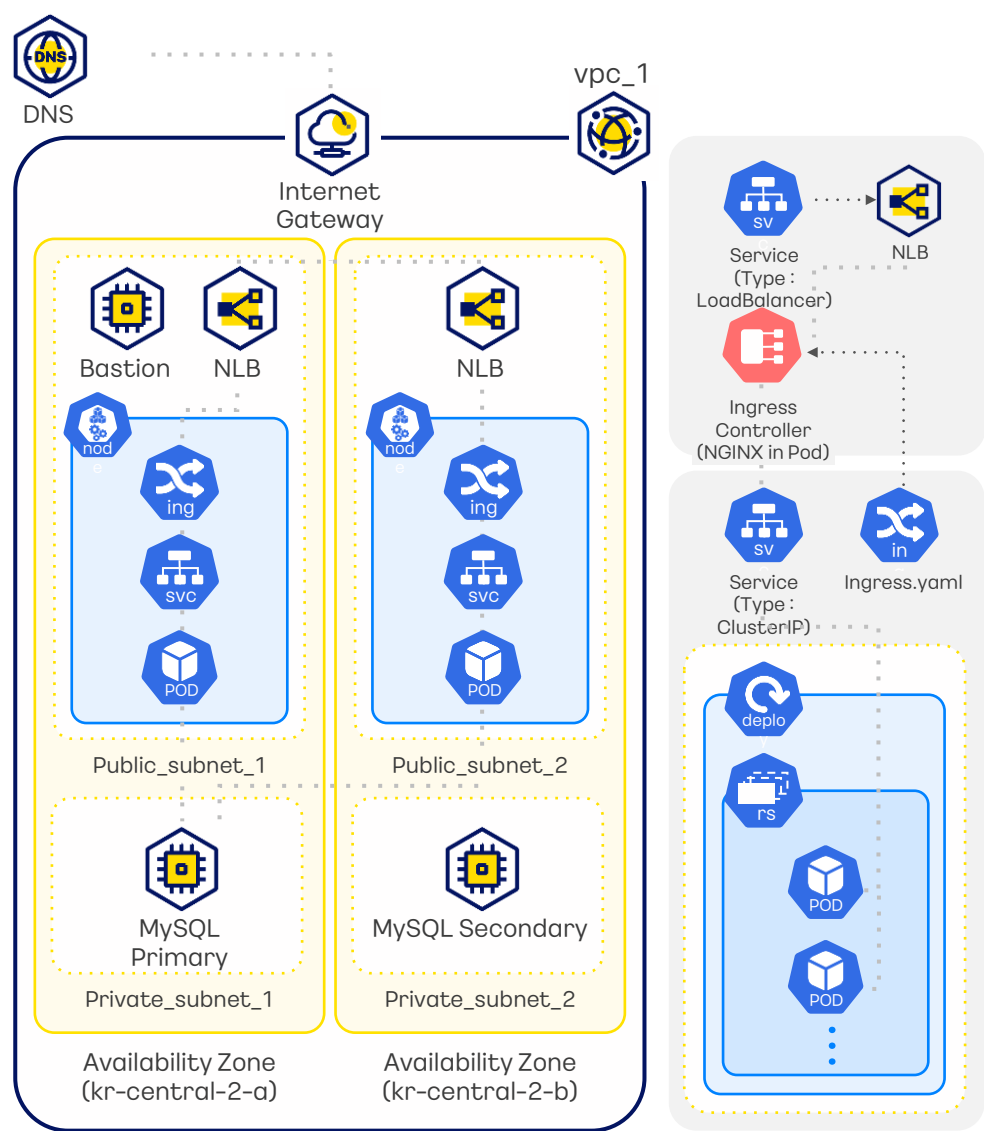


실습 내용

Spring application 배포를 위해서 다운 받은 YAML 파일을 확인 후 배포하고, 배포된 프로젝트를 웹에서 확인하는 실습입니다.

실습 순서

- 1 YAML 파일 다운로드 및 설정
- 2 YAML 파일 배포
- 3 배포한 프로젝트 웹에서 확인



실습 순서

1

YAML 파일 다운 및 설정

2

YAML 파일 배포

3

배포한 프로젝트 웹에서 확인

YAML 파일 다운 및 설정

1. 생성해 놓은 yaml 디렉터리 이동 및 확인

```
ubuntu@host-10-0-2-96:~/spring$ cd /home/ubuntu/yaml
ubuntu@host-10-0-2-96:~/yaml$ ls -al
total 12
drwxrwxrwx 2 root root 4096 Mar 1 13:37 .
drwxr-xr-x 9 ubuntu ubuntu 4096 Mar 1 15:19 ..
-rw-r--r-- 1 root root 4033 Mar 1 13:37 lab6-manifests.yaml
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab06 link](#))



1 `cd /home/ubuntu/yaml`

2 `ls -al`

2. lab6-manifests.yaml 확인

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab06 link](#))



`less lab6-manifests.yaml`

- 애플리케이션에 필요한 구성(config) 데이터를 저장하는 ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  WELCOME_MESSAGE: "Welcome to Kakao Cloud"
  BACKGROUND_COLOR: "#4a69bd"
---
```


YAML 파일 다운 및 설정

- SQL 스크립트를 포함하는 ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: sql-script
data:
  script.sql: |
    CREATE DATABASE IF NOT EXISTS history;
    USE history;
    CREATE TABLE IF NOT EXISTS access (
      id INT AUTO_INCREMENT PRIMARY KEY,
      date VARCHAR(255) NOT NULL
    );
    INSERT INTO access (date) VALUES ('hello:'));
    CALL mysql.mnms_grant_right_user('admin', '%', 'all', '*', '*');
    ALTER USER 'admin'@ '%' IDENTIFIED WITH mysql_native_password BY 'admin1234';
```

- 애플리케이션에서 사용하는 비밀 정보를 안전하게 저장하는 Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB1_PORT: 'MzMwNg=='
  DB1_URL: '"YXotYS5oeXVuZGlwMjAzLjNjNTk4ZmJmZGNIbD01MzY5MGFhMTEuMmYxMTYwY2Lm15c3FsLm1hbmFnZWQtc2VydmlJZS5rci1jZW50cmFsLTlua2FrYW9jbG91ZC5jb20="'
  DB1_ID: 'YWRTaW4='
  DB1_PW: 'YWRTaW4xMjM0'
  DB2_PORT: 'MzMwNg=='
  DB2_URL: '"YXotYi5oeXVuZGlwMjAzLjNjNTk4ZmJmZGNIbD01MzY5MGFhMTEuMmYxMTYwY2Lm15c3FsLm1hbmFnZWQtc2VydmlJZS5rci1jZW50cmFsLTlua2FrYW9jbG91ZC5jb20="'
  DB2_ID: 'YWRTaW4='
  DB2_PW: 'YWRTaW4xMjM0'
```

- SQL 스크립트를 실행하는 작업(Job)

```
apiVersion: batch/v1
kind: Job
metadata:
  name: sql-job
spec:
  template:
    spec:
      containers:
      - name: mysql
        image: mysql:5.7
        env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: DB1_PW
        - name: MYSQL_HOST
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: DB1_URL
        - name: MYSQL_PORT
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: DB1_PORT
        - name: MYSQL_USER
          valueFrom:
```

실습 순서

1

YAML 파일 다운 및 설정

2

YAML 파일 배포

3

배포한 프로젝트 웹에서 확인

YAML 파일 다운 및 설정

- 애플리케이션에 대한 외부 접근을 관리하는 Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kc-nginx-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: kc-spring-service
                port:
                  number: 80
```

- 애플리케이션의 내부 네트워크 서비스를 정의하는 Kubernetes 서비스

```
apiVersion: v1
kind: Service
metadata:
  name: kc-spring-service
spec:
  type: ClusterIP
  selector:
    app: kc-spring-demo
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

실습 순서

1

YAML 파일 다운 및 설정

2

YAML 파일 배포

3

배포한 프로젝트
웹에서 확인

YAML 파일 다운 및 설정

- 애플리케이션의 배포를 관리하는 Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-deployment
  labels:
    app: kc-spring-demo
spec:
  replicas: 2 # 복제본 갯수
  selector:
    matchLabels:
      app: kc-spring-demo
  template:
    metadata:
      labels:
        app: kc-spring-demo
    spec:
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchExpressions:
                    - key: app
                      operator: In
                      values:

```

•
•
•

실습 순서

1

YAML 파일 다 룬 및 설정

2

YAML 파 일 배포

3

배포한 프로젝트
웹에서 확인

YAML 파일 다운 및 설정

3. 컨테이너 레지스트리 인증을 위한 시크릿 키 생성 및 확인

- 사용 중인 Kubernetes에서 이미지를 가져오기(Pull) 위해서는 인증 설정이 필요함

```
ubuntu@host-172-16-1-159:~$ kubectl create secret docker-registry regcred \
> --docker-server=${PROJECT_NAME}.kr-central-2.kcr.dev \
> --docker-username=${ACC_KEY} \
> --docker-password=${SEC_KEY} \
> --docker-email=${EMAIL_ADDRESS}
secret/regcred created
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab06 link)



```
kubectl create secret docker-registry regcred \
--docker-server=${PROJECT_NAME}.kr-central-2.kcr.dev \
--docker-username=${ACC_KEY} \
--docker-password=${SEC_KEY} \
--docker-email=${EMAIL_ADDRESS}
```

```
ubuntu@host-172-30-0-225:~/yaml$ kubectl get secret regcred -o jsonpath='{.data.\.dockerconfigjson}' |
base64 --decode | jq
{
  "auths": {
    "kc-edu-40.kr-central-2.kcr.dev": {
      "username": "5d81149044674144873cda68b8c782d3",
      "password": "YjliZDE2YTBlkYjQ1MTBmZmQ2MzlmYmYzYzBlM2I3ZDdiODY0Nw==",
      "email": "yml@kcr.dev",
      "auth": "YjliZDE2YTBlkYjQ1MTBmZmQ2MzlmYmYzYzBlM2I3ZDdiODY0Nw=="
    }
  }
}
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab06 link)



```
kubectl get secret regcred -o jsonpath='{.data.\.dockerconfigjson}' | base64 --decode | jq
```

실습 순서

1

YAML 파일 다
운 및 설정

2

YAML 파
일 배포

3

배포한 프로젝트
웹에서 확인

YAML 파일 배포

1. 원활한 실습 진행을 위한 리소스 초기화

```
ubuntu@host-172-30-0-190:~/yaml$ kubectl delete -A ValidatingWebhookConfiguration ingress-nginx-admission
validatingwebhookconfiguration.admissionregistration.k8s.io "ingress-nginx-admission" deleted
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab06 link](#))



```
kubectl delete -A ValidatingWebhookConfiguration ingress-nginx-admission
```



2. YAML 파일 배포

```
ubuntu@host-10-0-2-135:~/yaml$ kubectl apply -f ./lab6-manifests.yaml
configmap/app-config created
configmap/sql-script created
secret/app-secret created
job.batch/sql-job created
ingress.networking.k8s.io/kc-nginx-ingress created
service/kc-spring-service created
deployment.apps/demo-deployment created
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab06 link](#))



```
kubectl apply -f ./lab6-manifests.yaml
```

실습 순서

1

YAML 파일 다
운 및 설정

2

YAML 파
일 배포

3

배포한 프로젝트
웹에서 확인

YAML 파일 배포

3. 배포한 내용 확인

```

ubuntu@host-172-30-0-190:~/yaml$ kubectl get all -o wide
NAME                                READY    STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE
READINESS GATES
pod/demo-deployment-b8849b6/b-m/mzb 1/1      Running   0           18m   192.168.29.134  host-172-30-3-26                    <none>
pod/demo-deployment-b8849b6/b-ntt9j 1/1      Running   0           18m   192.168.74.135  host-172-30-35-19                    <none>
pod/sql-job-z5mvll                    0/1      Completed 0           18m   192.168.29.133  host-172-30-3-26                    <none>

NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE   SELECTOR
service/kc-spring-service           ClusterIP           10.107.215.30 <none>         80/TCP     18m   app=kc-spring-demo
service/kubernetes                   ClusterIP           10.96.0.1     <none>         443/TCP    30h   <none>

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE   CONTAINERS    IMAGES
deployment.apps/demo-deployment      2/2      2              2            18m   kc-webserver   testtest.kr-central-2.kcr.dev/kakao-reg
istry/demo-spring-boot:1.0          app=kc-spring-demo

NAME                                DESIRED    CURRENT    READY    AGE   CONTAINERS    IMAGES
replicaset.apps/demo-deployment-b8849b6/b 2          2          2        18m   kc-webserver   testtest.kr-central-2.kcr.dev/kaka
o-registry/demo-spring-boot:1.0          app=kc-spring-demo,pod-template-hash=68849b67b

NAME                                COMPLETIONS    DURATION    AGE   CONTAINERS    IMAGES    SELECTOR
job.batch/sql-job 1/1            20s         18m   mysql         mysql:5.7  controller-uid=506feee0-dc7b-46d7-8f6c-8b56b6ae1ca4

```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab06 link](#))



```
kubectl get all -o wide
```



kubectl get all -o wide 결과에서 확인해야 할
점

◦Pod : 총 3개

- Spring App을 위한 pod 2개 (pod/demo-deployment) : **Running** 상태
- job을 위한 pod 1개 (pod/sql-job) : **Completed** 상태

◦Service : 총 2개

- Spring App을 위한 Service 1개 (service/kc-spring-service)
- 기본 Kubernetes Service 1개 (service/kubernetes)

◦Deployment : 총 1개

- Spring App을 위한 Deployment 1개 (deployment.apps/demo-deployment)

◦Replicaset : 총 1개

- Spring App을 위한 Replicaset 1개 (replicaset.apps/demo-deployment)

◦Job : 총 1개

- DB처리를 위한 Job 1개 (job.batch/sql-job)

실습 순서

1

YAML 파일 다
운 및 설정

2

YAML 파
일 배포

3

배포한 프로젝트
웹에서 확인

YAML 파일 배포

4. 배포한 내용 확인(Configmap, Secret)

◦ kubectl get all -o wide 명령어에서는 배포한 Configmap과 Secret의 내용이 표시되지 않음

◦ 확인을 위한 명령어를 입력하여 확인

```
ubuntu@host-172-30-0-190:~/yaml$ kubectl get configmap
NAME          DATA  AGE
app-config    2      54m
kube-root-ca.crt 1      30h
sql-script    1      54m
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab06 link](#))



```
kubectl get configmap
```



kubectl get configmap 결과에서 확인해야 할
점

◦ Configmap : 총 3개

- Spring App을 위한 configmap 1개 (app-config)
- DB설정을 위한 configmap 1개 (sql-script)
- 기본 configmap 1개 (kube-root-ca)

```
ubuntu@host-172-30-0-190:~/yaml$ kubectl get secret
NAME          TYPE          DATA  AGE
app-secret     Opaque        8      54m
default-token-vlg8l  kubernetes.io/service-account-token  3      30h
regcred       kubernetes.io/dockerconfigjson      1      101m
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab06 link](#))



```
kubectl get secret
```



kubectl get secret 결과에서 확인해야 할
점

◦ Secret : 총 3개

- Spring App을 위한 secret 1개 (app-secret)
- 컨테이너 레지스트리 인증을 위한 secret 1개 (regcred)

실습 순서

1

YAML 파일 다
운 및 설정

2

YAML 파
일 배포

3

배포한 프로젝트
웹에서 확인

잠깐 🍷Tip! : kubectl get ~ 결과가 사진과 다를 경우 수행 방법

Pod의 STATUS 가 사진과 다른 경우

NAME	NOMINATED NODE	READINESS GATES	READY	STATUS
pod/demo-deployment-844d78889f-2qxzq	<none>	<none>	1/1	Running
pod/demo-deployment-844d78889f-pg26q	<none>	<none>	0/1	ErrImagePull

◦ErrImagePull 인 경우

- 잘못된 이미지가 선택되어 배포된 경우 발생
- kubectl logs po {pod의 이름} 명령어를 통해 오류에 대한 정보를 확인할 수 있음
- kubectl describe {pod의 이름} 명령어를 통해 pod의 상태정보를 자세히 확인할 수 있음

NAME	NOMINATED NODE	READINESS GATES	READY	STATUS
pod/demo-deployment-844d78889f-2qxzq	<none>	<none>	1/1	Running
pod/demo-deployment-844d78889f-cvd9g	<none>	<none>	0/1	ImagePullBackOff

◦ImagePullBackOff인 경우

- image의 경로나 tag가 잘못 되어있는 경우 발생 -> Lab4의 실습을 다시 진행 (p.43)
- regcred 시크릿이 생성 되어있지 않은 경우 발생 -> Lab6의 YAML 파일 다운 및 설정 - p.64 진행

Configmap, Secret, job이 목록에 나타나지 않은 경우

- Configmap, secret, job의 경우 의존성 이슈로 인하여 꼭 순서대로 배포해야함
- Configmap -> ConfigmapDB -> Secret -> Job 순서로 다시 명령어 입력 진행 (p.64)
- 리소스 배포(apply) 직후 하나씩 확인(get) 해보기 (ex. kubectl get configmap, kubectl get secret ...)

Service, Deployment, Replicaset이 목록에 나타나지 않는 경우

- configmap, secret, job의 배포를 성공적으로 마친 후 진행해야 Service, Deployment, Replicaset이 원활하게 배포됨
- kubectl get ~ 명령어로 configmap, secret, job의 배포를 확인했다면, 목록에 나타나지 않은 리소스를 직접 배포(apply) 하기
(ex. kubectl apply -f lab6-Service.yaml ...) (p.65 2번 파일이름 확인하기)

실습 순서

1

YAML 파일 다
운 및 설정

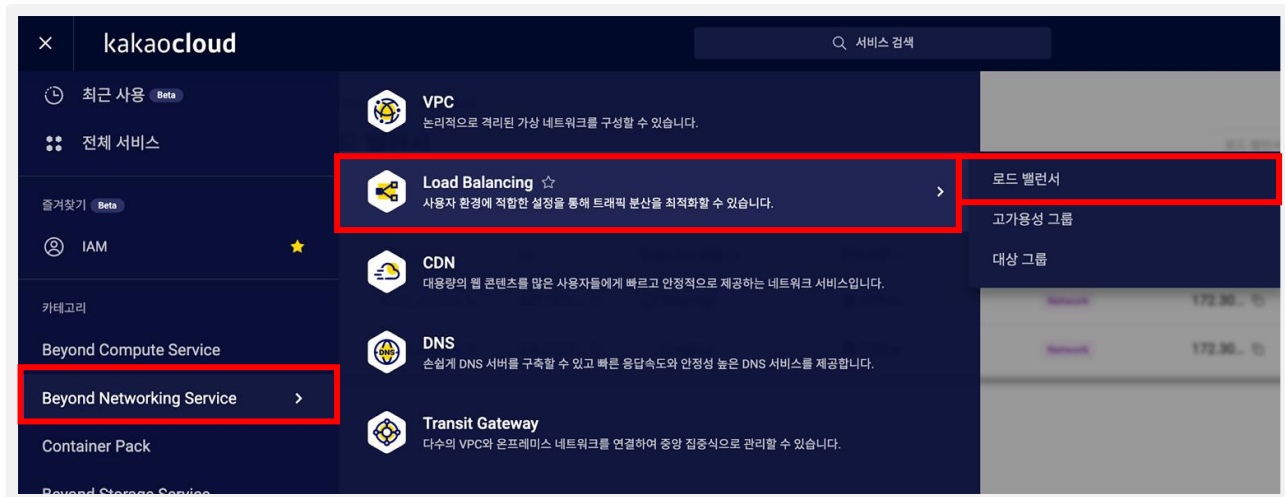
2

YAML 파
일 배포

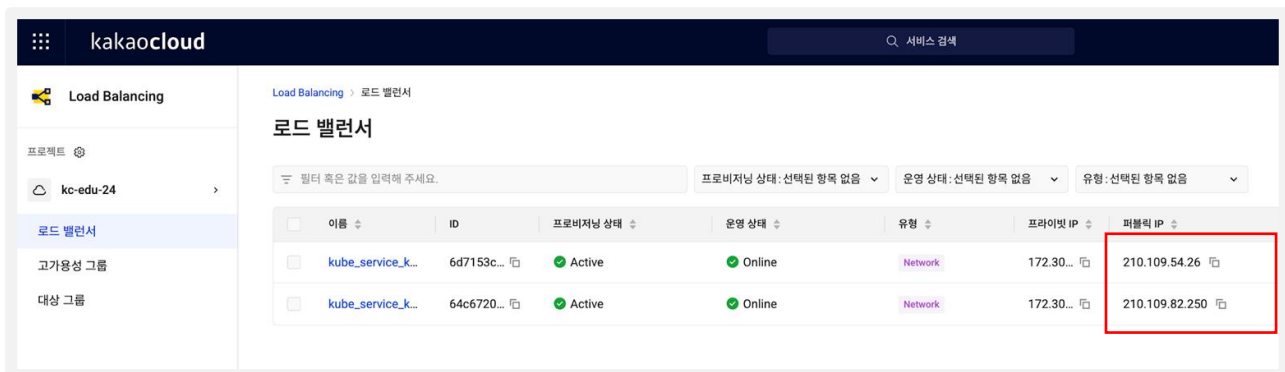
3

배포한 프로젝트
웹에서 확인

배포한 프로젝트 웹에서 확인

1. 카카오클라우드 로고 옆  버튼 클릭 > Beyond Networking Service > 로드 밸런서

2. 로드 밸런서의 퍼블릭 IP로 웹 브라우저에 접속



실습 순서

1

YAML 파일 다
운 및 설정

2

YAML 파
일 배포

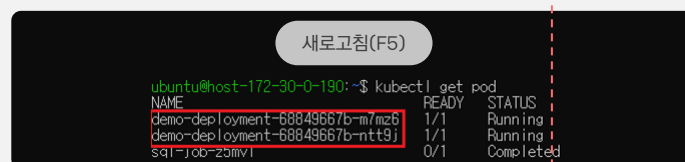
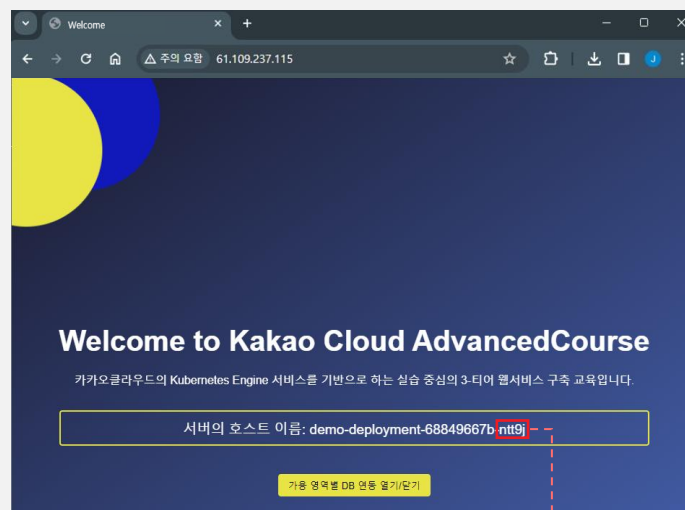
3

배포한 프로젝트
웹에서 확인

배포한 프로젝트 웹에서 확인

3. 배포한 프로젝트 구동 확인

kr-central-2-a LoadBalancer 접속 확인



◦ 새로 고침시 서버의 호스트 이름(Pod의 이름)이 번갈아 바뀌는 것을 볼 수 있음

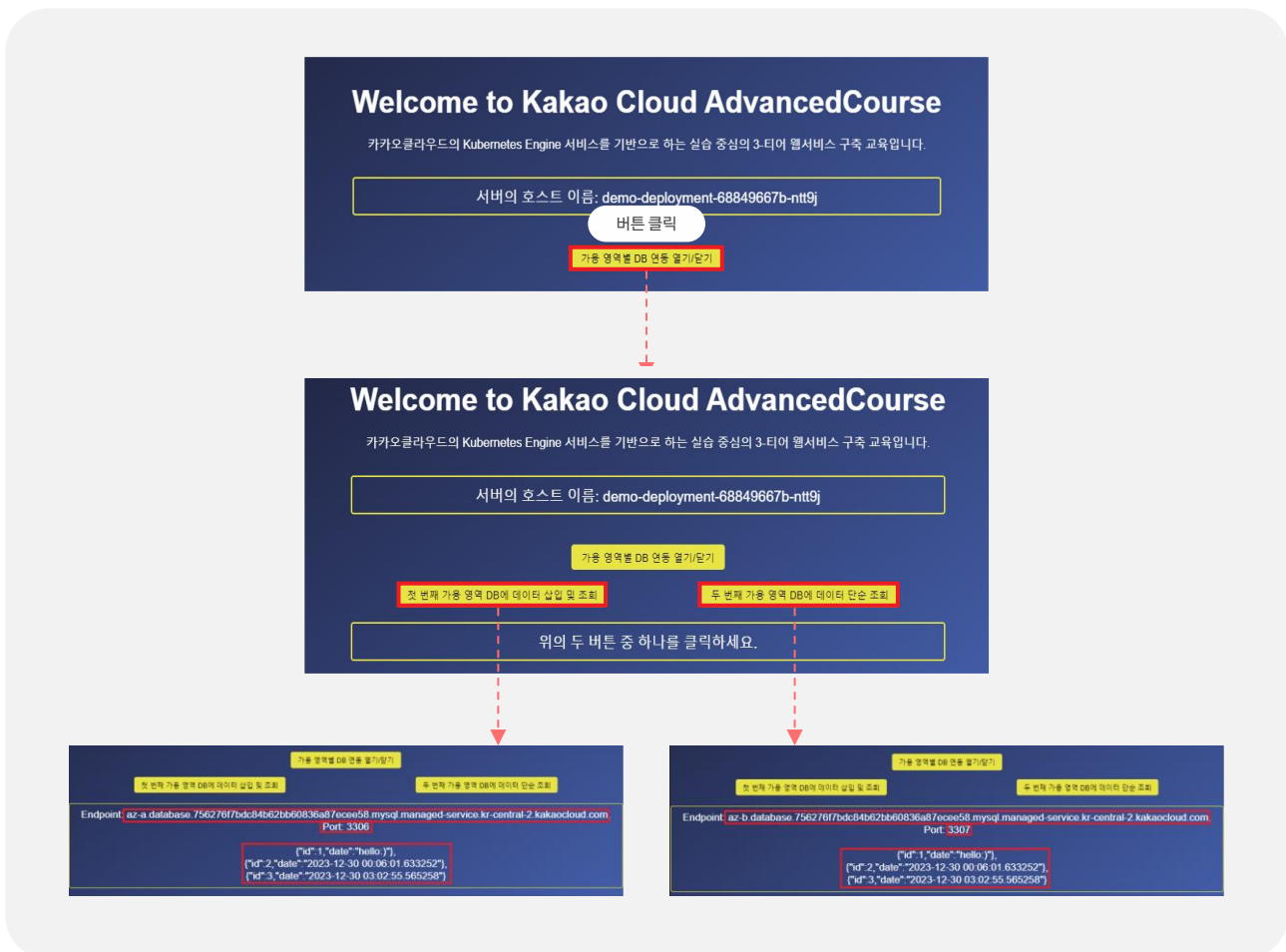
◦ 이는 이전에 배포했던 pod에 교차로 접속되고 있으며, 로드밸런싱이 잘 이루어짐을 알 수 있음

실습 순서

- 1 YAML 파일 다운로드 및 설정
- 2 YAML 파일 배포
- 3 배포한 프로젝트 웹에서 확인

배포한 프로젝트 웹에서 확인

3. 배포한 프로젝트 구동 확인



- 첫 번째 가용 영역 DB에 데이터 삽입 및 조회 버튼을 클릭하면 데이터가 추가되어 출력됨
- 첫 번째 DB는 AZ-a의 DB 엔드포인트에 3306포트로 연결된 것을 확인할 수 있음
- 두 번째 가용 영역 DB에 데이터 단순 조회 버튼을 클릭하면 데이터가 출력됨
- 두 번째 DB는 AZ_b의 DB 엔드포인트에 3307포트로 연결된 것을 확인할 수 있음
- 이는 AZ-a의 로드밸런서로 접근하더라도 다른 AZ의 DB에 접근이 가능함을 나타냄

실습 순서

1

YAML 파일 다
운 및 설정

2

YAML 파
일 배포

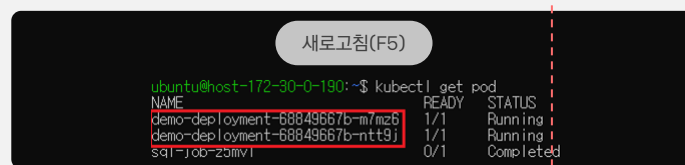
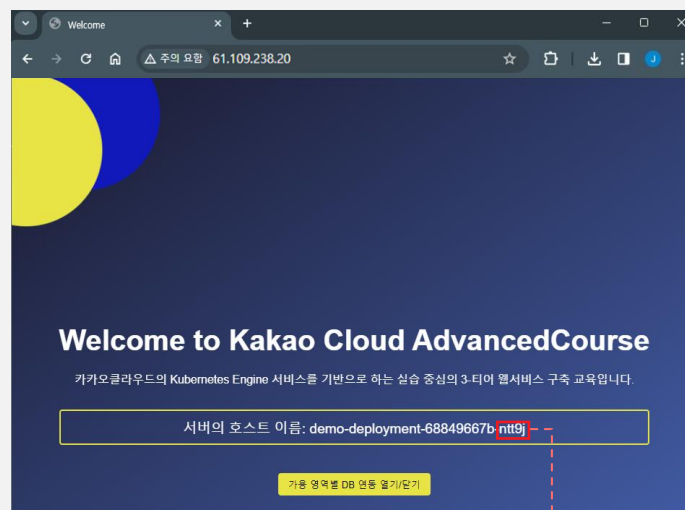
3

배포한 프로젝트
웹에서 확인

배포한 프로젝트 웹에서 확인

3. 배포한 프로젝트 구동 확인

kr-central-2-b LoadBalancer 접속 확인



◦kr-central-2-b LoadBalancer의 IP로 접속해도 이전과 동일하게 작동함을 볼 수 있음

◦이는 서로 다른 AZ에 같은 app이 성공적으로 배포되었음을 확인할 수 있음

Lab7 : Kubernetes Engine 활용하기

이론 교재 95p

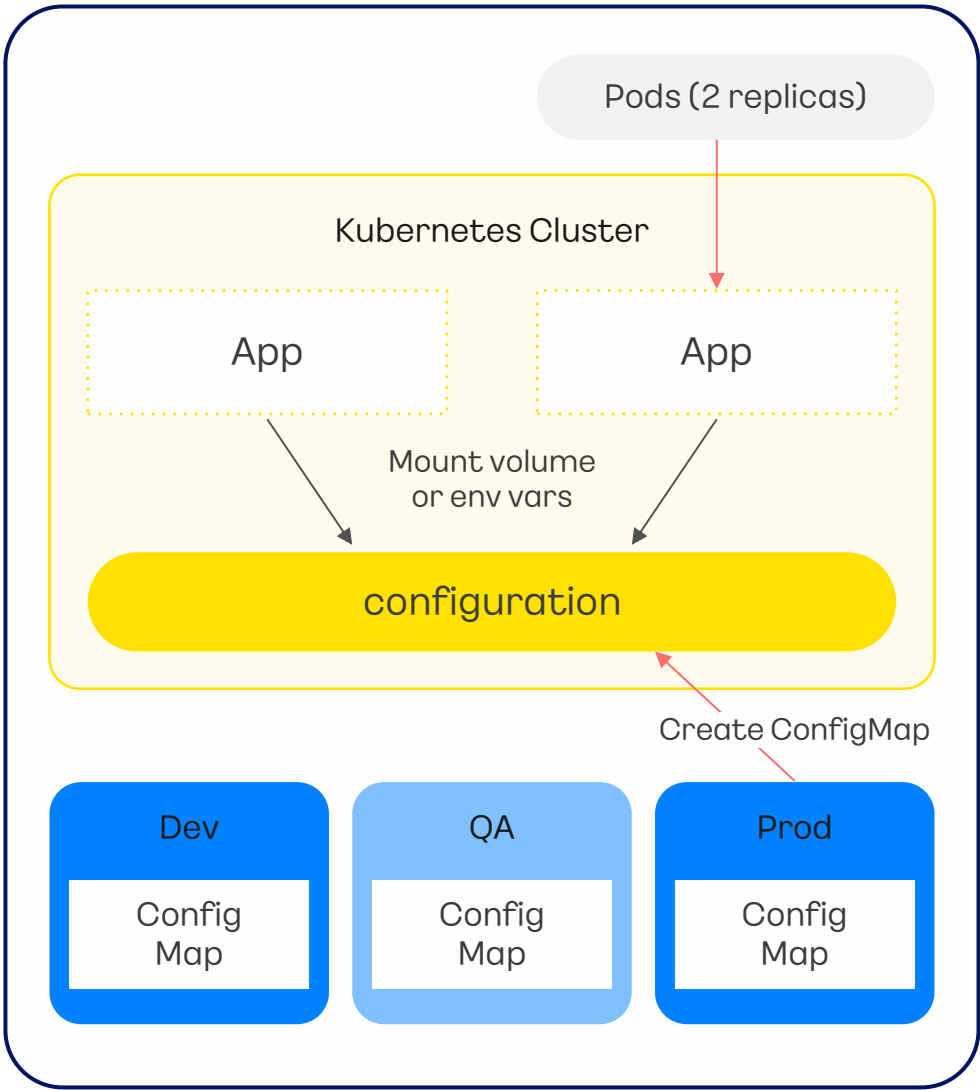


실습 내용

만들어진 웹을 ConfigMap을 변경해서 웹의 구조를 변경하고 확인합니다. 또한 Deployment 리소스의 replicas 값을 변경하여 Pod의 개수를 확인하는 실습과정을 진행합니다.

실습 순서

- 1 Deployment 리소스의 replicas 값 변경
- 2 YAML 파일을 이용해 배포된 내용 수정
- 3 YAML 파일 배포
- 4 변경된 내용 웹에서 확인



실습 순서

1

Deployment 리소스의
replicas 값 변경

2

YAML 파일을 이용해
배포된 내용 수정

3

YAML 파일 배포

4

변경된 내용 웹에
서 확인

Deployment 리소스의 replicas 값 변경

1. Pod들의 상태 변화 확인

```
munseongha@munseonghau-MacBookPro ~ % cd Downloads
munseongha@munseonghau-MacBookPro Downloads % ssh -i keypair.pem ubuntu@61.109.239.58
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-164-generic x86_64)
ubuntu@host-172-30-1-124:~$ kubectl get po -w
```

NAME	READY	STATUS	RESTARTS	AGE
demo-deployment-6558b4586-494ww	1/1	Running	0	28m
demo-deployment-6558b4586-bjm58	1/1	Running	0	28m
sql-job-h5swm	0/1	Completed	0	28m

아래의 스크립트를 복사 후 새로운 터미널 창에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 (Lab07 link)



- 1 `cd {keypair를 다운 받은 곳}`
- 2 `ssh -i keypair.pem ubuntu@{bastion의 public ip 주소}`
- 3 `kubectl get po -w`

2. Replicas 수 3개로 늘리기

```
ubuntu@host-172-30-1-124:~$ kubectl scale deployment demo-deployment --replicas=3
deployment.apps/demo-deployment scaled
ubuntu@host-172-30-1-124:~$
```

아래의 스크립트를 복사 후 기존 터미널 창에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 (Lab07 link)



```
kubectl scale deployment demo-deployment --replicas=3
```

3. Pod들의 상태 변화 확인

```
ubuntu@host-172-30-1-124:~$ kubectl get po -w
```

NAME	READY	STATUS	RESTARTS	AGE
demo-deployment-6558b4586-494ww	1/1	Running	0	28m
demo-deployment-6558b4586-bjm58	1/1	Running	0	28m
sql-job-h5swm	0/1	Completed	0	28m
demo-deployment-6558b4586-189c7	0/1	Pending	0	0s
demo-deployment-6558b4586-189c7	0/1	Pending	0	0s
demo-deployment-6558b4586-189c7	0/1	ContainerCreating	0	0s
demo-deployment-6558b4586-189c7	0/1	ContainerCreating	0	0s
demo-deployment-6558b4586-189c7	1/1	Running	0	0s

실습 순서

1

Deployment 리소스의 replicas 값 변경

2

YAML 파일을 이용해 배포된 내용 수정

3

YAML 파일 배포

4

변경된 내용 웹에서 확인

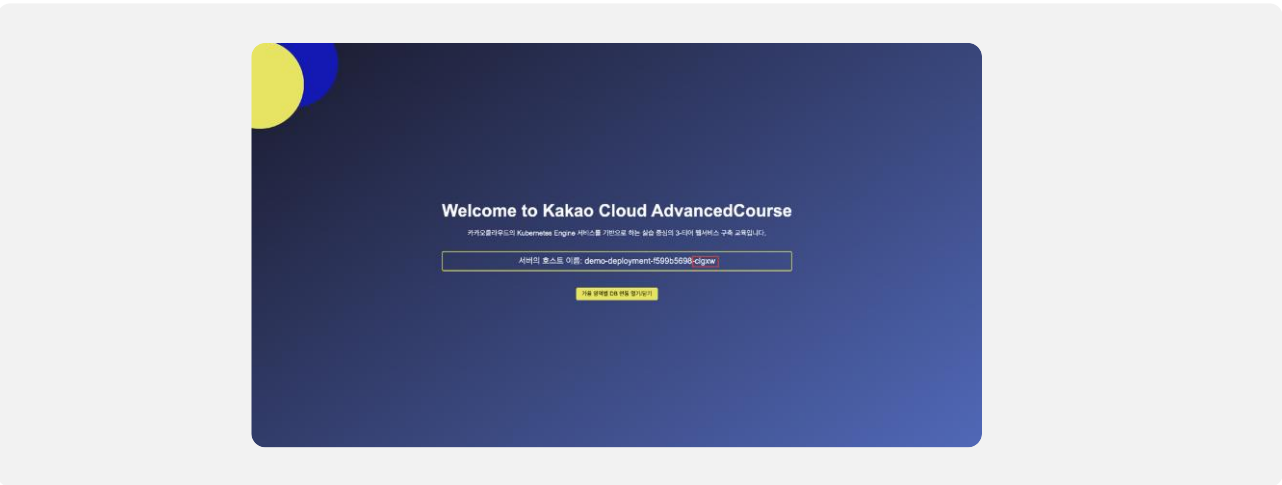
Deployment 리소스의 replicas 값 변경

!

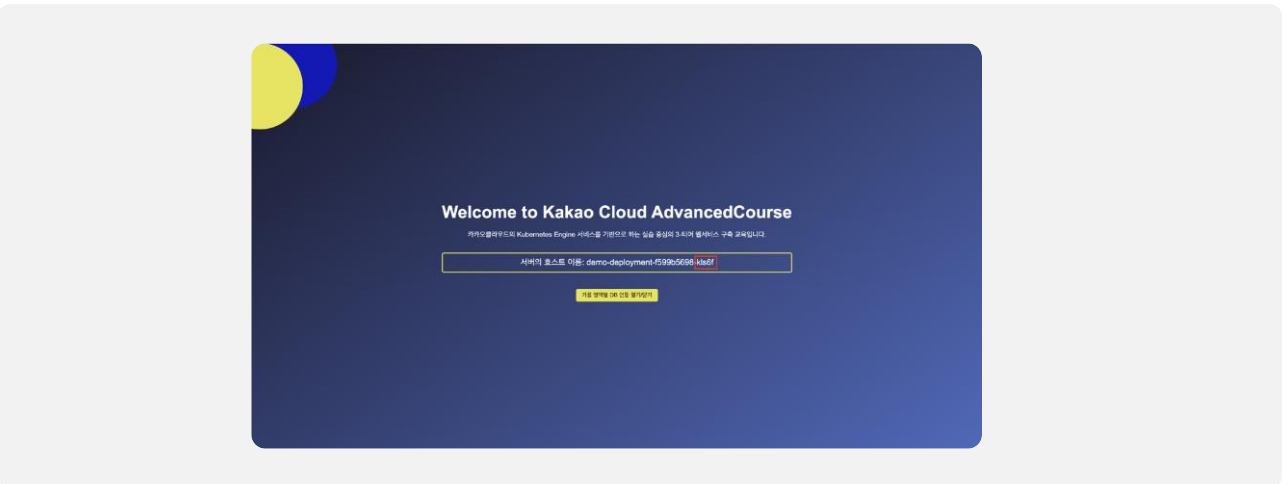
유의사항

서버의 호스트 이름이 세 개가 나와야 정상입니다.

4. 변경된 첫 번째 웹사이트 확인



5. 변경된 두 번째 웹사이트 확인



실습 순서

1

Deployment 리소스의 replicas 값 변경

2

YAML 파일을 이용해 배포된 내용 수정

3

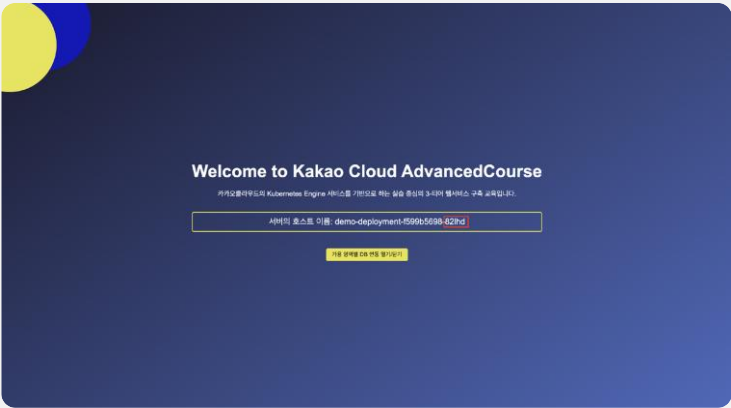
YAML 파일 배포

4

변경된 내용 웹에서 확인

Deployment 리소스의 replicas 값 변경

6. 변경된 세 번째 웹사이트 확인



실습 순서

1

Deployment 리소스의
replicas 값 변경

2

YAML 파일을 이용해
배포된 내용 수정

3

YAML 파일 배포

4

변경된 내용 웹에
서 확인

Deployment 리소스의 replicas 값 변경

7. Replicas 수 2개로 줄이기

```
ubuntu@host-172-30-1-124:~$ kubectl scale deployment demo-deployment --replicas=2
deployment.apps/demo-deployment scaled
ubuntu@host-172-30-1-124:~$
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab07 link](#))

```
kubectl scale deployment demo-deployment --replicas=2
```

8. Pod들의 상태 변화 확인

```
ubuntu@host-172-30-1-124:~$ kubectl get po -w
```

NAME	READY	STATUS	RESTARTS	AGE
demo-deployment-6558b4586-494ww	1/1	Running	0	28m
demo-deployment-6558b4586-bjm58	1/1	Running	0	28m
sql-job-h5swm	0/1	Completed	0	28m
demo-deployment-6558b4586-189c7	0/1	Pending	0	0s
demo-deployment-6558b4586-189c7	0/1	Pending	0	0s
demo-deployment-6558b4586-189c7	0/1	ContainerCreating	0	0s
demo-deployment-6558b4586-189c7	0/1	ContainerCreating	0	0s
demo-deployment-6558b4586-189c7	1/1	Running	0	0s
demo-deployment-6558b4586-189c7	1/1	Terminating	0	63s
demo-deployment-6558b4586-189c7	1/1	Terminating	0	64s
demo-deployment-6558b4586-189c7	0/1	Terminating	0	65s
demo-deployment-6558b4586-189c7	0/1	Terminating	0	65s
demo-deployment-6558b4586-189c7	0/1	Terminating	0	65s

실습 순서

1

Deployment 리소스의
replicas 값 변경

2

YAML 파일을 이용해
배포된 내용 수정

3

YAML 파일 배포

4

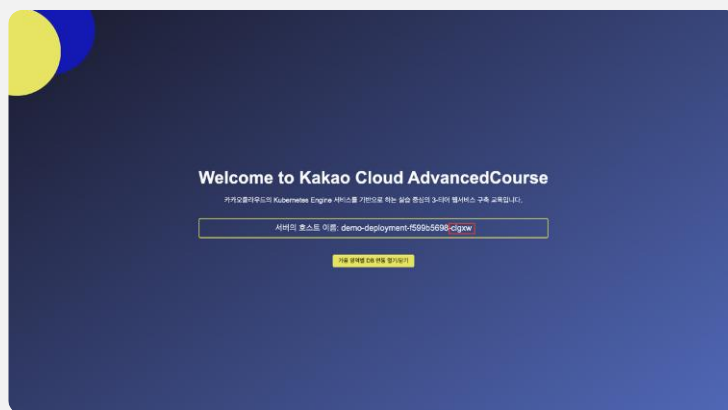
변경된 내용 웹에
서 확인

Deployment 리소스의 replicas 값 변경

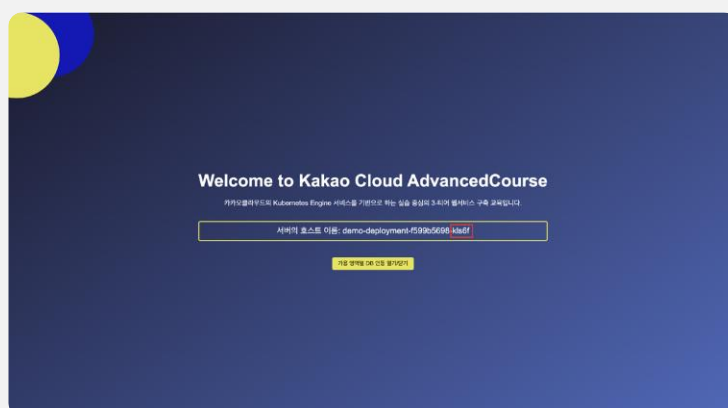
! 유의사항

서버의 호스트 이름이 두 개가 나와야 정상입니다.

9. 변경된 첫 번째 웹사이트 확인



10. 변경된 두 번째 웹사이트 확인



실습 순서

1

Deployment 리소스의
replicas 값 변경

2

YAML 파일을 이용해
배포된 내용 수정

3

YAML 파일 배포

4

변경된 내용 웹에
서 확인

YAML 파일을 이용해 배포된 내용 수정

1. lab6-manifests.yaml 파일 들어가기

```
ubuntu@host-10-0-2-135:~$ cd yaml
ubuntu@host-10-0-2-135:~/yaml$ sudo vi lab6-manifests.yaml
```

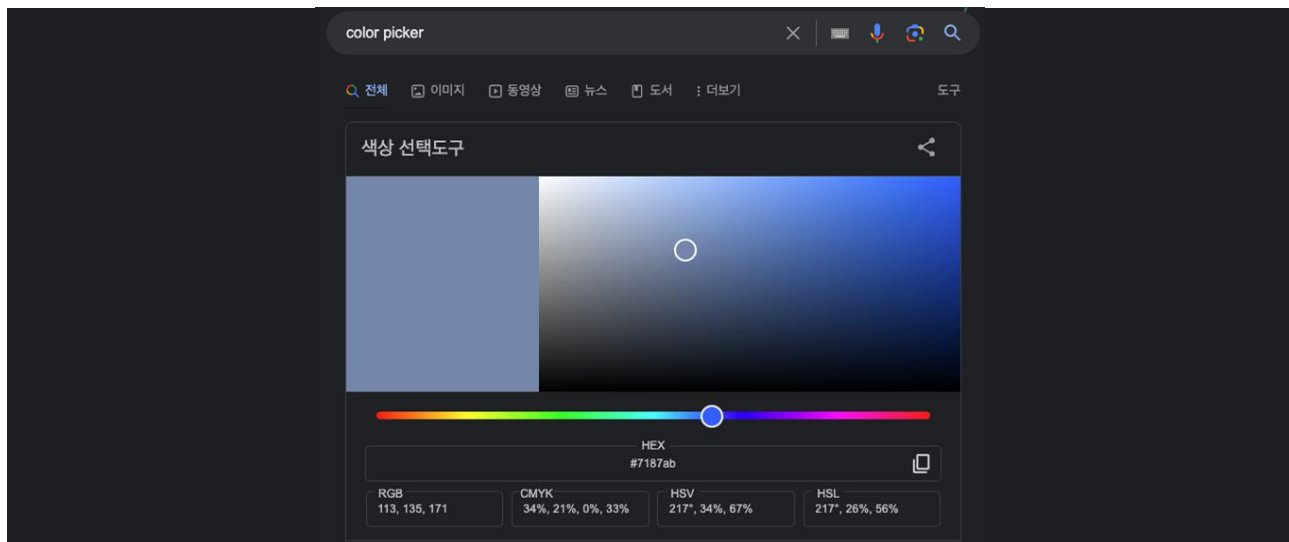
아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab07 link)



① `cd yaml`

② `sudo vi lab6-manifests.yaml`

2. Google에 color picker 검색 후 원하는 색상 HEX값 복사



3. ConfigMap.yaml 파일 내용 변경 후 저장

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  WELCOME_MESSAGE: "ConfigMap 변경 실습"
  BACKGROUND_COLOR: "#FFFF00"
~
~
~
:wq
```

- WELCOME_MESSAGE :
"ConfigMap 변경 실습"
- BACKGROUND_COLOR: "FFFF00"
- "Esc 버튼 + :wq + Enter 버튼"

실습 순서

1

Deployment 리소스의
replicas 값 변경

2

YAML 파일을 이용해
배포된 내용 수정

3

YAML 파일 배포

4

변경된 내용 웹에
서 확인

YAML 파일 배포

1. 변경된 yaml 파일 적용

```
ubuntu@host-10-0-2-135:~/yaml$ kubectl apply -f ./lab6-manifests.yaml
configmap/app-config configured
configmap/sql-script unchanged
secret/app-secret unchanged
job.batch/sql-job unchanged
ingress.networking.k8s.io/kc-nginx-ingress unchanged
service/kc-spring-service unchanged
deployment.apps/demo-deployment unchanged
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab07 link](#))



```
kubectl apply -f ./lab6-manifests.yaml
```

2. 실행 중인 Deployment를 재시작

```
ubuntu@host-10-0-2-135:~/yaml$ kubectl rollout restart deployment demo-deployment
deployment.apps/demo-deployment restarted
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab07 link](#))



```
kubectl rollout restart deployment demo-deployment
```

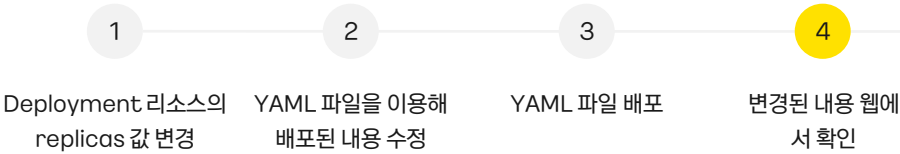
3. Pod들의 상태 변화 확인

```
ubuntu@host-172-30-1-124:~$ kubectl get po -w
```

NAME	READY	STATUS	RESTARTS	AGE
demo-deployment-6f56f4df47-r7jsq	1/1	Running	0	111s
demo-deployment-6f56f4df47-rs4vz	1/1	Running	0	112s
sql-job-h5swm	0/1	Completed	0	49m

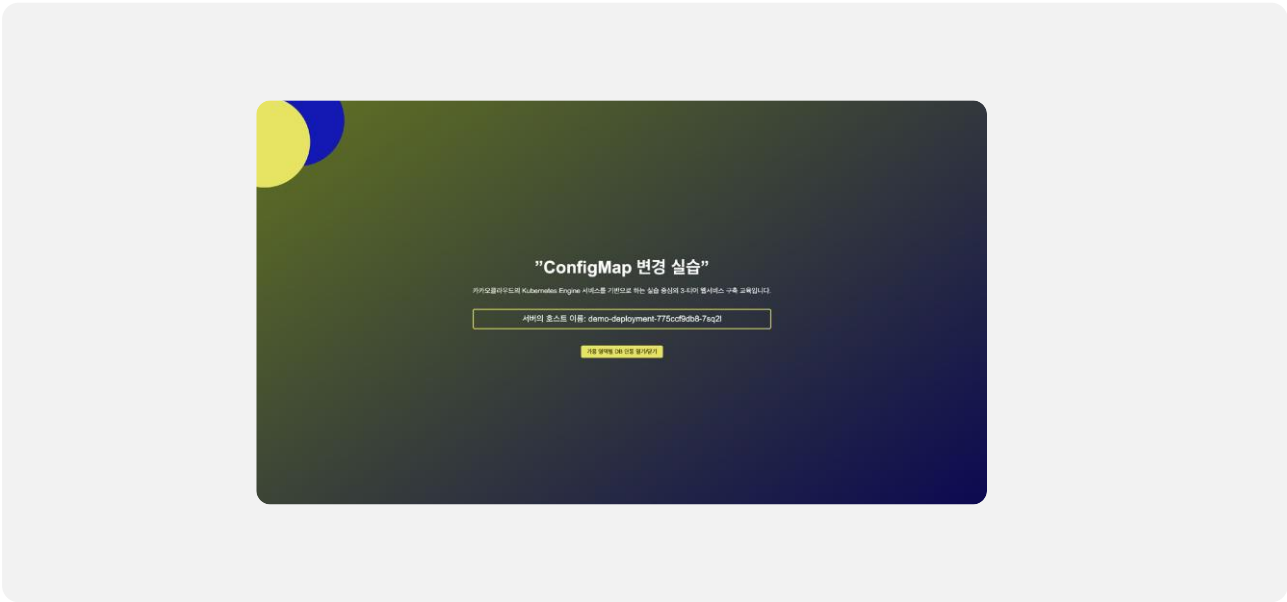

```
demo-deployment-7687d65695-6w55w 0/1 Pending 0 0s
demo-deployment-7687d65695-6w55w 0/1 Pending 0 0s
demo-deployment-7687d65695-6w55w 0/1 ContainerCreating 0 1s
demo-deployment-7687d65695-6w55w 0/1 ContainerCreating 0 1s
demo-deployment-7687d65695-6w55w 1/1 Running 0 2s
demo-deployment-6f56f4df47-rs4vz 1/1 Terminating 0 3m23s
demo-deployment-7687d65695-khlp5 0/1 Pending 0 0s
demo-deployment-7687d65695-khlp5 0/1 Pending 0 0s
demo-deployment-7687d65695-khlp5 0/1 ContainerCreating 0 0s
demo-deployment-6f56f4df47-rs4vz 1/1 Terminating 0 3m24s
demo-deployment-7687d65695-khlp5 0/1 ContainerCreating 0 1s
```

실습 순서

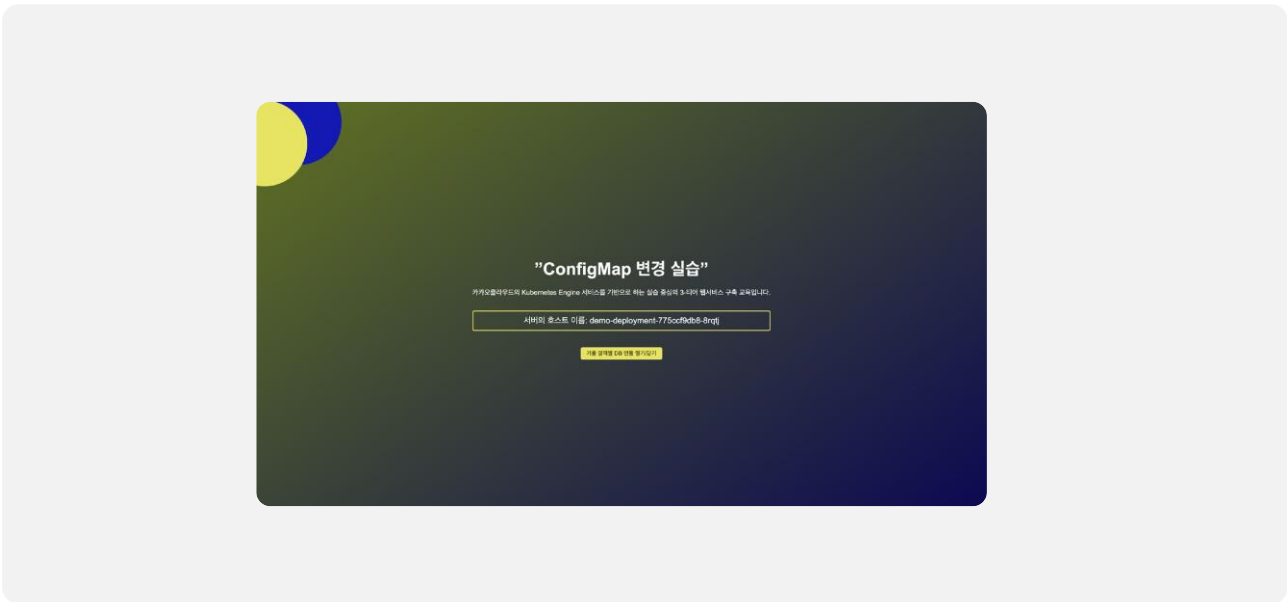


변경된 내용 웹에서 확인

1. 변경 된 첫 번째 웹사이트 확인



2. 변경 된 두 번째 웹사이트 확인



Lab8 :

Kubernetes Engine

클러스터에 웹서버 자동화 배포

이론 교재 96p



실습 내용

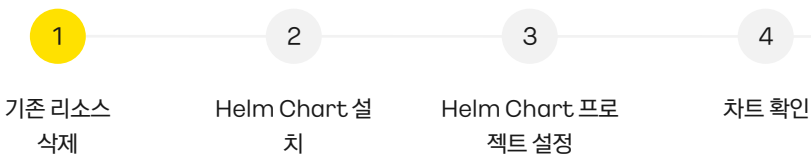
지금까지 만든 배포 방식을 자동화하는 도구인 Helm에 대해 실습합니다. Helm을 이용해 Chart를 만들어 배포, 업데이트, 롤백을 진행하는 실습입니다.

실습 순서

- | | | | | | |
|---|-----------------------|---|---------------|---|---------------------|
| 1 | 기존 리소스 삭제 | 2 | Helm Chart 설치 | 3 | Helm Chart 프로젝트 설정 |
| 4 | 차트 확인 | 5 | 차트 문제 검사 | 6 | 차트 설치 시뮬레이션 및 차트 설치 |
| 7 | Helm Chart를 이용한 버전 관리 | 8 | 롤백 | | |



실습 순서



기존 리소스 삭제

1. yaml 파일 삭제

```
ubuntu@host-10-0-2-135: ~/yaml$ rm -f lab6-manifests.yaml
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
rm -f lab6-manifests.yaml
```

2. 실행 중인 리소스 삭제

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



1

```
kubectl delete ingress --all
```

2

```
kubectl delete svc --all
```

3

```
kubectl delete deploy --all
```

4

```
kubectl delete job sql-job
```

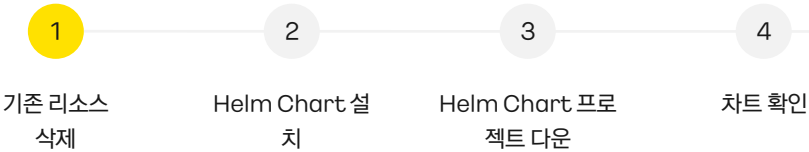
5

```
kubectl delete secret app-secret
```

6

```
kubectl delete configmap --all
```

실습 순서



기존 리소스 삭제

3. 실행 중인 리소스가 삭제되었는지 확인

service/kubernetes는 자동 생성되는 리소스로, 재생성되어도 무관합니다.

```
ubuntu@host-172-30-0-190:~$ kubectl get ingress
No resources found in default namespace.
ubuntu@host-172-30-0-190:~$ kubectl get svc
\NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes ClusterIP   10.96.0.1     <none>        443/TCP     5m32s
ubuntu@host-172-30-0-190:~$ kubectl get deploy
No resources found in default namespace.
ubuntu@host-172-30-0-190:~$ kubectl get job
No resources found in default namespace.
ubuntu@host-172-30-0-190:~$ kubectl get configmap
NAME      DATA  AGE
kube-root-ca.crt  1      4m16s
ubuntu@host-172-30-0-190:~$ kubectl get secret
NAME                                TYPE                                  DATA  AGE
default-token-vlg8l                kubernetes.io/service-account-token    3      3d4h
regcred                            kubernetes.io/dockerconfigjson        1      47h
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



1

`kubectl get ingress`

2

`kubectl get svc`

3

`kubectl get deploy`

4

`kubectl get po`

5

`kubectl get job`

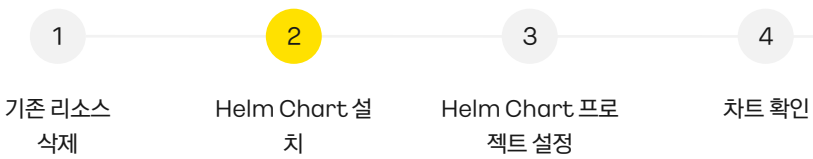
6

`kubectl get configmap`

7

`kubectl get secret`

실습 순서



Helm Chart 설치

1. Helm Chart 설치

```
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100 11664  100 11664    0     0  61068      0 --:--:-- --:--:-- --:--:-- 61068
Downloading https://get.helm.sh/helm-v3.13.1-linux-amd64.tar.gz
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | sudo bash
```

2. 설치 확인

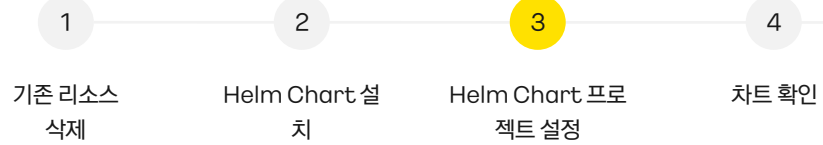
```
ubuntu@host-172-30-1-102:~$ helm version
version.BuildInfo{Version:"v3.13.1", GitCommit:"3547a4b5bf5edb5478ce352e18858d8a552a4110",
rsion:"go1.20.8"}
ubuntu@host-172-30-1-102:~$
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm version
```

실습 순서



Helm Chart 프로젝트 설정

1. 실습 진행을 위한 디렉터리 이동

```
ubuntu@host-10-0-2-135:~/yaml$ cd /home/ubuntu/tutorial/AdvancedCourse/src/helm
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
cd /home/ubuntu/tutorial/AdvancedCourse/src/helm
```

2. 미리 생성해 놓은 values.yaml 파일 helm 디렉터리로 이동

```
ubuntu@host-10-0-2-135:~/tutorial/AdvancedCourse/src/helm$ sudo mv /home/ubuntu/values.yaml values.yaml
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
sudo mv /home/ubuntu/values.yaml values.yaml
```

실습 순서

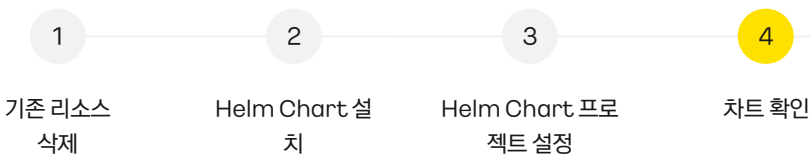


차트 확인

1. tree를 이용해 차트 확인

```
ubuntu@host-172-30-0-225:~/tutorial/AdvancedCourse/src/helm$ tree .
.
├── Chart.yaml
├── files
│   └── sql
│       └── script.sql
├── templates
│   ├── _helpers.tpl
│   ├── configmap.yaml
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── job.yaml
│   ├── secret.yaml
│   └── service.yaml
└── values.yaml

3 directories, 11 files
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
tree .
```

실습 순서

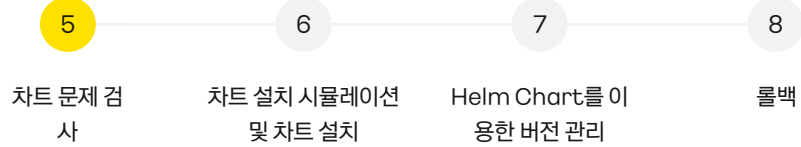


차트 문제 검사

1. lint 기능을 이용해 차트에 문제가 있는지 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm lint .
==> Linting .
[INFO] Chart.yaml: icon is recommended
1 chart(s) linted, 0 chart(s) failed
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm lint .
```

실습 순서

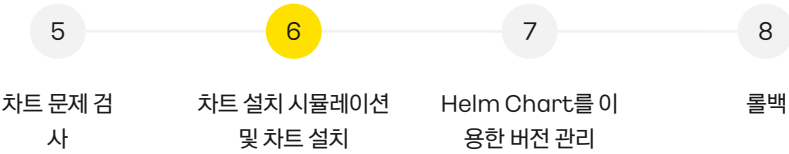


차트 설치 시뮬레이션 및 차트 설치

1. 차트 설치 전 랜더링 테스트

```
buntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm template . -f values.yaml
---
# Source: kc-spring-demo/templates/secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: release-name-secret
type: Opaque
data:
  DB1_PORT: "MzMwNg=="
  DB1_URL: "YXotYS5kYXRhYmFzZS4zYzllODk5YWl1ODAwYjc4ODU4YmI5ZDhlOTFiMzFkYS5teXNxbC5tYW5hZ2VkLXN
  Iua2FrYW9jbG91ZC5jb20="
  DB1_ID: "YWRTaW4="
  DB1_PW: "YWRTaW4xMjM0"
  DB2_PORT: "MzMwNg=="
  DB2_URL: "YXotYi5kYXRhYmFzZS4zYzllODk5YWl1ODAwYjc4ODU4YmI5ZDhlOTFiMzFkYS5teXNxbC5tYW5hZ2VkLXN
  Iua2FrYW9jbG91ZC5jb20="
  DB2_ID: "YWRTaW4="
  DB2_PW: "YWRTaW4xMjM0"
---
# Source: kc-spring-demo/templates/configmap.yaml
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm template . -f values.yaml
```

실습 순서

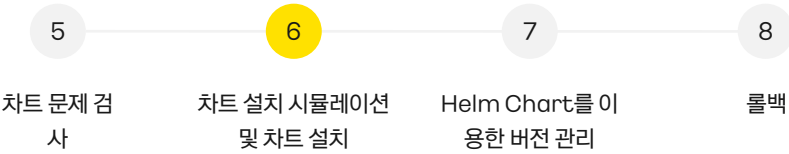


차트 설치 시뮬레이션 및 차트 설치

2. 디버그

```
ubuntu@host-10-0-2-96:~/tutorial/AdvancedCourse/src/helm$ helm install --dry-run --debug my-release . | tee ~/yamls
install.go:214: [debug] Original chart version: ""
install.go:231: [debug] CHART PATH: /home/ubuntu/tutorial/AdvancedCourse/src/helm
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm install --dry-run --debug my-release . | tee ~/yamls
```

◦차트의 설치가 클러스터에서 어떻게 이루어질지 시뮬레이션하고 상세한 디버그 정보를 제공함

3. 차트 설치

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm install my-release . -f values.yaml
NAME: my-release
LAST DEPLOYED: Fri Dec 29 18:26:16 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm install my-release . -f values.yaml
```

실습 순서

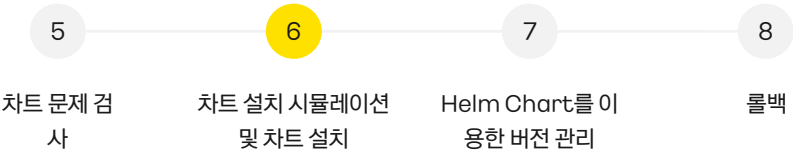


차트 설치 시뮬레이션 및 차트 설치

4. 차트 확인

! 유의사항

my-release 이름으로 차트가 생성되었는지 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm list
NAME          APP VERSION      NAMESPACE   REVISION   UPDATED
my-release    2023-12-29 18:26:16.0119068
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
helm list
```

5. 차트 세부 내용 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm status my-release
NAME: my-release
LAST DEPLOYED: Fri Dec 29 18:26:16 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
helm status my-release
```

실습 순서

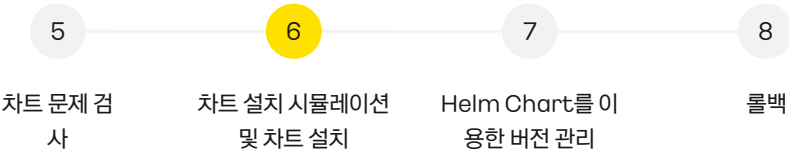


차트 설치 시뮬레이션 및 차트 설치

6. 파드 상태 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
my-release-kc-spring-demo-5f78677dc-5nnz6	1/1	Running	0	8m27s
my-release-kc-spring-demo-5f78677dc-gbrh6	1/1	Running	0	8m27s

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)

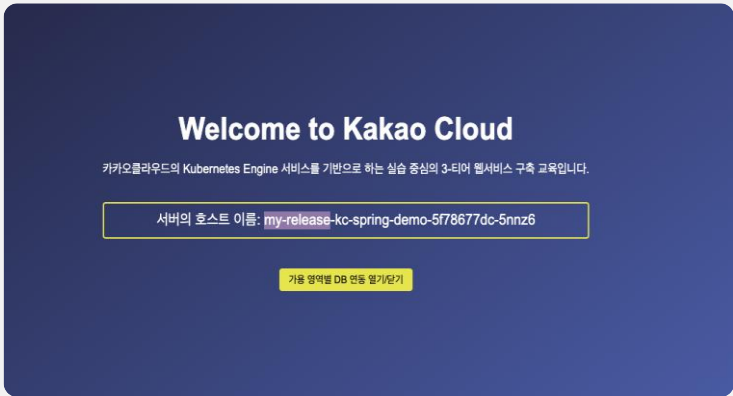


```
kubectl get all
```

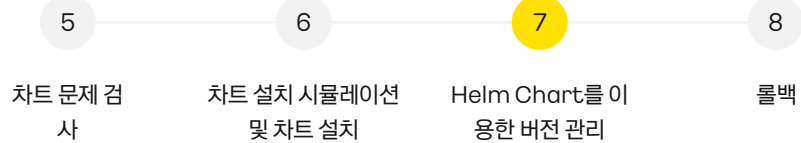
7. 웹 사이트 배포 상태 확인

! 유의사항

my-release-kc-spring-demo-... 이름으로 서버 호스트 이름이 변경되었는지 확인



실습 순서



Helm Chart를 이용한 버전 관리

1. replicaCount 수정

1라인 : replicaCount : 2



2 → 3 수정

저장



ESC 버튼 + :wq + Enter 버튼 입력

```

replicaCount: 3

deployment:
  repository: kakaocloud-k8s.kr-central-2.kcr.dev/kakao
  tag: "1.0"
  pullSecret: regcred

service:
  type: ClusterIP
  port: 80
  targetPort: 8080

```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
 *Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
sudo vi values.yaml
```

2. helm upgrade를 통한 차트 릴리즈 업그레이드

```

ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm upgrade my-release . --description "#pod 2->3" -f values.yaml
Release "my-release" has been upgraded. Happy Helming!
NAME: my-release
LAST DEPLOYED: Fri Dec 29 18:30:12 2023
NAMESPACE: default
STATUS: deployed
REVISION: 2
TEST SUITE: None

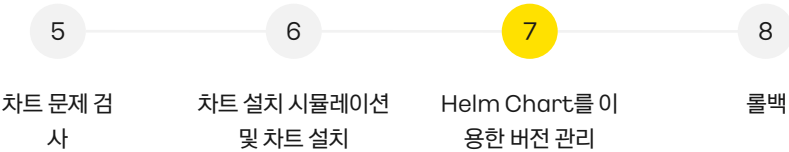
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
 *Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm upgrade my-release . --description "#pod 2->3" -f values.yaml
```

실습 순서



Helm Chart를 이용한 버전 관리

3. 업데이트 확인하기 - CHART REVISION 값 변경 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm status my-release
```

NAME: my-release
LAST DEPLOYED: Fri Dec 29 18:36:05 2023
NAMESPACE: default
STATUS: deployed
REVISION: 2
TEST SUITE: None

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
helm status my-release
```

4. 업데이트 확인하기 - Pod 개수 변경 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
my-release-kc-spring-demo-5f78677dc-25zcx	1/1	Running	0	108s
my-release-kc-spring-demo-5f78677dc-618zj	1/1	Running	0	108s
my-release-kc-spring-demo-5f78677dc-vg8cq	1/1	Running	0	108s

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
kubectl get pod
```

5. 릴리즈 히스토리 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm history my-release
```

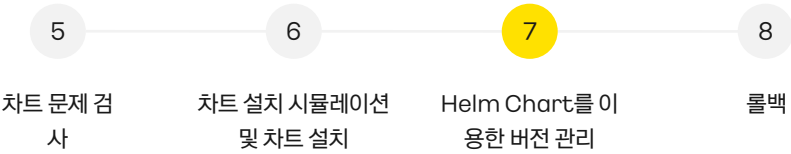
REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
1	Fri Dec 29 18:35:48 2023	superseded	kc-spring-demo-1.0.0		Install complete
2	Fri Dec 29 18:36:05 2023	superseded	kc-spring-demo-1.0.0		#pod 2->3

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
helm history my-release
```

실습 순서



Helm Chart를 이용한 버전 관리

6. 특정 REVISION의 value 값 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm get values my-release --revision 1

USER-SUPPLIED VALUES:
configMap:
  BACKGROUND_COLOR: '#4a69bd'
  WELCOME_MESSAGE: Welcome to Kakao Cloud
deployment:
  pullSecret: regcred
  repository: kakaocloud-k8s.kr-central-2.kcr.dev/kakao-registry/demo-spring-boot
  tag: "1.0"
hpa:
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm get values my-release --revision 1
```

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm get values my-release --revision 2

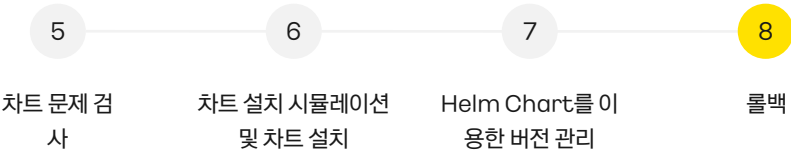
USER-SUPPLIED VALUES:
configMap:
  BACKGROUND_COLOR: '#4a69bd'
  WELCOME_MESSAGE: Welcome to Kakao Cloud
deployment:
  pullSecret: regcred
  repository: kakaocloud-k8s.kr-central-2.kcr.dev/kakao-registry/demo-spring-boot
  tag: "1.0"
hpa:
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab08 link](#))



```
helm get values my-release --revision 2
```

실습 순서



롤백

1. 롤백

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm rollback my-release 1
Rollback was a success! Happy Helming!
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
helm rollback my-release 1
```

2. 릴리즈 상태 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm history my-release
```

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
1	Fri Dec 29 18:35:48 2023	superseded	kc-spring-demo-1.0.0		Install complete
2	Fri Dec 29 18:36:05 2023	superseded	kc-spring-demo-1.0.0		#pod 2->3
3	Fri Dec 29 18:40:44 2023	deployed	kc-spring-demo-1.0.0		Rollback to 1

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
helm history my-release
```

3. 파드 수 감소 확인

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
my-release-kc-spring-demo-5f78677dc-25zcx	1/1	Running	0	9m4s
my-release-kc-spring-demo-5f78677dc-6l8zj	1/1	Running	0	9m4s

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab08 link)



```
kubectl get po
```

Lab9 :

HPA(Horizontal Pod Autoscaler)

테스트

이론 교재 97p

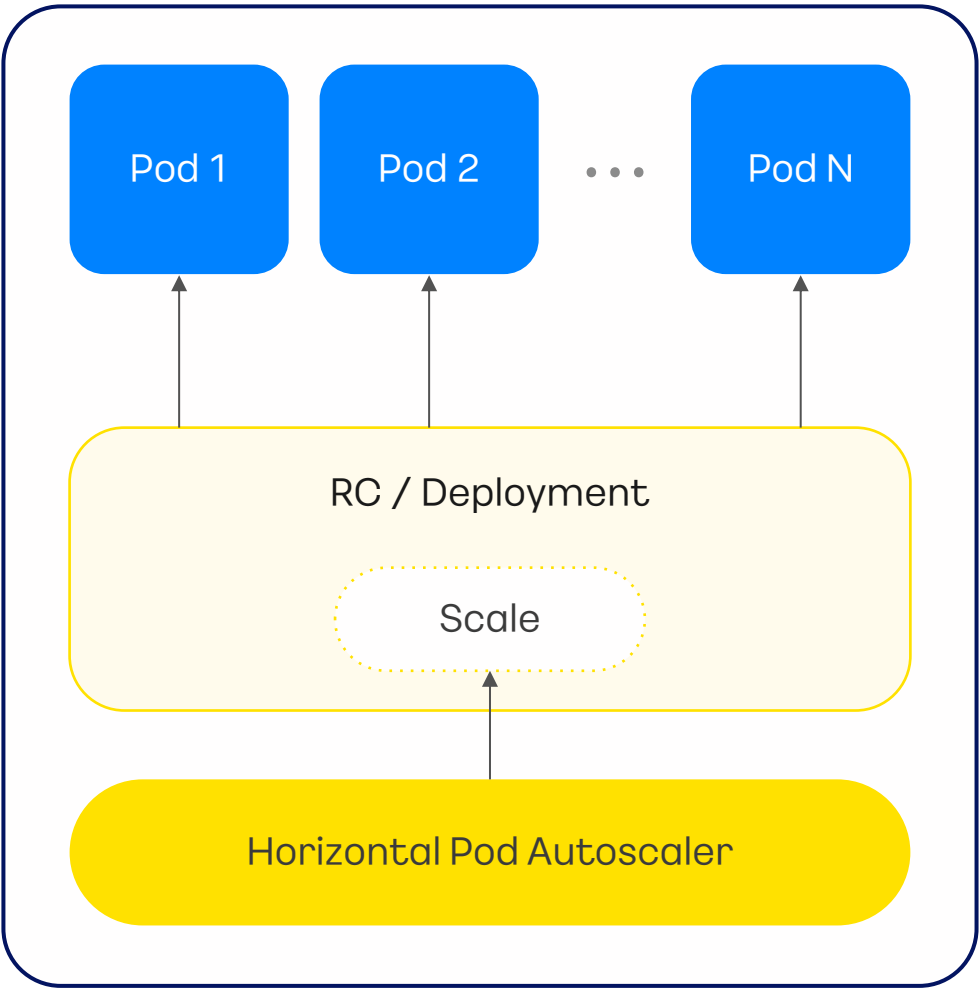


실습 내용

HPA 옵션을 주어 워크로드 리소스를 자동으로 증가시키는 오토스케일링 실습을 진행합니다.

실습 순서

- 1 HPA 설정
- 2 CPU 부하 발생 및 기능 확인



실습 순서

1

HPA 설정

2

CPU 부하 발생 및 기
능 확인

HPA 옵션 추가

1. metrics-server 저장소 추가

```
ubuntu@host-172-30-0-64: ~/yaml/tutorial/AdvancedCourse/src/helm$ helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
WARNING: Kubernetes configuration file is group-readable. This is insecure. Location: /home/ubuntu/.kube/config
WARNING: Kubernetes configuration file is world-readable. This is insecure. Location: /home/ubuntu/.kube/config
"metrics-server" has been added to your repositories
ubuntu@host-172-30-0-64: ~/yaml/tutorial/AdvancedCourse/src/helm$
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab09 link](#))

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
```

2. 노드의 리소스 사용량을 모니터링하는 metrics-server 설치

```
ubuntu@host-172-30-0-190: ~/yaml/tutorial/AdvancedCourse/src/helm$ helm upgrade --install metrics-server metrics-server/metrics-server --set hostNetwork.enabled=true --set containerPort=4443
WARNING: Kubernetes configuration file is group-readable. This is insecure. Location: /home/ubuntu/.kube/config
WARNING: Kubernetes configuration file is world-readable. This is insecure. Location: /home/ubuntu/.kube/config
Release "metrics-server" does not exist. Installing it now.
NAME: metrics-server
LAST DEPLOYED: Sun Dec 31 12:20:49 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
*****
* Metrics Server *
*****
Chart version: 3.11.0
App version: 0.6.4
Image tag: registry.k8s.io/metrics-server/metrics-server:v0.6.4
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab09 link](#))

```
helm upgrade --install metrics-server metrics-server/metrics-server --set hostNetwork.enabled=true --set containerPort=4443
```

실습 순서

1

HPA 설정

2

CPU 부하 발생 및 기
능 확인

HPA 옵션 추가

3. hpa.enabled 수정

40라인 : enabled: true



false -> true 수정

43라인 : averageUtilization: 1



10 -> 1 수정

```
hpa:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
  averageUtilization: 1
-- INSERT --
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab09 link](#))



```
sudo vi values.yaml
```

4. helm upgrade를 통한 차트 릴리즈 업그레이드

```
ubuntu@host-172-30-1-102:~/tutorial/AdvancedCourse/src/helm$ helm upgrade my-release . --description "enable hpa" -f values.yaml
Release "my-release" has been upgraded. Happy Helming!
NAME: my-release
LAST DEPLOYED: Sun Dec 31 03:53:12 2023
NAMESPACE: default
STATUS: deployed
REVISION: 4
TEST SUITE: None
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab09 link](#))



```
helm upgrade my-release . --description "enable hpa" -f values.yaml
```

실습 순서

1

HPA 설정

2

CPU 부하 발생 및 기
능 확인

HPA 옵션 추가

5. HPA 생성 확인

```

ubuntu@host-172-30-1-182:~/tutorial/AdvancedCourse/src/hoai$ kubectl get all
NAME                                READY    STATUS    RESTARTS   AGE
pod/my-release-kc-spring-demo-5f78677dc-fwqxc    1/1      Running   0           2m51s
pod/my-release-kc-spring-demo-5f78677dc-mj2cc    1/1      Running   0           3ms
pod/sql-job-nscct                                0/1      Completed 0           3ms

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP     10.96.0.1     <none>         443/TCP    175m
service/my-release-kc-spring-demo-service ClusterIP     10.96.19.124 <none>         80/TCP     3ms

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/my-release-kc-spring-demo 2/2      2              2            3ms

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/my-release-kc-spring-demo-5f78677dc 2          2          2        3ms

NAME                                REFERENCE                                TARGETS    MINPODS    MAXPODS    REPLICAS
horizontalpodautoscaler.autoscaling/my-release-kc-spring-demo Deployment/my-release-kc-spring-demo      <unknown>/50% 2          6          2

NAME                                COMPLETIONS    DURATION    AGE
job.batch/sql-job 1/1            4s          3ms

```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab09 link)



```
kubectl get all
```

6. Pod 확인용 새 터미널 열기

```

munseongha@munseonghai-MacBookPro ~ % cd Downloads
munseongha@munseonghai-MacBookPro Downloads % ssh -i keypair.pem ubuntu@61.109.239.58
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-104-generic x86_64)

ubuntu@host-172-30-1-182:~$ kubectl get po -w
NAME                                READY    STATUS    RESTARTS   AGE
my-release-kc-spring-demo-5f78677dc-fwqxc    1/1      Running   0           10m
my-release-kc-spring-demo-5f78677dc-mj2cc    1/1      Running   0           10m
sql-job-nscct                                0/1      Completed 0           10m

```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab09 link)



- 1 `cd {keypair를 다운 받은 곳}`
- 2 `ssh -i keypair.pem ubuntu@{bastion의 public ip 주소}`
- 3 `kubectl get po -w`

실습 순서

1

HPA 설정

2

CPU 부하 발생 및 기
능 확인

CPU 부하 발생 및 기능 확인

1. CPU 부하 발생

! 유의사항

기존에 사용하던 터미널 창에 아래 코드를 입력해 주시고, 6번에서 새로 열었던 터미널 창에서 결과를 확인해 주세요.

```
ubuntu@host-172-30-0-190:~/yaml/tutorial/AdvancedCourse/src/helm$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/load-generator	0/1	Error	0	3m16s
pod/metrics-server-79469cbf5f-rqcqm	1/1	Running	0	10m
pod/my-release-kc-spring-demo-5bf46bdb8d-f69zf	1/1	Running	0	13m
pod/my-release-kc-spring-demo-5bf46bdb8d-qvm5x	1/1	Running	0	15s
pod/my-release-kc-spring-demo-5bf46bdb8d-tfh8v	1/1	Running	0	15s
pod/my-release-kc-spring-demo-5bf46bdb8d-zgzj6	1/1	Running	0	7m31s
pod/sql-job-w996k	0/1	Completed	0	13m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	28m
service/metrics-server	ClusterIP	10.103.82.66	<none>	443/TCP	10m
service/my-release-kc-spring-demo-service	ClusterIP	10.99.187.213	<none>	80/TCP	13m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/metrics-server	1/1	1	1	10m
deployment.apps/my-release-kc-spring-demo	4/4	4	4	13m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/metrics-server-79469cbf5f	1	1	1	10m
replicaset.apps/my-release-kc-spring-demo-5bf46bdb8d	4	4	4	13m

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/my-release-kc-spring-demo	Deployment/my-release-kc-spring-demo	1%/3%	2	6	4	7m46s

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.

*Github 사이트에서도 복사 가능 ([Lab09 link](#))



```
kubectl run -i --tty load-generator --rm --image=ke-container-registry.kr-central-2.kcr.dev/ke-cr/busybox:1.28 --restart=Never -- /bin/sh -c "while sleep 0.01; do wget -q -O- http://[웹서버를 위한 Public IP]; done"
```

리소스 사용량에 맞추어 파드가 자동확장 되는 것을 확인

Lab10 :

DNS 서비스를 통해 도메인 주소 연결

이론 교재 98p

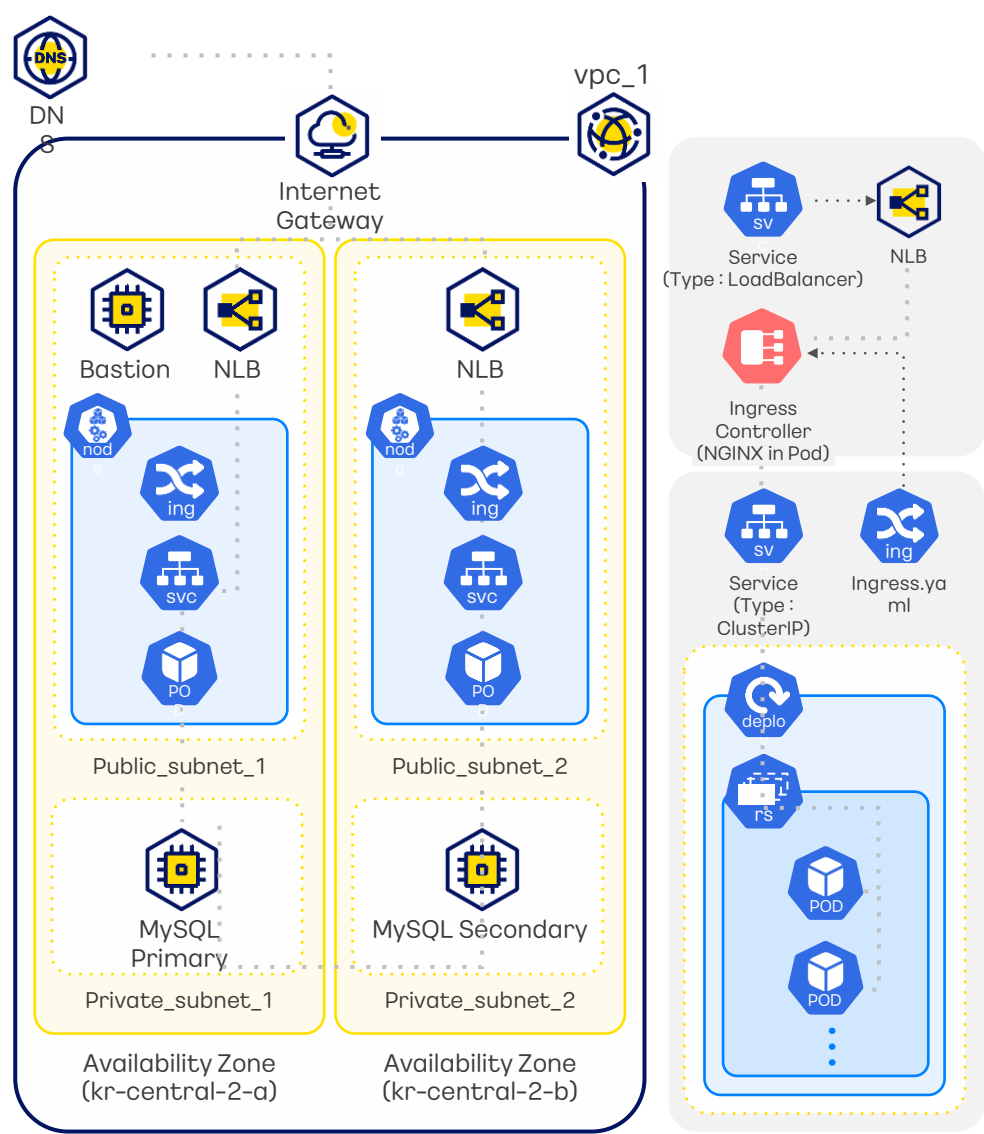


실습 내용

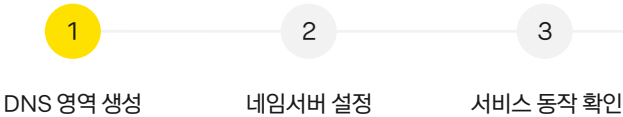
DNS 서비스를 통해 Public IP로 접속하던 웹서비스를 도메인 주소로 연결합니다. 연결한 도메인을 통해 DNS 서비스가 작동하는지 확인하는 데모를 진행합니다.

실습 순서

- 1 DNS 영역 생성
- 2 네임서버 설정
- 3 서비스 동작 확인



실습 순서



DNS 서비스 설정

DNS 영역 생성

1. 카카오클라우드 로고 옆 버튼 클릭 > Beyond Networking Service > DNS > **DNS 영역**

Beyond Networking Service

DNS ☆
손쉽게 DNS 서버를 구축할 수 있고 빠른 응답속도와 안

DNS 영역

2. [DNS 영역] 생성 클릭

DNS 영역 생성 **클릭**



DNS 영역 생성

새로운 DNS 영역을 생성하여, 레코드셋을 설정하여 쉽고 간편하게 서비스를 관리할 수 있습니다.

DNS 영역 이름
kakaocloud-edu.com

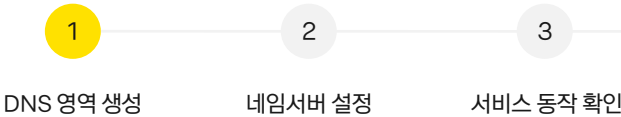
영문 소문자, 숫자, 하이픈(-), 언더바(_)만 사용 가능하며, 최대 253자까지 입력할 수 있습니다.
- 온영(,)은 시작과 끝에 위치할 수 없습니다.
- 온영(,) 사이의 글자 수는 최대 63자를 초과할 수 없습니다.
- 온영(.)을 연속해서 사용할 수 없습니다.

DNS 영역 설명 (선택)
100자 이내로 작성

생성 **클릭**

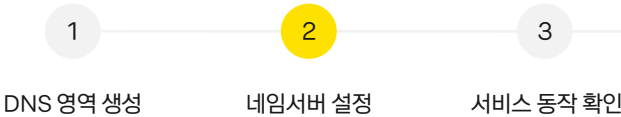
DNS 영역 이름 :
kakaocloud-edu.com
사용할 도메인과 같게 생성

실습 순서



DNS 서비스 설정
DNS 영역 생성

실습 순서



DNS 서비스 설정

DNS 영역 생성

- 또한, 사용 도메인의 네임서버를 카카오클라우드의 네임서버로 바꿔주어야함
- 도메인 구입처의 도메인 설정창에서 네임서버를 변경
- 본 실습에서는 '가비아' 라는 도메인 제공 서비스를 이용하였음

DNS > DNS 영역 > kakaocloud-edu.com

kakaocloud-edu.com

수정 삭제

Zone ID
ee4d570f-f214-4a85-a7f0-76d04bef766e

레코드 개수
3

생성일시
2025.04.25 (금) 16:12:37

네임서버
ns1.kakaocloud.com
ns2.kakaocloud.com

레코드

레코드 통계 보기 레코드 수정 레코드 삭제 레코드 생성

필터 혹은 값을 입력해 주세요.

총 3 건 중 1-3 건 이전 다음

☐	이름	유형	TTL(초)	값	생성일시	상태	
☐	kakaocloud-edu.com	A	60	61.109.239.94 210.109.55.152	2025.05.13 (화) 11:21:32	Active	⋮
☐	kakaocloud-edu.com	SOA	-	ns1.kakaocloud.com. ki...	2025.04.25 (금) 16:12:37	Active	⋮
☐	kakaocloud-edu.com	NS	-	ns2.kakaocloud.com. ns1.kakaocloud.com.	2025.04.25 (금) 16:12:37	Active	⋮

< kakaocloud-edu.com

등록 확인증

인증 코드

도메인 정보

만기일 맞춤 도메인 연장 연장 알림

도메인
kakaocloud-edu.com

등록일
2023-09-21

만기일
2024-09-21 (남은 기간: 365일)

소유자

수정

소유권 이전

관리자

수정

소유자
ahnnr...naver.com
010-7739-1149

관리자
ahnnr...naver.com
010-7739-1149

네임서버 설정

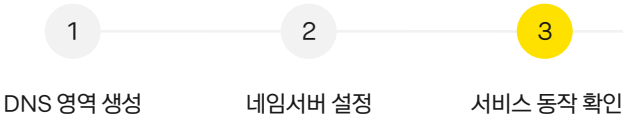
1차
2차
3차
4차
5차
6차
7차

ns2.kakaocloud.com
ns1.kakaocloud.com
데이터 없음
데이터 없음
데이터 없음
데이터 없음
데이터 없음

8차
9차
10차
11차
12차
13차

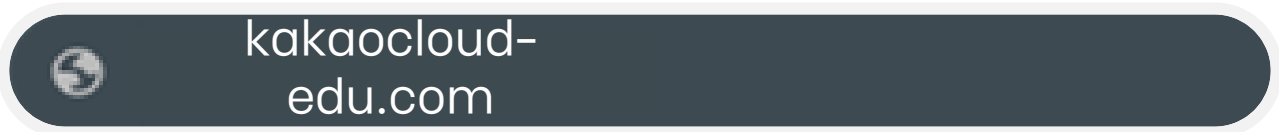
데이터 없음
데이터 없음
데이터 없음
데이터 없음
데이터 없음
데이터 없음

실습 순서

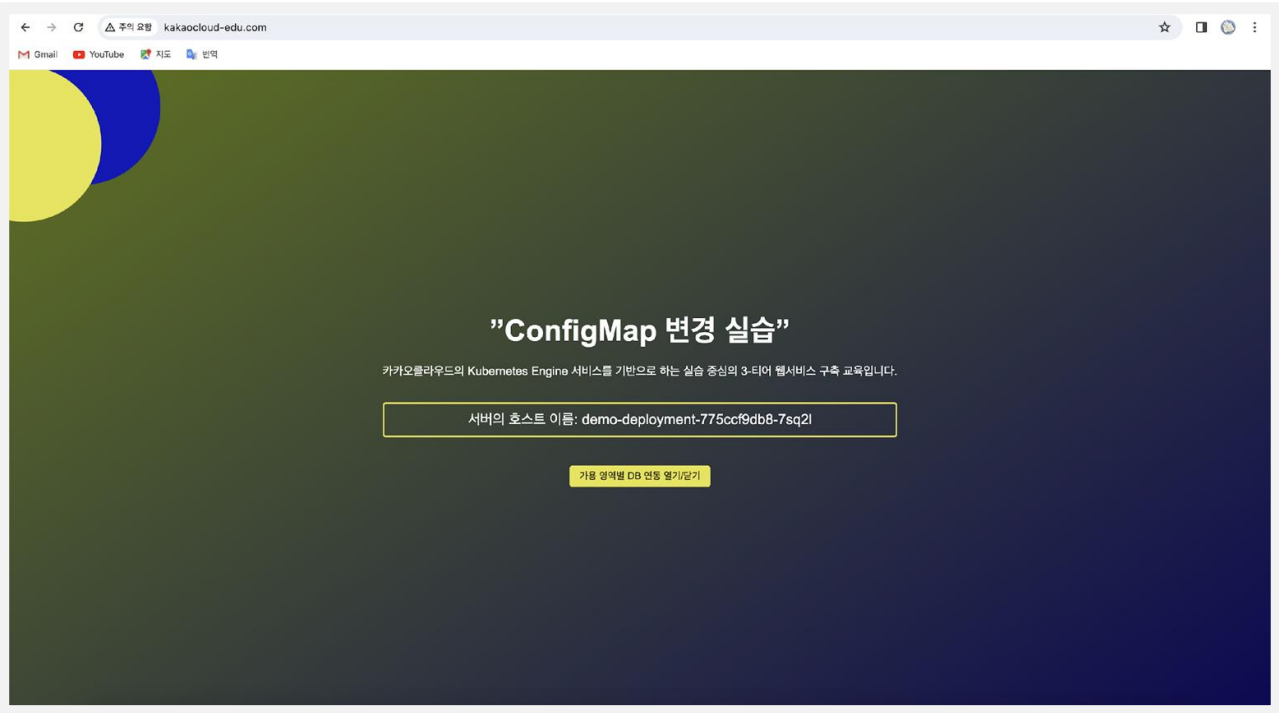


서비스 동작 확인

◦연결한 도메인을 브라우저 창에 입력



◦DNS 서비스 연결 확인



Lab11:

리소스 삭제

이론 교재 99p



실습 내용

불필요한 리소스 삭제는 비용을 절감하고 보안을 강화하며 자원을 최적화하며 확장성을 향상시키고 운영 관리를 간소화하는데 도움을 줍니다.

실습 순서

- 1 쿠버네티스 리소스 삭제
- 2 기타 클라우드 리소스 삭제

쿠버네티스 리소스 삭제

1. Helm 차트를 통해 설치한 Kubernetes 애플리케이션 삭제

```
ubuntu@host-10-0-2-135:~/tutorial/AdvancedCourse/src/helm$ helm uninstall my-release
WARNING: Kubernetes configuration file is group-readable. This is insecure. Location: /home/ubuntu/.kube/config
WARNING: Kubernetes configuration file is world-readable. This is insecure. Location: /home/ubuntu/.kube/config
release "my-release" uninstalled
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab09 link](#))



```
helm uninstall my-release
```

2. Helm 차트를 통해 설치한 Kubernetes 클러스터의 Metrics Server 삭제

```
ubuntu@host-10-0-2-135:~/tutorial/AdvancedCourse/src/helm$ helm uninstall metrics-server
release "metrics-server" uninstalled
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 ([Lab09 link](#))



```
helm uninstall metrics-server
```

실습 순서



쿠버네티스 리소스 삭제

3. Ingress-controller 삭제 (LB 자동 삭제)

```
ubuntu@host-172-30-0-225:~/tutorial/AdvancedCourse/src/helm$ kubectl delete -f https://github.com/kakaocloud-edu/tutorial/raw/main/AdvancedCourse/src/manifests/ingress-nginx-controller.yaml
namespace "ingress-nginx" deleted
serviceaccount "ingress-nginx" deleted
serviceaccount "ingress-nginx-admission" deleted
role.rbac.authorization.k8s.io "ingress-nginx" deleted
role.rbac.authorization.k8s.io "ingress-nginx-admission" deleted
clusterrole.rbac.authorization.k8s.io "ingress-nginx" deleted
clusterrole.rbac.authorization.k8s.io "ingress-nginx-admission" deleted
rolebinding.rbac.authorization.k8s.io "ingress-nginx" deleted
rolebinding.rbac.authorization.k8s.io "ingress-nginx-admission" deleted
clusterrolebinding.rbac.authorization.k8s.io "ingress-nginx" deleted
clusterrolebinding.rbac.authorization.k8s.io "ingress-nginx-admission" deleted
configmap "ingress-nginx-controller" deleted
service "ingress-nginx-controller" deleted
service "ingress-nginx-controller-admission" deleted
deployment.apps "ingress-nginx-controller" deleted
job.batch "ingress-nginx-admission-create" deleted
job.batch "ingress-nginx-admission-patch" deleted
ingressclass.networking.k8s.io "nginx" deleted
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab11 link)



```
kubectl delete -f https://github.com/kakaocloud-edu/tutorial/raw/main/AdvancedCourse/src/manifests/ingress-nginx-controller.yaml
```

4. 리소스 삭제 확인

```
ubuntu@host-10-0-2-135:~/tutorial/AdvancedCourse/src/helm$ kubectl get all
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/kubernetes                  ClusterIP           10.96.0.1        <none>            443/TCP          4d9h
```

아래의 스크립트를 복사 후 터미널에 붙여 넣습니다.
*Github 사이트에서도 복사 가능 (Lab09 link)



```
kubectl get all --all-namespaces
```


실습 순서

1

쿠버네티스
리소스 삭제

2

기타 클라우드
리소스 삭제

클라우드 리소스 삭제

1. Virtual Machine ▶ 인스턴스 ▶ bastion VM 선택 ▶ 인스턴스 삭제 ▶ 이름 입력 ▶ 삭제 버튼 클릭

2. DNS ▶ DNS 영역 ▶ DNS 이름 ▶ 추가했던 상단 레코드 오른쪽(...) 클릭 ▶ 레코드 삭제 클릭 ▶ 레코드 이름 입력 ▶ 삭제 버튼 클릭 ▶ DNS 영역 ▶ 생성한 DNS 영역 오른쪽(...) 클릭 ▶ 삭제 클릭 ▶ DNS 영역 이름 입력 ▶ 삭제 버튼 클릭

3. Container Registry ▶ 리포지토리 ▶ 생성되어 있는 리포지토리 클릭 ▶ 상단 오른쪽 삭제 클릭 ▶ 이름 입력 ▶ 삭제 버튼 클릭

4. MySQL ▶ 인스턴스 그룹 ▶ 생성되어 있는 인스턴스 그룹 오른쪽(...) 클릭 ▶ 삭제 클릭 ▶ 인스턴스 그룹 이름 입력 ▶ 삭제 버튼 클릭

5. Kubernetes Engine Cluster ▶ 클러스터 ▶ 생성되어 있는 클러스터 오른쪽(...) 클릭 ▶ 클러스터 삭제 클릭 ▶ 이름 입력 ▶ 삭제 버튼 클릭

6. VPC ▶ 퍼블릭 IP ▶ 모두 선택 ▶ 좌측 하단 삭제 버튼 클릭 ▶ 영구 삭제 입력 ▶ 삭제 버튼 클릭

7. VPC ▶ 생성되어 있는 VPC 오른쪽(...) 클릭 ▶ VPC 삭제 클릭 ▶ VPC 이름 입력 ▶ 삭제 버튼 클릭

8. 계정 설정 ▶ 자격 증명 ▶ IAM 액세스 키 탭 클릭 ▶ 생성되어 있는 IAM 액세스 키 오른쪽(...) 클릭 ▶ 삭제 클릭 ▶ IAM 액세스 키 삭제 입력 및 삭제 버튼 클릭

9. Virtual Machine ▶ 키 페어 ▶ 생성되어 키 페어 오른쪽(...) 클릭 ▶ 키 페어 삭제 클릭 ▶ 키 페어 이름 입력 ▶ 삭제 버튼 클릭

10. VPC ▶ 보안 그룹 ▶ 생성되어 있는 보안 그룹 오른쪽(...) 클릭 ▶ 보안 그룹 삭제 클릭 ▶ 보안그룹 이름 입력 ▶ 삭제 버튼 클릭

kakaoenterprise

(주)카카오엔터프라이즈

경기도 성남시 분당구 판교역로 235 에이치스퀘어 N동 7층

7F, 235, Pangyoyeok-ro, Bundang-gu, Seongnam-si,
Gyeonggi-do, 13494, Republic of Korea

KakaoCloud Education Center

<https://edu.kakaocloud.com>



이 문서의 내용과 디자인은 저작권의 보호대상이 되고 부정경쟁방지 및 영업비밀보호에 관한 법률 등 관련 법령에 따라 보호되는 영업비밀 또는 기밀정보를 포함할 수 있습니다. 본 문서에 포함된 정보의 무단 배포, 복사 및 사용은 엄격히 금지됩니다.

The content and design of the document are subject to copyright and may contain trade secrets, privileged and confidential information otherwise protected by applicable law, including the Unfair Competition Prevention and Trade Secret Protection Act. Any unauthorized dissemination, distribution, copying, or use of the information contained in this document is strictly prohibited.

kakaoenterprise